THE COMPLETE

AI / ML Interview Mastery Guide

From arithmetic to architectures: everything you need to pass an international-level AI/ML interview

Mathematics · Classical ML · The Confusion-Matrix Chapter
ANN · CNN · RNN · LSTM · GRU · Transformers · LLMs
GANs · VAEs · Diffusion · RL · Time Series · Recommenders · GNNs
16 System Designs (ML + GenAI) · 44 Scenario Questions
A Question for Every Concept · 42 Portfolio Projects
From-Scratch Code · Frontier-Lab Prep

Companion volume to
The Beginner's Guide to Building AI Applications
and The Production & MLOps Companion

The Complete AI / ML Interview Mastery Guide

From arithmetic to architectures: everything you need to pass an international-level AI/ML interview

Companion volume to *The Beginner's Guide to Building AI Applications* (LangChain · LangGraph · RAG · LlamaIndex · Vector DBs) **and** *The Production & MLOps Companion*.

That first guide taught you how to *build with* models. This one teaches you what the models *are*: the mathematics, the classical algorithms, the metrics, every major neural architecture (ANN, CNN, RNN, LSTM, GRU, Transformer, GAN, VAE, Diffusion), how to implement each one, how to debug them, and how to talk about all of it under pressure.

Concepts · Mathematics · Diagrams · Implementations · Scenario questions · Coding problems · 300+ interview questions

---

How to Use This Guide

Who this is for

You can write Python. You may have built an LLM app. You have never been asked "derive the gradient of the logistic loss" or "your model has 99% accuracy and is useless, why" in a room with three interviewers.

This guide assumes no machine learning background. Every concept is built from the ground up, in an order where nothing depends on something explained later.

The three-pass method

Pass 1 (understanding). Read the concepts. Skip the code. Aim to be able to explain each idea to a non-technical friend in three sentences.

Pass 2 (implementation). Type out every code block. Do not copy-paste. Run them. Break them deliberately: remove the regularization, unbalance the classes, set the learning rate to 10.

Pass 3 (articulation). Answer the interview questions out loud, without notes, recorded. The gap between knowing something and saying it fluently under stress is enormous, and closing it is the entire point of preparation.

The callout boxes

Same four as the first guide, plus two new ones:

> **ANALOGY** A hard idea compared to something you already understand. Read these when a concept isn't landing.
>
> **KEY IDEA** The single most important sentence in a section. If you remember nothing else, remember these.
>
> **COMMON MISTAKE** Something beginners get wrong, or a trap in the library. These save you hours of debugging and points in interviews.
>
> **INTERVIEW QUESTION** Q: a question you will actually be asked, with a strong answer. Practise saying these aloud.
>
> **THE MATH** The formula, stated precisely, with every symbol defined. You can skip these on pass 1. You cannot skip them before a senior interview.
>
> **SCENARIO** A realistic production situation and how to reason through it. These are what distinguishes a strong candidate from a memoriser.

Environment setup

```
python -m venv venv
source venv/bin/activate          # macOS / Linux
venv\Scripts\activate             # Windows

# Core scientific stack
pip install numpy pandas scipy matplotlib seaborn

# Classical ML
pip install scikit-learn xgboost lightgbm catboost imbalanced-learn statsmodels

# Deep learning
pip install torch torchvision torchaudio
pip install tensorflow

# NLP and transformers
pip install transformers datasets tokenizers sentence-transformers

# Interpretability, tuning, tracking
pip install shap lime optuna mlflow

# Time series
pip install prophet statsforecast
```

Everything runs on CPU except where noted. For the deep learning parts, Google Colab gives you a free GPU: that is the correct place to run anything that trains for more than a minute.

The single most important framing

> **KEY IDEA** Interviewers are not testing whether you memorised definitions. They can read Wikipedia too. They are testing three things: (1) can you explain a concept simply, which proves you actually understand it; (2) do you know the **trade-offs**, which proves you have made real choices; (3) can you **debug**, which proves you have shipped something. Every section of this guide is written to give you all three, not just the first.

Table of Contents

Part I — Mathematical Foundations

You do not need a mathematics degree. You need fluency in about a dozen ideas, because every model in this guide is built from them and interviewers probe them constantly. Read this part once now, and re-read it after Part VIII, when you will suddenly see why it mattered.

1.1 Linear Algebra

The four objects

Object	Shape	Notation	Example
Scalar	single number	`x`	learning rate 0.001
Vector	1D array	`v` (n,)	one data point's features, or one embedding
Matrix	2D array	`A` (m, n)	a whole dataset: m rows, n features
Tensor	3D+ array	`T`	a batch of RGB images: (batch, height, width, channels)

```
import numpy as np

scalar = 5.0
vector = np.array([1, 2, 3])                # shape (3,)
matrix = np.array([[1, 2, 3], [4, 5, 6]])   # shape (2, 3)
tensor = np.random.rand(32, 28, 28, 3)      # shape (32, 28, 28, 3)

print(vector.shape, matrix.shape, tensor.shape)
```

COMMON MISTAKE Not checking `.shape` constantly. Roughly 70% of all deep learning bugs are shape mismatches. Print shapes at every step until it becomes reflex.

Matrix multiplication: the operation that runs the world

For `A` of shape (m, n) and `B` of shape (n, p), the product `A @ B` has shape (m, p). The inner dimensions must match and they vanish.

```
(m, n) @ (n, p)  ->  (m, p)
      ^^^^^^ these must be equal
```

Element (i, j) of the result is the dot product of row i of A with column j of B.

```
A = np.array([[1, 2], [3, 4]])       # (2, 2)
B = np.array([[5, 6], [7, 8]])       # (2, 2)
print(A @ B)
# [[19 22]
#  [43 50]]
# 19 = 1*5 + 2*7
```

KEY IDEA A neural network layer *is* a matrix multiplication: `output = input @ W + b`. A deep network is a chain of them with a non-linear function squeezed between each pair. GPUs exist in ML because they multiply large matrices in parallel extremely fast. That single sentence explains both why deep learning works and why it needed 2010s hardware to become practical.

Dot product and its geometric meaning

THE MATH For vectors **a** and **b**: $a \cdot b = \Sigma\, a_i b_i = \|a\|\ \|b\| \cos(\theta)$, where ‖a‖ is the Euclidean length (L2 norm) of **a** and θ is the angle between them.

Consequences you will be asked about: - $a \cdot b = 0$ means the vectors are **orthogonal** (perpendicular, unrelated). - Dividing out the magnitudes gives **cosine similarity**: $\cos(\theta) = (a \cdot b) / (\|a\|\ \|b\|)$, which ranges from -1 to 1. This is the standard similarity measure for text embeddings, because it ignores document length and only compares direction.

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

Norms

Norm	Formula	Also called	Used for
L0	count of non-zero entries	sparsity	feature selection (not a true norm)
L1	$\Sigma \lvert x_i \rvert$	Manhattan / taxicab	Lasso regularisation, produces sparsity
L2	$\sqrt{\Sigma x_i^2}$	Euclidean	Ridge regularisation, weight decay, distances
L∞	$\max \lvert x_i \rvert$	max norm	gradient clipping, adversarial robustness

> **KEY IDEA** L1 vs L2 is one of the most reliably asked questions in the entire field. L1's contour is a diamond with sharp corners on the axes, so the optimum often lands exactly on an axis, meaning a coefficient becomes exactly zero. L2's contour is a circle with no corners, so coefficients shrink toward zero but rarely reach it. **L1 selects features; L2 shrinks them.**

Matrix properties worth knowing by name

- **Transpose** A^T: flip rows and columns.
- **Identity** I: ones on the diagonal, zeros elsewhere. $A @ I = A$.
- **Inverse** A^{-1}: satisfies $A @ A^{-1} = I$. Only exists for square, non-singular matrices. In practice you almost never compute an inverse numerically: you solve the linear system instead (`np.linalg.solve`), which is faster and far more numerically stable.
- **Rank**: the number of linearly independent rows/columns. Low rank means redundant information, which is exactly what dimensionality reduction exploits.
- **Determinant**: a scalar measuring how much the matrix scales volume. Zero determinant means the matrix is singular (not invertible) and collapses space onto a lower dimension.
- **Trace**: sum of diagonal entries.
- **Symmetric**: $A = A^T$. Covariance matrices are always symmetric.
- **Positive semi-definite**: $x^TAx \geq 0$ for all x. Covariance matrices and valid kernel matrices are always PSD.

Eigenvalues and eigenvectors

THE MATH For a square matrix A, a vector **v** is an eigenvector with eigenvalue λ if $A v = \lambda v$. Multiplying by A does not rotate **v**; it only stretches it by λ.

Why this matters in ML: PCA finds the eigenvectors of the covariance matrix. Each eigenvector is a direction in feature space, and its eigenvalue is how much variance the data has along that direction. Keeping the top k eigenvectors keeps the most information in the fewest dimensions.

```
cov = np.cov(np.random.randn(100, 3).T)
eigvals, eigvecs = np.linalg.eig(cov)
order = np.argsort(eigvals)[::-1]      # descending
print("explained variance ratio:", eigvals[order] / eigvals.sum())
```

Singular Value Decomposition (SVD)

Any matrix (not just square) factorises as $A = U \Sigma V^T$, where U and V are orthogonal and Σ is diagonal with non-negative entries called singular values. Truncating to the top k singular values gives the best possible rank-k approximation of A (the Eckart-Young theorem). This underlies PCA, latent semantic analysis, and matrix-factorisation recommenders.

> **INTERVIEW QUESTION Q: Why does PCA use eigenvectors of the covariance matrix?** Because the covariance matrix encodes how features vary together, and its eigenvectors are the orthogonal directions along which the data varies independently. The eigenvalue attached to each is the variance captured in that direction. Sorting eigenvectors by eigenvalue and keeping the top k therefore keeps the maximum possible variance for a given number of dimensions, which is exactly what "compress with least information loss" means. In practice libraries compute it via SVD on the centred data matrix rather than forming the covariance matrix explicitly, because that is more numerically stable.

1.2 Calculus and Gradients

Derivative: the sensitivity number

The derivative df/dx answers: if I nudge x up slightly, how much does f change, and in which direction? That is all training ever asks.

Partial derivatives and the gradient

A model has millions of parameters, so we take the derivative with respect to each one while holding the others fixed. Collect them all into a vector and you have the **gradient**, written ∇f.

THE MATH $\nabla f = [\partial f/\partial w_1, \partial f/\partial w_2, \ldots, \partial f/\partial w_n]$

The gradient points in the direction of **steepest increase** of f. To minimise loss we therefore step in the **negative** gradient direction. That single sentence is gradient descent.

The chain rule: the engine of backpropagation

If `y = f(g(x))`, then `dy/dx = f'(g(x)) · g'(x)`.

A neural network is a deeply nested composition of functions. Backpropagation is nothing more than the chain rule applied mechanically, right to left, through that composition. Every framework's autograd is an implementation of this.

```
import torch

w = torch.tensor([2.0], requires_grad=True)
x = torch.tensor([3.0])

y = w * x          # y = 6
loss = y ** 2      # loss = 36
loss.backward()

print(w.grad)      # tensor([36.])
# By hand: loss = (wx)^2 = 9w^2, d/dw = 18w = 36 at w=2. Correct.
```

Gradient descent, in three variants

Variant	Updates per epoch	Pros	Cons
Batch GD	1 (uses all data)	stable, smooth convergence	slow, needs all data in memory
Stochastic GD (SGD)	N (one sample each)	fast updates, noise can escape local minima	very noisy, unstable
Mini-batch GD	N/B	best of both, GPU-efficient	one more hyperparameter (batch size)

In practice "SGD" in every library means mini-batch. Typical batch sizes: 32, 64, 128, 256.

Jacobian, Hessian, and second-order methods

- **Jacobian**: the matrix of all first partial derivatives of a vector-valued function. Shape (outputs, inputs).
- **Hessian**: the matrix of all second partial derivatives. It describes curvature.
- **Newton's method** uses the Hessian to take smarter steps. It is rarely used in deep learning because the Hessian is (parameters × parameters), which for a billion-parameter model is unimaginably large. Adam and friends are cheap approximations that capture some curvature information without ever forming the Hessian.

> **INTERVIEW QUESTION Q: Why do we minimise the loss rather than maximise accuracy directly?** Because accuracy is a step function of the parameters: it is piecewise constant, so its gradient is zero almost everywhere and undefined at the jumps. There is no direction to descend in. We instead minimise a smooth, differentiable **surrogate** loss (cross-entropy, MSE) that is a good proxy for the metric we care about. This mismatch is also why a model can improve its loss while its accuracy stays flat, which confuses people early on.

1.3 Probability

The vocabulary

- **Random variable**: a quantity whose value is uncertain. Discrete (a die) or continuous (a height).
- **PMF / PDF**: probability mass function (discrete) or probability density function (continuous).
- **CDF**: `F(x) = P(X ≤ x)`.
- **Expectation** `E[X]`: the long-run average. `E[X] = Σ x·P(x)`.
- **Variance** `Var(X) = E[(X - E[X])²]`: average squared spread.
- **Covariance** `Cov(X,Y) = E[(X-E[X])(Y-E[Y])]`: how two variables move together.
- **Correlation**: covariance normalised to [-1, 1]. Unitless, comparable across pairs.

Conditional probability and independence

`P(A|B) = P(A ∩ B) / P(B)`

A and B are independent if `P(A|B) = P(A)`, equivalently `P(A ∩ B) = P(A)P(B)`.

Bayes' Theorem

THE MATH `P(A|B) = P(B|A) · P(A) / P(B)`

Named: **posterior** = (**likelihood** × **prior**) / **evidence**.

This one formula underlies Naive Bayes, Bayesian optimisation for hyperparameter tuning, Bayesian neural networks, and A/B testing done properly.

INTERVIEW QUESTION (a classic, asked verbally with no calculator) Q: A disease affects 1 in 1,000 people. A test is 99% accurate in both directions. You test positive. What is the probability you have the disease? Take 100,000 people. 100 have the disease, and 99 of those test positive. 99,900 do not have it, and 1% of them, which is 999, test positive anyway. So 99 + 999 = 1,098 positives, of which 99 are true. 99/1098 ≈ **9%**. The lesson is that when the base rate is very low, even an accurate test produces mostly false positives. This is exactly why precision collapses on imbalanced classification problems, and it is the single best intuition to bring to the metrics chapter.

The distributions you must recognise

Distribution	Type	Describes	Shows up in
Bernoulli	discrete	one yes/no trial	binary classification labels
Binomial	discrete	k successes in n trials	A/B test conversions
Multinomial	discrete	counts across k categories	bag-of-words, softmax outputs
Poisson	discrete	events per fixed interval	arrivals, click counts
Uniform	either	all outcomes equally likely	random initialisation, sampling
Normal (Gaussian)	continuous	bell curve, μ and σ	errors, weight init, everything via CLT
Exponential	continuous	waiting time between events	survival analysis, churn
Beta	continuous	a probability itself (0 to 1)	Bayesian priors, Thompson sampling
Gamma	continuous	positive skewed quantities	priors on variance

Central Limit Theorem

The mean of a large enough sample of independent, identically distributed variables is approximately normally distributed, **regardless of the underlying distribution**. This is why the normal distribution is everywhere, and why standard errors and confidence intervals work at all.

Maximum Likelihood Estimation (MLE)

Choose the parameters that make the observed data most probable.

THE MATH `θ = argmax_θ Π P(xᵢ | θ)`, and because products of small numbers underflow, we always maximise the **log**-likelihood instead: `θ = argmax_θ Σ log P(xᵢ | θ)`.

KEY IDEA Almost every loss function you will meet is a negative log-likelihood in disguise. Minimising **mean squared error** is MLE under an assumption of Gaussian noise. Minimising **cross-entropy** is MLE under an assumption of a Bernoulli or categorical distribution. Saying that sentence in an interview signals real depth, because it shows you know where the loss functions came from rather than just which one to import.

MAP (maximum a posteriori) adds a prior: `argmax P(data|θ)P(θ)`. Adding an L2 penalty to a loss is exactly MAP estimation with a Gaussian prior on the weights; adding an L1 penalty corresponds to a Laplace prior.

1.4 Statistics and Hypothesis Testing

Descriptive statistics

- **Mean**: sensitive to outliers. **Median**: robust. **Mode**: most frequent.
- **Standard deviation**: spread in the original units.
- **Skewness**: asymmetry. Right/positive skew has a long right tail (income, for example).
- **Kurtosis**: heaviness of tails, which is how prone the variable is to extreme values.
- **Quantiles / IQR**: `IQR = Q3 - Q1`. The classic outlier rule is anything outside `Q1 - 1.5·IQR` to `Q3 + 1.5·IQR`.

Hypothesis testing

1. State a null hypothesis H_0 (usually "no effect") and an alternative H_1.
2. Choose a significance level α, typically 0.05.
3. Compute a test statistic and its p-value.
4. If $p < \alpha$, reject H_0.

The p-value is the probability of observing data at least this extreme *if the null hypothesis were true.* It is emphatically **not** the probability that the null hypothesis is true. Interviewers ask this precisely because so many people get it wrong.

Error	Meaning	Also called
Type I	rejecting a true null (false positive)	α
Type II	failing to reject a false null (false negative)	β
Power	probability of correctly rejecting a false null	1 - β

> **KEY IDEA** Type I / Type II error maps exactly onto the confusion matrix in Part IV. A false positive is a Type I error; a false negative is a Type II error. If you understand one, you understand the other, and pointing out the connection in an interview is a nice touch.

Common tests

Test	Use when
t-test (one-sample, two-sample, paired)	comparing means, small samples, unknown variance
z-test	comparing means, large samples, known variance
Chi-square	independence between two categorical variables; goodness of fit
ANOVA	comparing means across 3+ groups
Mann-Whitney U	non-parametric alternative to the t-test
Kolmogorov-Smirnov	do two samples come from the same distribution (used for drift detection)

Confidence intervals

A 95% CI means: if we repeated the sampling procedure many times, 95% of the intervals constructed this way would contain the true parameter. It does **not** mean there is a 95% probability the parameter is in this particular interval (that is the Bayesian credible interval, which is a different object).

Multiple comparisons

Run 20 tests at α=0.05 and you expect one false positive by chance alone. Corrections: **Bonferroni** (divide α by the number of tests, conservative) or **Benjamini-Hochberg** (controls false discovery rate, less conservative and usually preferred). This matters when you evaluate many model variants or many features.

Simpson's Paradox

A trend that appears in several groups can reverse when the groups are combined. The classic case is a treatment that looks better in every subgroup but worse overall, because of an unbalanced confounder. Always segment your metrics.

1.5 Information Theory

Entropy

THE MATH `H(X) = -Σ p(x) log p(x)`

Entropy measures uncertainty, in bits if the log is base 2. A fair coin has 1 bit of entropy. A two-headed coin has 0. Decision trees use entropy to choose splits: the split that most reduces entropy is the most informative.

Cross-entropy

THE MATH `H(p, q) = -Σ p(x) log q(x)`

The average number of bits needed to encode data from the true distribution p using a code optimised for the predicted distribution q. Minimising cross-entropy pushes q toward p. This is *the* classification loss.

KL divergence

THE MATH `KL(p || q) = Σ p(x) log(p(x)/q(x)) = H(p,q) - H(p)`

The extra cost of using q when the truth is p. It is always ≥ 0, and it is **asymmetric**: `KL(p||q) ≠ KL(q||p)`. That asymmetry matters in VAEs and in RLHF, where a KL penalty keeps the fine-tuned policy near the reference model.

Because H(p) is fixed by the data, minimising cross-entropy and minimising KL divergence are the same optimisation problem. That is a nice one-liner to have ready.

Mutual information

`I(X;Y) = H(X) - H(X|Y)`: how much knowing Y reduces uncertainty about X. Used for feature selection, and it captures non-linear dependence that correlation misses.

1.6 Optimisation

Convex vs non-convex

A convex function has exactly one minimum, so gradient descent is guaranteed to find the global optimum. Linear regression, logistic regression, and SVMs with convex losses are all convex problems.

Neural networks are **non-convex**: the loss surface has many local minima and vastly more saddle points. In practice this matters far less than theory feared, because in very high dimensions most critical points are saddles rather than bad local minima, and the many local minima that exist tend to have similar loss values.

The constrained optimisation vocabulary

- **Lagrange multipliers**: turn a constrained problem into an unconstrained one. This is how SVMs are derived.
- **KKT conditions**: the generalisation to inequality constraints. Mentioning them when discussing SVM support vectors is a strong signal.
- **Duality**: solving the dual problem instead of the primal. The SVM dual is what makes the kernel trick possible.

CHECKPOINT — Part I self-test 1. Why does L1 produce sparsity and L2 not? 2. What does the gradient point toward, and why do we subtract it? 3. State Bayes' theorem and name each term. 4. What exactly is a p-value? 5. Why is minimising cross-entropy the same as minimising KL divergence? 6. What is the shape of `(64, 128) @ (128, 10)` ? 7. Why is MSE the MLE estimator under Gaussian noise?

If any of these are shaky, re-read the relevant section before continuing. Everything after this point assumes them.

Part II — What Machine Learning Actually Is

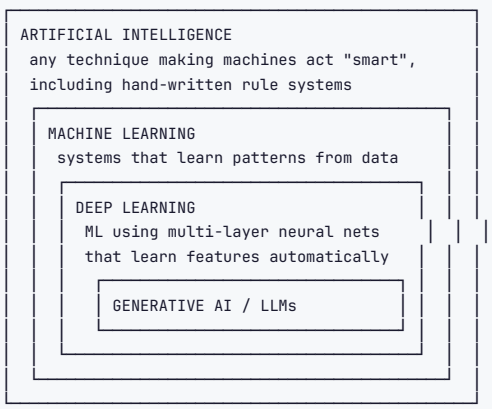
2.1 The definition

> **KEY IDEA** Traditional programming: you write the rules, feed in data, and get answers. Machine learning: you feed in data **and** answers, and the computer writes the rules. That inversion is the whole field.

Tom Mitchell's formal definition, which is worth being able to recite: *a computer program is said to learn from experience E with respect to some task T and performance measure P, if its performance at T, as measured by P, improves with E.*

In an interview, always attach the three: what is the task, what is the data, what is the metric. Candidates who answer "we used a random forest" without naming the metric sound junior.

AI vs ML vs DL vs Data Science



Data science overlaps but is not a subset: it includes analysis, experimentation, visualisation, and communication, much of which involves no model at all.

> **INTERVIEW QUESTION Q: When would you NOT use machine learning?** When a rule covers the case deterministically (tax brackets, validation logic), because a rule is cheaper, testable, and explainable. When you have too little data. When errors are catastrophic and unexplainable decisions are unacceptable without a compliance path. When the relationship is genuinely random and there is no signal to learn. And when a simple heuristic gets you 90% of the value at 5% of the cost, which is more often than people admit. Being willing to say "I would not use ML here" is a maturity signal, not a weakness.

2.2 The learning paradigms

Supervised learning

You have inputs X and correct labels y. The model learns the mapping $X \rightarrow y$.

- **Classification**: the label is a category. Binary (spam / not spam), multi-class (digit 0-9), multi-label (an article can be tagged both "politics" and "economics").
- **Regression**: the label is a continuous number (house price, temperature).

Unsupervised learning

You have X only and no labels. The model finds structure.

- **Clustering**: group similar points (K-Means, DBSCAN).
- **Dimensionality reduction**: compress features while retaining information (PCA, UMAP).
- **Density estimation / anomaly detection**: learn what normal looks like and flag deviations.
- **Association rule mining**: "people who buy X also buy Y".

Semi-supervised learning

A small labelled set plus a large unlabelled set. Common in the real world, where labels are expensive. Techniques: self-training (pseudo-labelling), consistency regularisation, graph-based label propagation.

Self-supervised learning

The labels are generated *from the data itself*, so no human annotation is needed. This is how every modern foundation model is trained.

- Masked language modelling (BERT): hide 15% of tokens and predict them.
- Next-token prediction (GPT): predict the following token.
- Contrastive learning (SimCLR, CLIP): make two augmented views of the same image agree and different images disagree.

> **KEY IDEA** Self-supervised learning is the single biggest reason the last decade happened. It converted the internet's unlabelled text and images from useless into training data. If asked "what changed to make LLMs possible," the three-part answer is: the transformer architecture, self-supervised pretraining at scale, and GPU compute.

Reinforcement learning

An agent takes actions in an environment and receives rewards. It learns a policy that maximises cumulative reward. No labelled examples: only a delayed, sparse signal. Covered in Part XIV.

The comparison table

Paradigm	Data needed	Output	Example algorithms
Supervised	X + y	prediction of y	Linear/logistic regression, trees, SVM, most NNs
Unsupervised	X only	structure	K-Means, PCA, DBSCAN, autoencoders
Semi-supervised	X + a little y	prediction of y	pseudo-labelling, label spreading
Self-supervised	X only (labels derived)	representations	BERT, GPT, SimCLR, CLIP
Reinforcement	environment + reward	policy	Q-learning, DQN, PPO

2.3 Parametric vs non-parametric

	Parametric	Non-parametric
Definition	fixed number of parameters, independent of dataset size	number of effective parameters grows with data
Examples	linear/logistic regression, neural nets (fixed architecture), Naive Bayes	k-NN, decision trees, SVM with RBF kernel, Gaussian processes
Training	learn coefficients	often store or partition the data
Assumptions	strong (a specific functional form)	weak
Data hunger	works with less data	needs more data
Inference cost	cheap and constant	can grow with dataset size (k-NN is the extreme case)

> **COMMON MISTAKE** Thinking "non-parametric" means "no parameters". It means the *number* of parameters is not fixed in advance. A decision tree grown on a million rows has vastly more structure than one grown on a hundred.

2.4 Discriminative vs generative

- **Discriminative** models learn `P(y | x)` directly: the decision boundary. Logistic regression, SVM, most neural classifiers. Usually more accurate for pure classification.
- **Generative** models learn `P(x, y)` or `P(x | y)` and use Bayes' rule to classify. Naive Bayes, GMMs, VAEs, GANs, diffusion, and autoregressive LLMs. They can generate new samples, handle missing features more gracefully, and often need less data, but tend to be less accurate at classification when data is plentiful.

> **INTERVIEW QUESTION Q: Is an LLM discriminative or generative?** Generative. It models `P(next token | previous tokens)`, and by chaining that it defines a distribution over whole sequences, which is what lets it sample new text. Note that "generative" here is the statistical term, not the marketing one: a GAN is generative, and so is Naive Bayes, which produces no impressive output at all.

2.5 The end-to-end ML workflow

```
 1. PROBLEM FRAMING
    what decision does this improve? what is the metric?
    what is the baseline? is ML even needed?
              │
              ▼
 2. DATA COLLECTION
    sources, volume, labels, licensing, privacy
              │
              ▼
 3. EDA (exploratory data analysis)
    distributions, missingness, correlations, leakage hunting
              │
              ▼
 4. PREPROCESSING & FEATURE ENGINEERING
    cleaning, encoding, scaling, feature creation
              │
              ▼
 5. SPLIT
    train / validation / test  (or time-based split)
              │
              ▼
 6. BASELINE MODEL
    always. a majority-class predictor or a linear model.
              │
              ▼
 7. MODEL SELECTION & TUNING  ◄──┐
    cross-validation, hyperparameter search      │
              │                  │
              ▼                  │
 8. EVALUATION on validation ────┘  iterate here
              │
              ▼
 9. FINAL EVALUATION on the test set   (ONCE. only once.)
              │
              ▼
10. DEPLOYMENT   (see the MLOps companion)
              │
              ▼
11. MONITORING  → drift detected → back to 2
```

KEY IDEA Steps 1 through 4 consume roughly 80% of a real project's time and produce roughly 80% of its value. Model choice, which is what beginners obsess over, is usually the least important decision. Interviewers know this, which is why "how would you approach this problem" answers that start with "I'd use XGBoost" score badly and answers that start with "first I'd clarify the metric and look at the label distribution" score well.

COMMON MISTAKE — the one that ends interviews Touching the test set more than once. Every time you look at test performance and then change something, you leak information from it into your decisions, and your final number becomes optimistic. The test set is opened once, at the end, to produce the number you report. Everything else happens on validation folds.

Part III — Data: The Part That Decides Everything

3.1 Data types and how to encode them

Type	Sub-type	Examples	Encoding
Numerical	continuous	price, temperature	scale it
Numerical	discrete	number of rooms	often fine as-is
Categorical	nominal (no order)	city, colour	one-hot, target, embedding
Categorical	ordinal (ordered)	small/medium/large	ordinal integers
Temporal		timestamps	decompose into parts, cyclical encoding
Text		reviews	TF-IDF or embeddings
Image / audio		pixels, waveforms	normalise, feed to a CNN or transformer

Encoding categoricals

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

df = pd.DataFrame({"city": ["Lahore", "London", "Tokyo", "Lahore"],
                   "size": ["small", "large", "medium", "large"]})

# 1. ONE-HOT: one binary column per category. No false ordering implied.
pd.get_dummies(df["city"], prefix="city")
#    city_Lahore  city_London  city_Tokyo

# 2. ORDINAL: only when the categories genuinely have an order
order = [["small", "medium", "large"]]
OrdinalEncoder(categories=order).fit_transform(df[["size"]])
# [[0.], [2.], [1.], [2.]]

# 3. LABEL ENCODING: for the TARGET variable only
from sklearn.preprocessing import LabelEncoder
LabelEncoder().fit_transform(["cat", "dog", "cat"])   # [0, 1, 0]
```

Technique	Good for	Danger
One-hot	low cardinality nominal	explodes dimensionality when cardinality is high
Ordinal	genuinely ordered categories	invents a false ordering if misused
Target / mean encoding	high cardinality (zip codes, user IDs)	**leaks the target** unless computed inside CV folds with smoothing
Frequency encoding	high cardinality	collides categories with equal counts
Hashing	very high cardinality, streaming	hash collisions, no inverse mapping
Learned embeddings	very high cardinality, in a neural net	needs enough data to learn

COMMON MISTAKE Using `LabelEncoder` on an input feature. It turns Lahore=0, London=1, Tokyo=2, and a linear model or distance-based model then believes Tokyo is "twice" London and that Lahore and London are closer than Lahore and Tokyo. That is nonsense. Trees can partially survive it; linear models, k-NN, and neural nets cannot. Use one-hot for nominal inputs.

COMMON MISTAKE — target encoding leakage Computing the mean target per category on the full dataset and then splitting. Each row's encoded value then contains its own label. Validation scores look brilliant and production collapses. Target encoding must be fitted inside each CV fold, ideally with smoothing toward the global mean for rare categories.

Cyclical encoding for time

Hour 23 and hour 0 are adjacent, but numerically they are 23 apart. Fix this with sine and cosine:

```
import numpy as np
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
```

3.2 Missing values

First, diagnose **why** the data is missing, because the mechanism determines the valid fix.

Mechanism	Meaning	Example	Safe approach
MCAR (completely at random)	missingness unrelated to anything	a sensor randomly dropped a reading	dropping rows is unbiased
MAR (at random)	missingness depends on *observed* features	income missing more often for younger users, and age is recorded	impute conditional on observed features
MNAR (not at random)	missingness depends on the *unobserved value itself*	high earners refuse to state income	hardest case: model the missingness, add an indicator flag

```python
# Strategies
df.dropna()                                  # only when missingness is tiny and MCAR
df.fillna(df.median())                       # robust to outliers; median beats mean
df.fillna(df.mode()[0])                      # categorical
df["col"].fillna("MISSING")                  # treat missingness as its own category
df["col_was_missing"] = df["col"].isna().astype(int)   # ALWAYS consider this flag

# Model-based imputation
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

KNNImputer(n_neighbors=5).fit_transform(X)      # uses similar rows
IterativeImputer(random_state=0).fit_transform(X)  # MICE: models each column from the others
```

KEY IDEA Adding a binary "was missing" indicator column is nearly free and frequently improves the model, because the *fact* of missingness is often predictive. Someone who declines to give their income is a different kind of customer.

COMMON MISTAKE Computing the imputation value on the whole dataset before splitting. The test set's median then influences the training pipeline. Fit the imputer on train, then `transform` validation and test. `sklearn.pipeline.Pipeline` exists to make this impossible to get wrong, which is why you should always use it.

Note that LightGBM, XGBoost, and CatBoost handle missing values natively by learning a default direction at each split. Sometimes the best imputation is none at all.

3.3 Outliers

Detect: - **IQR rule**: outside `[Q1 - 1.5·IQR, Q3 + 1.5·IQR]`. - **Z-score**: `|z| > 3` (assumes roughly normal data). - **Modified z-score** using the median absolute deviation: more robust. - **Isolation Forest / Local Outlier Factor**: multivariate, catches points that are only strange in combination.

Handle: - **Investigate first.** An outlier is either an error (fix or drop it) or the most interesting data point you have (fraud detection is entirely about outliers). - **Cap / winsorise** at a percentile. - **Transform**: a log transform tames right-skewed variables such as income or price. - **Use robust models**: trees are essentially immune to outliers in the feature space; linear regression with MSE is extremely sensitive because the error is squared. - **Use robust losses**: MAE or Huber instead of MSE.

3.4 Scaling and normalisation

Method	Formula	Range	Use when
Standardisation (z-score)	`(x - μ) / σ`	mean 0, std 1	default; good for linear models, SVM, PCA, NNs
Min-Max	`(x - min) / (max - min)`	[0, 1]	you need a bounded range; image pixels
Robust scaling	`(x - median) / IQR`	unbounded	data has outliers
MaxAbs	`x / max|x|`	[-1, 1]	sparse data, preserves zeros
Log / Box-Cox / Yeo-Johnson			skewed distributions
L2 normalisation (per row)	$x / \|\|x\|\|_2$	unit length	text vectors, embeddings, cosine similarity

Which models need scaling?

Needs scaling	Does not need scaling
Linear/logistic regression **with regularisation**	Decision trees
k-NN, K-Means (distance-based)	Random Forest
SVM (especially RBF)	Gradient boosting (XGBoost, LightGBM, CatBoost)
PCA	Naive Bayes
Neural networks	

> **KEY IDEA — a very common interview question** Tree-based models split on thresholds within a single feature at a time, so any monotonic rescaling of that feature produces exactly the same splits. That is why they are scale-invariant. Distance-based and gradient-based methods compare or combine features, so a feature measured in millions will dominate one measured in fractions.

3.5 Feature engineering

> **KEY IDEA** "Applied machine learning is basically feature engineering." Better features beat better algorithms almost every time on tabular data. Deep learning reduced the need for manual feature engineering on images, audio, and text, because those networks learn their own representations. On tabular data, it did not, which is why gradient boosting still wins most Kaggle tabular competitions.

Techniques worth having on the tip of your tongue:

- **Interactions**: `price_per_sqft = price / area`. Domain ratios are gold.
- **Polynomial features**: x^2, x^3, x_1x_2. Powerful, but explodes dimensionality.
- **Binning / discretisation**: turn age into bands. Adds non-linearity to linear models.
- **Aggregations**: per-user mean, count, std, min, max over a history window. This is the single highest-value family for behavioural data.
- **Date parts**: year, month, day-of-week, is_weekend, is_holiday, days_since_last_event.
- **Text**: length, word count, sentiment score, TF-IDF, embeddings.
- **Lag features** for time series: value at t-1, t-7, rolling mean over 30 days.
- **Domain features**: for credit, debt-to-income ratio. For telecom, calls in the last 30 days versus the previous 30. These come from talking to the business, and mentioning that in an interview scores well.

Feature selection

Family	How	Examples
Filter	statistics, model-independent, fast	correlation, chi-square, mutual information, variance threshold
Wrapper	search over subsets using a model	recursive feature elimination, forward/backward selection
Embedded	selection happens during training	Lasso (L1), tree feature importances

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif, RFE
from sklearn.linear_model import LassoCV

X_new = SelectKBest(mutual_info_classif, k=20).fit_transform(X, y)
lasso = LassoCV(cv=5).fit(X, y)
kept = X.columns[lasso.coef_ != 0]
```

Also drop features with near-zero variance, and one of each pair with correlation above roughly 0.95 (multicollinearity destabilises linear model coefficients even when it barely affects predictions).

3.6 Splitting the data, and cross-validation

The basic split

```
┌──────────────────────────────┬──────────────┬──────────┐
│           TRAIN              │ VALIDATION   │   TEST   │
│  fit the model parameters    │  tune the    │  final,  │
│                              │  hyper-      │  ONE     │
│                              │  parameters  │  report  │
└──────────────────────────────┴──────────────┴──────────┘
       60-80%                     10-20%       10-20%
```

k-Fold cross-validation

Split the training data into k folds. Train on k-1 and validate on the held-out one. Rotate. Average the k scores.

```
Fold 1: [VAL][ tr ][ tr ][ tr ][ tr ]
Fold 2: [ tr ][VAL][ tr ][ tr ][ tr ]
Fold 3: [ tr ][ tr ][VAL][ tr ][ tr ]
Fold 4: [ tr ][ tr ][ tr ][VAL][ tr ]
Fold 5: [ tr ][ tr ][ tr ][ tr ][VAL]
                                  → mean ± std of the metric
```

The standard deviation across folds matters as much as the mean: a high variance tells you the model is unstable or the dataset is too small.

The variants, and when each is mandatory

Variant	Use when
K-Fold	the default, i.i.d. data
Stratified K-Fold	**classification, always.** Preserves the class ratio in every fold
Group K-Fold	multiple rows per entity (patients, users) and no entity may span folds
TimeSeriesSplit	**temporal data, always.** Train only on the past, validate on the future
Leave-One-Out	tiny datasets. Nearly unbiased but very high variance and expensive
Repeated K-Fold	small datasets, to reduce split-luck variance
Nested CV	when you tune hyperparameters *and* need an unbiased performance estimate

```python
from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     GroupKFold, TimeSeriesSplit, cross_val_score)

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)   # stratify: do not forget

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_tr, y_tr, cv=cv, scoring="f1_macro")
print(f"{scores.mean():.3f} +/- {scores.std():.3f}")

# Time series: never shuffle
tscv = TimeSeriesSplit(n_splits=5)
#  train [0:100]      -> val [100:120]
#  train [0:120]      -> val [120:140]
#  train [0:140]      -> val [140:160]  ... expanding window
```

INTERVIEW QUESTION Q: Why can you not use standard k-fold cross-validation on time series? Because a random split lets the model train on data from the future relative to what it validates on, which is a form of leakage that does not exist at inference time. The score is then wildly optimistic. You use a forward-chaining split instead: always train on a prefix and validate on the segment immediately after, optionally with a gap between them if your features use rolling windows that could span the boundary.

Nested cross-validation

Inner loop tunes hyperparameters; outer loop estimates generalisation. Necessary when you want an honest performance estimate on a small dataset, because tuning on the same folds you report is itself a form of overfitting.

3.7 Data leakage: the silent killer

KEY IDEA Data leakage is when information that would not be available at prediction time gets into training. The signature is a model that performs suspiciously well offline and fails in production. If your AUC is 0.99 on a hard problem, assume leakage until proven otherwise. This is one of the top three questions asked of senior candidates.

The catalogue:

Type	Example	Fix
Target leakage	a `total_charges` column that is only populated after churn	drop anything computed after the prediction moment
Train-test contamination	scaling / imputing / selecting features before splitting	fit transformers on train only; use a `Pipeline`
Temporal leakage	random split on time-ordered data	time-based split
Duplicate rows	the same record in train and test	deduplicate before splitting
Group leakage	the same patient in both splits	GroupKFold
Target encoding leakage	category means computed on the full data	compute inside folds
Feature-selection leakage	selecting features using the full dataset's labels	select inside the CV loop

```python
# The correct pattern: everything inside a Pipeline, fitted only on train folds
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

numeric = Pipeline([("impute", SimpleImputer(strategy="median")),
                    ("scale",  StandardScaler())])
categorical = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                        ("onehot", OneHotEncoder(handle_unknown="ignore"))])

pre = ColumnTransformer([("num", numeric, num_cols),
                         ("cat", categorical, cat_cols)])

clf = Pipeline([("pre", pre), ("model", RandomForestClassifier())])
# Now cross_val_score(clf, X, y, cv=5) is leak-free by construction.
```

SCENARIO *Your churn model gets 0.97 AUC in validation and 0.61 in production. Walk me through the diagnosis.* First I check for leakage, because that gap is its signature. I list every feature and ask, for each one, "would this value exist, with this value, at the moment we make the prediction?" Columns like `cancellation_reason`, `final_invoice_amount`, or anything with `_at_churn` in the name are immediate suspects. Second I check the split: if it was random over a time-ordered dataset, I redo it as a temporal split and re-measure. Third I check for duplicates and for the same customer appearing in both splits. Fourth, if none of that explains it, I look at distribution shift between the training window and production traffic, comparing feature distributions with a KS test. In my experience the answer is leakage roughly nine times out of ten.

3.8 Imbalanced data

99% of transactions are legitimate. A model that predicts "legitimate" always gets 99% accuracy and catches zero fraud.

Fix it at the data level

Technique	What it does	Watch out for
Random undersampling	drop majority rows	throws away information
Random oversampling	duplicate minority rows	overfits to the duplicated points
SMOTE	synthesise new minority points by interpolating between neighbours	can create nonsense points in mixed / categorical spaces; **only apply to the training fold**
ADASYN	SMOTE weighted toward hard regions	amplifies noise
Tomek links / ENN	clean the boundary by removing ambiguous majority points	usually combined with SMOTE

Fix it at the algorithm level

```python
# Class weights: penalise minority mistakes more. Usually the first thing to try.
LogisticRegression(class_weight="balanced")
RandomForestClassifier(class_weight="balanced_subsample")

# XGBoost
XGBClassifier(scale_pos_weight=(n_negative / n_positive))

# PyTorch
loss = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([n_neg / n_pos]))

# Focal loss: down-weights easy examples so training focuses on the hard ones
# FL = -alpha * (1 - p_t)^gamma * log(p_t)
```

Fix it at the evaluation level (the most important one)

Stop using accuracy. Use precision, recall, F1, PR-AUC, and tune the decision threshold. This is Part IV.

COMMON MISTAKE — the most common SMOTE error in existence Applying SMOTE before splitting, or before cross-validation. Synthetic minority points are interpolated from real ones, so a synthetic point in the training set can be built from a real point that then lands in validation. Scores look excellent and mean nothing. Always resample **inside** the fold, which `imblearn.pipeline.Pipeline` does correctly (unlike sklearn's, which will apply the sampler at transform time too).

```python
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

pipe = ImbPipeline([("scale", StandardScaler()),
                    ("smote", SMOTE(random_state=42)),   # applied to train folds only
                    ("model", LogisticRegression())])
```

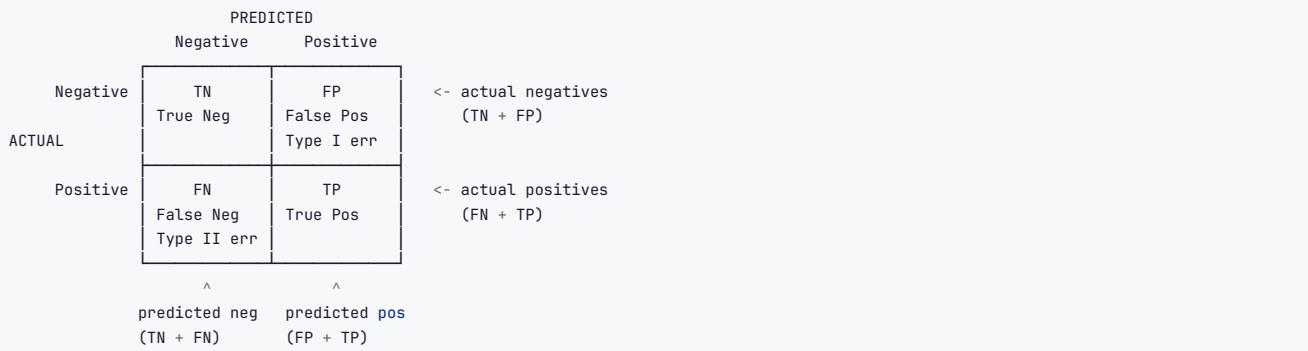
INTERVIEW QUESTION Q: You have a 1:1000 imbalance. Walk me through your approach. I start by fixing the metric, because with that ratio accuracy is meaningless: I use PR-AUC as the headline number and report precision and recall at a chosen operating point, chosen from the business cost of a false positive versus a false negative. Then a baseline: logistic regression with `class_weight="balanced"`, which is cheap and often surprisingly competitive. Then gradient boosting with `scale_pos_weight`. Only then do I try resampling, inside the CV folds, and I compare it honestly against class weighting, because SMOTE frequently does not beat it. Finally I tune the decision threshold on the validation set rather than leaving it at 0.5, since 0.5 is almost never the right operating point on imbalanced data. If recall is still poor I would look at whether this is better framed as anomaly detection, and I would look hard at feature engineering, because on extreme imbalance features usually matter more than any sampling trick.

Part IV — Evaluation Metrics

This is the chapter that gets tested hardest, in every interview, at every level. Not because it is difficult, but because it is where candidates who have only followed tutorials fall apart. Know this part cold.

4.1 The confusion matrix

Everything in binary classification comes from one 2x2 table.



How to read the names, once and forever: the second word is what the model **predicted**. The first word is whether it was **right**. So a "False Positive" is a prediction of positive that was wrong.

Cell	Meaning	Fraud example	Medical example
TP	predicted positive, was positive	fraud caught	disease correctly detected
TN	predicted negative, was negative	legitimate transaction approved	healthy person cleared
FP	predicted positive, was negative	legitimate card blocked (annoyed customer)	healthy person told they are ill (Type I)
FN	predicted negative, was positive	fraud missed (money lost)	sick person told they are fine (Type II)

> **KEY IDEA** The entire art of classification metrics is deciding **which error hurts more**, FP or FN, and then choosing the metric and the threshold that reflect that. There is no universally correct metric. If an interviewer asks "which metric would you use," the correct answer always begins with "it depends on the cost of each error type, so let me ask about the business impact."

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)
tn, fp, fn, tp = cm.ravel()      # sklearn order: TN, FP, FN, TP
print(cm)

ConfusionMatrixDisplay(cm, display_labels=["neg", "pos"]).plot(cmap="Blues")
plt.show()
```

> **COMMON MISTAKE** Assuming sklearn's confusion matrix is laid out the way textbooks draw it. sklearn puts **actual on rows, predicted on columns**, with labels sorted ascending, so `cm[0,0]` is TN and `cm.ravel()` gives `(tn, fp, fn, tp)`. Many textbooks and some other libraries transpose this. Always check with a tiny known example before trusting the numbers.

4.2 The core rates

Accuracy

THE MATH `Accuracy = (TP + TN) / (TP + TN + FP + FN)`

The fraction of predictions that were correct. Intuitive, and almost always the wrong headline metric.

> **KEY IDEA — the accuracy paradox** With 1% positives, always predicting negative gives 99% accuracy and a completely useless model. Accuracy is only meaningful when classes are roughly balanced **and** both error types cost about the same. Say this in an interview whenever accuracy comes up.

Precision

THE MATH `Precision = TP / (TP + FP)`

Of everything the model **flagged as positive**, what fraction actually was? Also called **positive predictive value (PPV)**.

Read it as: *"when it says yes, how often is it right?"*

Optimise precision when **false positives are expensive**: spam filters (never send a real email to junk), automated content removal, a system that automatically blocks transactions, recommending a costly medical procedure.

Recall

THE MATH `Recall = TP / (TP + FN)`

Of all the **actual positives**, what fraction did the model find? Also called **sensitivity**, **true positive rate (TPR)**, **hit rate**, and (in epidemiology) **power**.

Read it as: *"of everything it should have caught, how much did it catch?"*

Optimise recall when **false negatives are expensive**: cancer screening, fraud detection, safety-critical fault detection, security threat detection.

> **KEY IDEA — the recall / sensitivity naming, which trips people up Recall = Sensitivity = True Positive Rate = TP/(TP+FN).** Three names, one formula. "Recall" is the ML term, "sensitivity" is the medical/statistics term, "TPR" is the signal-detection term. Interviewers sometimes switch vocabulary mid-question specifically to see whether you notice they are the same thing.

Specificity

THE MATH `Specificity = TN / (TN + FP)`

Of all the **actual negatives**, what fraction were correctly identified? Also called the **true negative rate (TNR)**. It is the mirror image of recall.

`1 - Specificity = FPR = FP / (TN + FP)`, the **false positive rate**, which is the x-axis of the ROC curve.

> **ANALOGY — sensitivity vs specificity, the fishing net** A **sensitive** net has tiny holes: it catches every fish (high recall) but also drags up boots, weeds, and plastic bags (low precision, low specificity). A **specific** net has huge holes: everything it catches is definitely a fish (high specificity, few false alarms) but plenty of fish swim straight through (low sensitivity). Screening tests are built sensitive so nothing is missed; confirmatory tests are built specific so nobody is wrongly diagnosed. Real medical practice runs both in sequence, and that two-stage pattern is a great answer to "how do you get both high precision and high recall."

Negative predictive value and the rest of the family

Metric	Formula	Also called	Reads as
Accuracy	(TP+TN)/all		overall correctness
Precision	TP/(TP+FP)	PPV	when it says yes, is it right
Recall	TP/(TP+FN)	Sensitivity, TPR, hit rate	did it find them all
Specificity	TN/(TN+FP)	TNR, selectivity	did it correctly clear the negatives
NPV	TN/(TN+FN)		when it says no, is it right
FPR	FP/(FP+TN)	fall-out, Type I rate	1 - specificity
FNR	FN/(FN+TP)	miss rate, Type II rate	1 - recall
FDR	FP/(FP+TP)	false discovery rate	1 - precision
Prevalence	(TP+FN)/all	base rate	how common the positive class is

```python
def all_metrics(tn, fp, fn, tp):
    return {
        "accuracy":    (tp + tn) / (tp + tn + fp + fn),
        "precision":   tp / (tp + fp) if (tp + fp) else 0,
        "recall":      tp / (tp + fn) if (tp + fn) else 0,   # = sensitivity = TPR
        "specificity": tn / (tn + fp) if (tn + fp) else 0,   # = TNR
        "npv":         tn / (tn + fn) if (tn + fn) else 0,
        "fpr":         fp / (fp + tn) if (fp + tn) else 0,
        "fnr":         fn / (fn + tp) if (fn + tp) else 0,
    }
```

> **KEY IDEA — precision and NPV depend on prevalence; recall and specificity do not** Sensitivity and specificity are properties of the *test itself*: they are computed within the actual-positive and actual-negative rows, so changing the class balance does not change them. Precision and NPV are computed down the predicted columns, mixing both rows, so they shift as prevalence shifts. This is the formal version of the disease-testing puzzle in Part I, and it explains why a model with excellent recall and specificity can still have terrible precision on a rare class. It is one of the most impressive things you can say in a metrics discussion.

4.3 F1 and the F-beta family

Precision and recall trade off against each other. F1 combines them.

THE MATH `F1 = 2 · (Precision · Recall) / (Precision + Recall)`

This is the **harmonic** mean, not the arithmetic mean. The harmonic mean punishes imbalance: precision 1.0 with recall 0.0 gives an arithmetic mean of 0.5 but an F1 of 0. That is the point: F1 only gets high when *both* are high.

F-beta generalises it:

`F_β = (1 + β²) · (P · R) / (β²·P + R)`

- β = 1: balanced (F1).
- β = 2: recall weighted twice as heavily as precision. Use when missing positives is worse.
- β = 0.5: precision weighted twice as heavily. Use when false alarms are worse.

```python
from sklearn.metrics import f1_score, fbeta_score, classification_report

f1_score(y_true, y_pred)
fbeta_score(y_true, y_pred, beta=2)      # recall-leaning
print(classification_report(y_true, y_pred, digits=3))
```

`classification_report` gives per-class precision, recall, F1, and support in one call. Get in the habit of printing it instead of a bare accuracy number.

Matthews Correlation Coefficient (MCC)

THE MATH `MCC = (TP·TN - FP·FN) / sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN))`

Ranges from -1 (perfectly wrong) through 0 (random) to +1 (perfect). MCC is the only common single-number metric that uses **all four** cells of the confusion matrix and stays honest under heavy imbalance. Many practitioners consider it strictly better than F1, because F1 ignores TN entirely. Mentioning MCC unprompted is a strong signal.

Cohen's Kappa

Agreement corrected for chance: `κ = (p_o - p_e) / (1 - p_e)`, where p_o is observed agreement and p_e is expected-by-chance agreement. Used for inter-annotator agreement on labelling projects, and occasionally for imbalanced classification.

Balanced accuracy

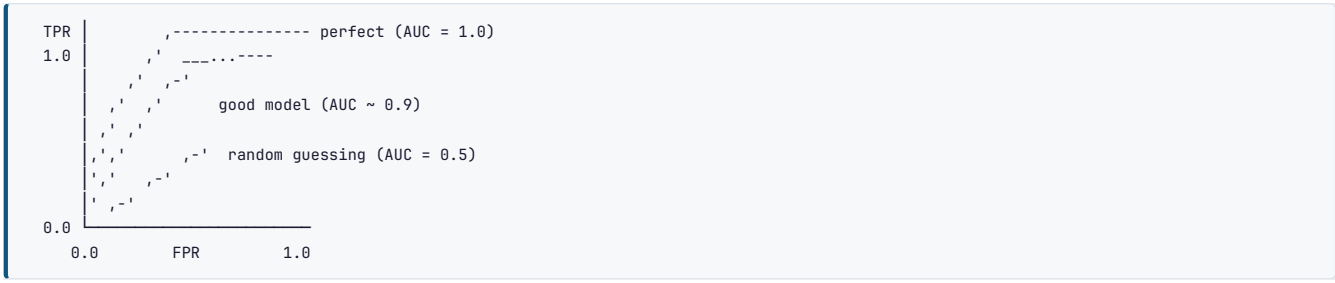
`(Sensitivity + Specificity) / 2`. The multi-class version is the mean of per-class recalls. A simple, defensible headline metric on imbalanced problems.

4.4 ROC-AUC and PR-AUC

Every classifier really outputs a **probability**. The confusion matrix only appears after you pick a **threshold** to convert probability into a label. Threshold-free metrics evaluate the ranking quality across all possible thresholds.

The ROC curve

Plot **TPR (recall)** on the y-axis against **FPR (1 - specificity)** on the x-axis, sweeping the threshold from 1 down to 0.



AUC is the area under that curve. Its beautiful probabilistic interpretation:

> **KEY IDEA AUC is the probability that the model ranks a randomly chosen positive example above a randomly chosen negative example.** 0.5 is random, 1.0 is perfect, below 0.5 means your model is worse than a coin flip (invert its predictions and you have a good model). This interpretation, stated crisply, is one of the highest-value sentences in this entire guide for interviews.

AUC is threshold-independent and invariant to class balance in the sense that it does not change if you resample the negatives. That last property is both its strength and its trap.

The Precision-Recall curve

Plot **precision** (y) against **recall** (x), again sweeping the threshold. The area under it is **PR-AUC**, also known as **average precision (AP)**.

KEY IDEA — the single most important metric choice on imbalanced data On heavily imbalanced problems, **prefer PR-AUC over ROC-AUC**. Reason: FPR has a huge denominator (all the negatives), so even thousands of false positives barely move it, and ROC-AUC can look excellent (0.95+) while precision is 3%. The PR curve puts false positives in the numerator's competition directly, via precision, so it stays sensitive to exactly the failure mode you care about. Note also that a random classifier's PR-AUC equals the positive class prevalence, not 0.5, so always report the baseline alongside it.

```
from sklearn.metrics import (roc_auc_score, roc_curve,
                             average_precision_score, precision_recall_curve)

y_prob = model.predict_proba(X_val)[:, 1]     # probabilities, not labels

roc_auc_score(y_val, y_prob)                  # ROC-AUC
average_precision_score(y_val, y_prob)        # PR-AUC

fpr, tpr, roc_thresh = roc_curve(y_val, y_prob)
prec, rec, pr_thresh = precision_recall_curve(y_val, y_prob)
```

COMMON MISTAKE Passing predicted **labels** to `roc_auc_score` instead of predicted **probabilities**. It runs without error and gives you a meaningless number equal to balanced accuracy. AUC needs a ranking, so it needs scores.

Lift and gain charts

Business stakeholders often prefer these. "If we contact the top 10% ranked by the model, we capture 45% of all churners, which is 4.5x lift over random." Being able to translate a model into that sentence is a real skill and it comes up in interviews for applied roles.

4.5 Choosing the threshold

The default 0.5 is arbitrary. It is correct only when classes are balanced and errors cost the same, which is almost never.

```
import numpy as np
from sklearn.metrics import precision_recall_curve

prec, rec, thresholds = precision_recall_curve(y_val, y_prob)

# Option A: maximise F1
f1 = 2 * prec * rec / (prec + rec + 1e-12)
best_f1_threshold = thresholds[np.argmax(f1[:-1])]

# Option B: the highest threshold that still meets a recall floor
#           "we must catch 95% of fraud; maximise precision subject to that"
ok = rec[:-1] >= 0.95
best_threshold = thresholds[ok][np.argmax(prec[:-1][ok])]

# Option C: minimise expected business cost. The most defensible in an interview.
COST_FN, COST_FP = 500, 10       # missing fraud costs 500; a false alarm costs 10
costs = []
for t in np.linspace(0.01, 0.99, 99):
    pred = (y_prob >= t).astype(int)
    fp = ((pred == 1) & (y_val == 0)).sum()
    fn = ((pred == 0) & (y_val == 1)).sum()
    costs.append((fp * COST_FP + fn * COST_FN, t))
min_cost, optimal_threshold = min(costs)
```

KEY IDEA Tune the threshold on the **validation** set, not the test set, and re-tune it whenever prevalence shifts in production. In a system design interview, saying "the threshold is a business decision, not a modelling one, and it should be configurable at serving time rather than baked into the model artifact" is exactly the kind of production thinking they are listening for.

The Youden J statistic

`J = Sensitivity + Specificity - 1`. The threshold that maximises J is the point on the ROC curve furthest from the diagonal. Common in medical contexts. Useful, but a cost-based threshold is almost always better when you can get cost estimates.

4.6 Multi-class and multi-label

Averaging strategies

With more than two classes, precision/recall/F1 are computed per class and then combined.

Averaging	How	Use when
micro	pool all TP/FP/FN across classes, then compute once	you care about overall performance; equals accuracy in single-label multi-class
macro	compute per class, then take an unweighted mean	**all classes matter equally, including rare ones**
weighted	per class, averaged weighted by support	you want a single number reflecting the actual class distribution
samples	per instance, then averaged	multi-label only

KEY IDEA Macro-F1 is the right default when you care about minority classes, because a class with 10 examples counts exactly as much as a class with 10,000. Weighted-F1 lets the majority class dominate and will hide the failure. Micro-F1 in a single-label problem is mathematically identical to accuracy, so quoting "micro-F1" as though it were more sophisticated is a tell.

```
from sklearn.metrics import f1_score, confusion_matrix
f1_score(y_true, y_pred, average="macro")
f1_score(y_true, y_pred, average="weighted")

# The multi-class confusion matrix: rows are actual, columns predicted.
# The diagonal is correct; every off-diagonal cell is a specific confusion
# you should look at by name (e.g. the model keeps calling 4s 9s).
print(confusion_matrix(y_true, y_pred))
```

Multi-class ROC-AUC

Two schemes: **One-vs-Rest (OvR)** computes an AUC per class against all others; **One-vs-One (OvO)** averages the AUC of every pair. Use `roc_auc_score(y, y_proba, multi_class="ovr", average="macro")`.

Multi-label metrics

Each sample can have several labels at once.

- **Exact match ratio / subset accuracy**: all labels must be right. Very strict.
- **Hamming loss**: fraction of individual label slots that are wrong. Lower is better.
- **Micro / macro F1**: as above, computed across the label matrix.

4.7 Regression metrics

Metric	Formula	Units	Notes
MAE	`mean(|y - ŷ|)`	same as y	robust to outliers, easy to explain
MSE	`mean((y - ŷ)²)`	y squared	penalises large errors heavily, differentiable everywhere
RMSE	`sqrt(MSE)`	same as y	the default; interpretable, still outlier-sensitive
RMSLE	RMSE on log1p values	relative	for targets spanning orders of magnitude; penalises under-prediction more
MAPE	`mean(|y-ŷ|/|y|)·100`	%	intuitive for business, **breaks when y is near zero**, asymmetric
SMAPE	symmetric version	%	fixes some of MAPE's asymmetry
R^2	`1 - SS_res/SS_tot`	unitless	fraction of variance explained; can be negative
Adjusted R^2	penalises extra features	unitless	use when comparing models with different feature counts
Huber	MSE near zero, MAE far away		robust loss and metric hybrid
Quantile / pinball loss	asymmetric		when over- and under-prediction cost differently

KEY IDEA — RMSE vs MAE, asked constantly RMSE squares the errors before averaging, so one prediction that is off by 10 contributes as much as a hundred that are off by 1. Use RMSE when large errors are disproportionately bad (predicting server capacity, structural loads). Use MAE when all errors are equally bad in proportion to size and the data has outliers you do not want to chase (delivery time estimates). A quiet mathematical note that impresses: the value that minimises MSE is the conditional **mean**, and the value that minimises MAE is the conditional **median**, which is exactly why MAE is robust.

COMMON MISTAKE Reporting R^2 without context. R^2 of 0.85 is superb for predicting human behaviour and dismal for predicting a physical process. R^2 also increases mechanically as you add features, even useless ones, which is what adjusted R^2 corrects for. And R^2 can be negative: that means your model is worse than predicting the mean.

```python
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             r2_score, mean_absolute_percentage_error)
import numpy as np

mae  = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2   = r2_score(y_true, y_pred)
```

4.8 Ranking, clustering, and probabilistic metrics

Ranking / recommendation / retrieval

Metric	What it captures
Precision@k	how many of the top k are relevant
Recall@k	how many of all relevant items appeared in the top k
MAP (mean average precision)	precision averaged over the positions of the relevant items
MRR (mean reciprocal rank)	1 / rank of the first correct item, averaged. The RAG retrieval metric
NDCG@k	discounted cumulative gain, normalised. Handles graded relevance and position discounting. The industry standard for search
Hit rate @k	did the correct item appear at all in the top k

These are the metrics that connect this guide to the RAG guide: evaluating a retriever is a ranking problem, and hit rate plus MRR are exactly what you measure there.

Clustering (no labels available)

Metric	Range	Meaning
Silhouette score	-1 to 1	how much closer a point is to its own cluster than the next nearest. Higher is better
Davies-Bouldin	0 up	average similarity between each cluster and its most similar one. Lower is better
Calinski-Harabasz	0 up	between-cluster over within-cluster dispersion. Higher is better
Inertia / WCSS	0 up	within-cluster sum of squares. Only used for the elbow method

Clustering (labels available for evaluation)

Adjusted Rand Index (ARI), Normalised Mutual Information (NMI), homogeneity, completeness, V-measure.

Probabilistic metrics

Metric	Formula	Notes
Log loss / cross-entropy	`-mean(y·log p + (1-y)·log(1-p))`	punishes confident mistakes brutally. The training loss for classification
Brier score	`mean((p - y)²)`	MSE on probabilities. Lower is better. Decomposes into calibration + refinement
Perplexity	`exp(cross-entropy)`	the language modelling metric. "How many equally likely options is the model choosing among"

4.9 Calibration

A model is **calibrated** if, among all the cases where it says 70%, about 70% really are positive. Ranking quality (AUC) and calibration are independent: a model can rank perfectly and still be badly calibrated.

Why it matters: any decision that multiplies probability by a cost (expected loss, expected revenue, risk pricing) needs calibrated probabilities, not just a good ranking.

```python
from sklearn.calibration import CalibratedClassifierCV, calibration_curve

prob_true, prob_pred = calibration_curve(y_val, y_prob, n_bins=10)
# Plot prob_true against prob_pred; the diagonal is perfect calibration.

# Fix miscalibration with a held-out set:
calibrated = CalibratedClassifierCV(base_model, method="isotonic", cv=5)
#   method="sigmoid"  -> Platt scaling: a logistic fit. Better for small data
#   method="isotonic" -> non-parametric, more flexible. Needs more data
```

Known behaviours worth quoting: **logistic regression is naturally well calibrated** because it optimises log loss directly. **SVMs are not** (their decision function is a margin, not a probability), which is why `SVC(probability=True)` internally runs Platt scaling. **Random Forests** are typically under-confident at the extremes (averaging pushes predictions toward the middle). **Boosted trees and modern deep nets** tend to be over-confident. For neural networks, **temperature scaling** (dividing logits by a learned scalar T before softmax) is the standard one-parameter fix.

> **SCENARIO** *Your fraud model has 0.94 ROC-AUC and the business says it is useless. What happened?* Almost certainly the model ranks well but the operating point is wrong, and the metric is hiding it. With fraud at, say, 0.2% prevalence, an FPR of 5% looks fine on a ROC curve but means 5,000 false alarms per 100,000 legitimate transactions, drowning the 200 real fraud cases: precision would be about 4%. I would immediately switch the headline metric to PR-AUC, plot precision and recall against the threshold, and pick the operating point from the review team's actual capacity, for instance "we can manually review 500 alerts a day, so what recall do we get at that volume." I would also check calibration, because if the scores feed an expected-loss calculation the probabilities need to be honest, not just correctly ordered.

CHECKPOINT — Part IV self-test (answer out loud) 1. Draw the confusion matrix from memory and label all four cells plus Type I and Type II errors. 2. Give three names for TP/(TP+FN). 3. Why is precision affected by class prevalence but recall is not? 4. Why the harmonic mean in F1 rather than the arithmetic mean? 5. State the probabilistic interpretation of AUC in one sentence. 6. When do you prefer PR-AUC to ROC-AUC, and why exactly? 7. Macro vs weighted F1: which hides a failing minority class? 8. Why is RMSE more outlier-sensitive than MAE, and which conditional statistic does each minimise? 9. What does it mean for a model to be calibrated, and name two fixes.

Part V — Classical Supervised Learning

For each model: what it is, the mathematics, the assumptions, hyperparameters, when to use it, when not to, implementation, and the interview questions.

5.1 Linear Regression

The idea

Fit a straight line (or hyperplane) through the data that minimises squared error.

THE MATH Prediction: $\hat{y} = w_0 + w_1x_1 + w_2x_2 + ... + w_nx_n = Xw$ Loss (MSE): $L = (1/n) \Sigma (y_i - \hat{y}_i)^2$

There are two ways to solve it:

1. **Closed form (Normal Equation)**: $w = (X^TX)^{-1} X^Ty$. Exact, no iterations, no learning rate. But it costs $O(n^3)$ in the number of features because of the inverse, so it becomes impractical beyond roughly 10,000 features, and it fails outright if X^TX is singular (perfect multicollinearity).
2. **Gradient descent**: iterate $w := w - \alpha \nabla L$. Scales to huge data, needs a learning rate and scaled features.

The five assumptions (a guaranteed interview question)

Assumption	Meaning	How to check	If violated
Linearity	the relationship between X and y is linear in the parameters	residuals vs fitted plot should show no pattern	add polynomial terms, transform, or use a non-linear model
Independence	observations are independent	Durbin-Watson test; think about the data's structure	use time series or mixed-effects models
Homoscedasticity	constant error variance across the range of predictions	residual plot should be an even band, not a fan/cone	log-transform y, weighted least squares, robust standard errors
Normality of residuals	errors are normally distributed	Q-Q plot, Shapiro-Wilk	matters mainly for confidence intervals and p-values, not for point predictions
No multicollinearity	predictors are not near-linear combinations of each other	**VIF > 5 or 10** is the standard flag	drop one of the pair, combine them, or use Ridge

> **KEY IDEA** Multicollinearity does not much hurt *predictions*, but it makes individual *coefficients* unstable and uninterpretable: tiny data changes flip signs and inflate standard errors. So if your goal is prediction you may tolerate it; if your goal is inference ("does advertising spend cause sales") you must fix it. Knowing that distinction is what separates a data scientist's answer from a bootcamp answer.

```python
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

lr = LinearRegression().fit(X_train, y_train)
print(lr.coef_, lr.intercept_)

# statsmodels when you need p-values, confidence intervals, and diagnostics
model = sm.OLS(y_train, sm.add_constant(X_train)).fit()
print(model.summary())

# Variance Inflation Factor
vif = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
```

Polynomial regression

Still a *linear* model, because it is linear in the coefficients: you just add x^2, x^3, and interaction columns as new features. Degree is the key hyperparameter and it overfits ferociously as it rises.

> **INTERVIEW QUESTION Q: When would you use the normal equation instead of gradient descent?** When the feature count is small (roughly under ten thousand) and the dataset fits in memory, because the closed form is exact, needs no learning rate, and needs no iterations. I would switch to gradient descent for high-dimensional or very large data, since the matrix inversion is cubic in the number of features and memory-heavy, and for anything where I want to train in a streaming or mini-batch fashion. I would also note that `sklearn.LinearRegression` actually uses an SVD-based least squares solver rather than literally inverting X^TX, which is more numerically stable and handles rank-deficient matrices gracefully.

5.2 Regularisation: Ridge, Lasso, Elastic Net

Regularisation adds a penalty on coefficient size to the loss, trading a little bias for a large reduction in variance.

	Ridge (L2)	Lasso (L1)	Elastic Net
Penalty	$\lambda \sum w_j^2$	$\lambda \sum \lvert w_j \rvert$	$\lambda(\rho \sum \lvert w_j \rvert + (1-\rho)/2 \sum w_j^2)$
Effect on coefficients	shrinks toward zero, never exactly zero	drives some **exactly to zero**	both
Feature selection	no	**yes**	yes
With correlated features	shares the weight among them	arbitrarily picks one	groups them together
Closed form	yes	no (needs coordinate descent)	no
Bayesian view	Gaussian prior on weights	Laplace prior on weights	mixture

KEY IDEA — the geometric explanation, which is what they want to hear Draw the constraint region. L2's is a circle; L1's is a diamond with corners sitting on the axes. The optimisation finds the point where the loss contours first touch that region. A diamond's corners stick out, so contact happens at a corner far more often than at a face, and a corner is exactly a point where one coefficient is zero. A circle has no corners, so contact almost never lands exactly on an axis. That is why L1 gives sparsity and L2 does not.

```python
from sklearn.linear_model import Ridge, Lasso, ElasticNet, RidgeCV, LassoCV

Ridge(alpha=1.0).fit(X, y)                    # alpha is lambda
Lasso(alpha=0.1).fit(X, y)
ElasticNet(alpha=0.1, l1_ratio=0.5).fit(X, y)  # l1_ratio is rho

# Always tune alpha by cross-validation, never guess it
best = RidgeCV(alphas=np.logspace(-3, 3, 50), cv=5).fit(X, y)
print(best.alpha_)
```

COMMON MISTAKE Forgetting to scale features before regularising. The penalty is applied to the raw coefficient magnitudes, so a feature measured in dollars gets a much smaller coefficient than one measured in millions of dollars, and the penalty therefore punishes them unequally and arbitrarily. **Always standardise before Ridge/Lasso/Elastic Net.** Also: do not penalise the intercept, which sklearn already handles for you.

As $\lambda \to 0$ you recover ordinary least squares. As $\lambda \to \infty$ all coefficients go to zero and the model predicts the mean.

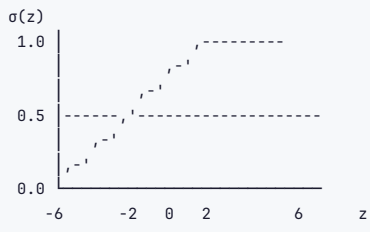
5.3 Logistic Regression

Despite the name, this is a **classification** algorithm. It is the single most important model in this guide to understand deeply, because it is the baseline for everything and it is a one-neuron neural network.

The mechanism

Take a linear combination, then squash it into (0,1) with the sigmoid.

THE MATH `z = w·x + b` `σ(z) = 1 / (1 + e^(-z))` → this is `P(y=1 | x)`



The loss is **binary cross-entropy** (log loss), which is the negative log-likelihood of a Bernoulli:

$L = -(1/n) \sum [\, y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \,]$

Why not MSE for classification?

Two reasons, and interviewers want both. First, with the sigmoid, MSE becomes **non-convex** in the weights, so gradient descent can get stuck in local minima. Cross-entropy is convex for logistic regression. Second, MSE's gradient contains a $\sigma'(z)$ factor that goes to zero when the model is confidently wrong, so learning stalls exactly when it most needs to move. Cross-entropy's gradient is beautifully simple:

THE MATH — the result worth memorising $\partial L/\partial w = (1/n)\, X^T (\sigma(Xw) - y)$

The gradient is just `(prediction - truth)` times the input. That elegance is not a coincidence: it is what you get when the loss is the negative log-likelihood of the distribution matching your output activation. The same clean form appears for softmax with categorical cross-entropy.

Odds, log-odds, and interpretation

`odds = p / (1-p)`, and `log(odds) = w·x + b`. So a coefficient w_j means: a one-unit increase in x_j multiplies the odds by `e^w_j`, holding everything else fixed. That interpretability is why regulated industries (credit scoring, insurance, medicine) still run logistic regression on problems where a boosted tree would score higher.

Multi-class: softmax

`P(y=k|x) = e^(z_k) / Σ_j e^(z_j)`. Also called multinomial logistic regression or maximum entropy. sklearn's `multi_class="multinomial"` does this properly; "ovr" trains one binary model per class instead.

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(
    penalty="l2",           # "l1", "l2", "elasticnet", None
    C=1.0,                  # INVERSE regularisation strength: smaller C = stronger penalty
    solver="lbfgs",         # "liblinear" small data; "saga" for l1/elasticnet & big data
    max_iter=1000,
    class_weight="balanced",
    random_state=42,
).fit(X_train, y_train)

proba = clf.predict_proba(X_test)[:, 1]
```

COMMON MISTAKE Confusing `C` with `alpha`. In sklearn's linear models, `alpha` is the regularisation strength (higher = more regularisation) but in `LogisticRegression` and `SVC` the parameter is `C = 1/alpha` (higher = **less** regularisation). Getting this backwards in an interview is a small but memorable error.

INTERVIEW QUESTION Q: Is logistic regression a linear model? Yes, and the precision matters. It is linear in the **log-odds**: the decision boundary in feature space is a hyperplane. The sigmoid is a monotonic transformation applied to that linear score to turn it into a probability; it does not bend the boundary. That is why logistic regression cannot solve XOR without engineered interaction features, and it is the exact same limitation that killed the single-layer perceptron in the 1960s.

5.4 k-Nearest Neighbours

The idea

There is no training. To classify a new point, find the k closest training points and take a majority vote (or, for regression, the mean).

Hyperparameters: - `k`: small k means low bias and high variance (noisy, jagged boundary); large k means high bias and low variance (smooth, possibly underfit). Use an odd k for binary classification to avoid ties. - `metric`: Euclidean (L2), Manhattan (L1), Minkowski (general), cosine (for text/embeddings), Hamming (for binary features). - `weights`: `"uniform"` or `"distance"` (closer neighbours vote more).

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5, weights="distance", metric="minkowski", p=2)
knn.fit(X_train_scaled, y_train)     # scaling is MANDATORY here
```

Costs: training is O(1) (just store the data). Prediction is O(n·d) per query with brute force, which is why k-NN is unusable at scale without an index. KD-trees help in low dimensions; ball trees a bit higher; above roughly 20 dimensions you need approximate methods, which is exactly what a **vector database's HNSW index** is (this is the direct link to the RAG guide: semantic search is k-NN with an ANN index).

KEY IDEA k-NN is called a **lazy learner** because it defers all computation to prediction time. It is also the clearest illustration of the **curse of dimensionality**: in high dimensions all pairwise distances converge toward the same value, so "nearest" stops being meaningful. Always scale features, and consider reducing dimensionality first.

5.5 Naive Bayes

The idea

Apply Bayes' theorem with the (usually false) assumption that features are conditionally independent given the class.

THE MATH $P(y|x_1..x_n) \propto P(y) \cdot \Pi\ P(x_i \mid y)$

The "naive" part is that product: it assumes no feature interacts with another. In text, that means assuming word order and co-occurrence do not matter. That is obviously wrong, and it works remarkably well anyway, because for **classification** you only need the correct class to have the highest score, not the correct probability.

The variants

Variant	Feature type	Typical use
GaussianNB	continuous	assumes each feature is normal within a class
MultinomialNB	counts	text classification with word counts / TF-IDF
BernoulliNB	binary	text with presence/absence, short documents
ComplementNB	counts	imbalanced text, more stable than MultinomialNB
CategoricalNB	categorical	discrete non-ordinal features

Laplace (additive) smoothing (`alpha=1.0`) prevents the zero-frequency problem: an unseen word in a class would otherwise give the whole product a probability of exactly zero.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import make_pipeline

model = make_pipeline(TfidfVectorizer(ngram_range=(1,2)), MultinomialNB(alpha=1.0))
model.fit(texts_train, y_train)
```

Pros: extremely fast, works with very little data, natural multi-class, great baseline for text. Cons: the independence assumption breaks probability estimates (its outputs are badly calibrated and pushed toward 0 and 1), and it cannot learn interactions.

5.6 Support Vector Machines

The idea

Find the hyperplane that separates the classes with the **largest possible margin** (the widest street between them). Only the points on the edge of that street, the **support vectors**, determine the solution: every other point could be deleted without changing anything.

Hard margin requires perfect separation and is useless on real data. **Soft margin** introduces slack variables and the parameter **C**:

- Small C: a wide margin, more violations tolerated. More regularisation, higher bias.
- Large C: a narrow margin, few violations tolerated. Less regularisation, higher variance, risk of overfitting.

The kernel trick

Data that is not linearly separable in the original space often is in a higher-dimensional space. The trick is that the SVM's dual formulation only ever needs **dot products** between points, so you can replace the dot product with a **kernel function** that computes the dot product in the high-dimensional space **without ever constructing it**.

Kernel	`K(x, x')`	Notes
Linear	`x · x'`	fast; use for text and high-dimensional sparse data
Polynomial	`(γ x·x' + r)^d`	degree d interactions
RBF / Gaussian	`exp(-γ‖x - x'‖²)`	the default. Infinite-dimensional feature space
Sigmoid	`tanh(γ x·x' + r)`	rarely better than RBF

gamma (γ) controls the reach of a single training example. Low gamma means far reach and a smooth boundary; high gamma means each point influences only its immediate neighbourhood, which produces a wiggly boundary and overfits.

```
from sklearn.svm import SVC, SVR, LinearSVC

svm = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True)
svm.fit(X_train_scaled, y_train)     # scaling is mandatory
print(svm.support_vectors_.shape)

# For large n, LinearSVC is far faster than SVC(kernel="linear")
```

Complexity: training is roughly $O(n^2)$ to $O(n^3)$ in the number of samples, which makes SVMs impractical above roughly 100,000 rows. That is the main reason they lost ground to gradient boosting and neural networks.

INTERVIEW QUESTION Q: Explain the kernel trick. The SVM's dual optimisation only involves inner products between pairs of training points, never the points themselves in isolation. A kernel function computes the inner product that *would* result from mapping both points into some high-dimensional (sometimes infinite-dimensional) feature space, but computes it directly in the original space. So you get the expressive power of that space without paying the cost of constructing it, which is what "trick" refers to. The RBF kernel corresponds to an infinite-dimensional space, which is why it can carve out arbitrarily complex boundaries, and also why it overfits if gamma is set too high.

5.7 Decision Trees

The idea

Recursively split the data on the feature and threshold that best separate the classes, forming a tree of if/else rules.

```
                [age < 30?]
               /           \
            yes             no
            /                 \
   [income < 50k?]           [owns_home?]
     /        \                /       \
  CHURN      STAY           STAY      CHURN
```

Split criteria

Criterion	Formula	Used by
Gini impurity	$1 - \Sigma\ p_i^2$	CART, sklearn default. Slightly faster (no log)
Entropy / Information Gain	$-\Sigma\ p_i\ \log_2\ p_i$	ID3, C4.5
Gain ratio	information gain normalised by split entropy	C4.5: corrects the bias toward high-cardinality features
MSE / variance reduction		regression trees
Chi-square		CHAID

Gini and entropy almost always produce the same tree. Gini is the default because it avoids a logarithm.

Information gain = parent impurity minus the weighted average of the children's impurity. The tree greedily picks the split that maximises it.

Hyperparameters (all of them fight overfitting)

```python
from sklearn.tree import DecisionTreeClassifier, plot_tree

tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,                # THE main lever. None = grow until pure = guaranteed overfit
    min_samples_split=20,       # do not split a node with fewer than this
    min_samples_leaf=10,        # every leaf must keep at least this many samples
    max_features="sqrt",        # consider a random subset of features per split
    max_leaf_nodes=None,
    ccp_alpha=0.01,             # cost-complexity (post-)pruning strength
    class_weight="balanced",
    random_state=42,
)
tree.fit(X_train, y_train)
plot_tree(tree, feature_names=cols, class_names=["no","yes"], filled=True)
```

Pre-pruning stops growth early (max_depth, min_samples_leaf). **Post-pruning** grows the full tree then removes branches that do not pay for themselves, via cost-complexity pruning with `ccp_alpha`.

Pros: interpretable, no scaling needed, handles mixed feature types, captures interactions automatically, fast inference. **Cons:** high variance (a small data change can restructure the whole tree), greedy so not globally optimal, biased toward features with many possible split points, cannot extrapolate beyond the training range in regression, axis-aligned boundaries only.

KEY IDEA A single deep tree is a textbook high-variance model. Every ensemble method in the next two sections exists to fix precisely that weakness, which is why trees are the base learner of choice for both bagging and boosting: they are strong enough to be useful and unstable enough to benefit enormously from averaging.

5.8 Bagging and Random Forest

Bagging (Bootstrap Aggregating)

1. Draw B bootstrap samples (sample n rows **with replacement**) from the training set.
2. Train one model on each.
3. Average the predictions (regression) or take a majority vote (classification).

Because the models are trained on different resamples, their errors are partly independent, and averaging cancels variance without increasing bias. Roughly 37% of rows are left out of each bootstrap sample (the limit of `(1 - 1/n)^n = 1/e`), and those **out-of-bag (OOB)** samples give you a free validation estimate with no separate holdout.

Random Forest

Bagging plus one crucial extra: **at each split, consider only a random subset of features** (`max_features`, default `sqrt(p)` for classification).

> **KEY IDEA — why the feature subsampling matters** Without it, if one feature is dominant every bootstrapped tree splits on it first and the trees end up highly correlated, so averaging them barely reduces variance. Forcing each split to consider a random feature subset **decorrelates** the trees, and the variance reduction from averaging is proportional to how uncorrelated the members are. That sentence, delivered clearly, is the whole answer to "what is the difference between bagging and random forest."

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500,          # more is always better for quality, just slower. No overfitting risk
    max_depth=None,            # forests tolerate deep trees; averaging handles the variance
    max_features="sqrt",       # "sqrt" classification, 1.0 or "log2" regression
    min_samples_leaf=1,
    bootstrap=True,
    oob_score=True,            # free validation estimate
    n_jobs=-1,                 # trees are embarrassingly parallel
    class_weight="balanced_subsample",
    random_state=42,
).fit(X_train, y_train)

print("OOB score:", rf.oob_score_)

# Feature importance: prefer permutation importance to the built-in one
from sklearn.inspection import permutation_importance
r = permutation_importance(rf, X_val, y_val, n_repeats=10, random_state=42)
```

> **COMMON MISTAKE** Trusting `rf.feature_importances_` (mean decrease in impurity). It is biased toward high-cardinality and continuous features, and it is computed on the training data, so a feature that only helps the model overfit will look important. Use **permutation importance** on a held-out set, or SHAP. Interviewers love this one because it separates people who read the docs from people who read the tutorial.

Extra Trees (Extremely Randomised Trees) go a step further: thresholds are chosen at random rather than optimised. Faster, more bias, less variance, sometimes better.

5.9 Boosting

The core distinction

> **KEY IDEA — bagging vs boosting, asked in essentially every interview Bagging** trains many models **in parallel** on different resamples and averages them. It attacks **variance**, so its base learners are deliberately low-bias/high-variance (deep trees). **Boosting** trains models **sequentially**, each one focusing on the errors the previous ones made, and adds them up. It attacks **bias**, so its base learners are deliberately weak (shallow trees, often depth 3 to 6). Consequence: adding trees to a random forest is safe, but adding too many to a boosted model **will** overfit, which is why boosting needs early stopping and bagging does not.

AdaBoost (Adaptive Boosting, 1995)

Each round, increase the weight of misclassified samples so the next weak learner concentrates on them. Each learner also gets a vote weight based on its accuracy. Base learner is usually a **decision stump** (depth-1 tree). Sensitive to noise and outliers, since they get up-weighted repeatedly.

Gradient Boosting

The generalisation: instead of reweighting samples, fit each new tree to the **negative gradient of the loss** with respect to the current predictions. For squared error, that gradient is simply the residual, so the intuitive story "each tree predicts the previous ensemble's errors" is exactly right for regression, and the gradient framing extends it to any differentiable loss.

```
prediction = F₀(x)                          (initial guess: the mean or log-odds)
for m in 1..M:
    residuals rᵢ = -∂L/∂F  at the current F
    fit tree hₘ to the residuals
    F(x) := F(x) + learning_rate * hₘ(x)
```

The **learning rate** (shrinkage) scales each tree's contribution. Lower learning rate plus more trees is nearly always better than the reverse: 0.01 to 0.1 with several hundred to several thousand trees is the standard recipe.

XGBoost, LightGBM, CatBoost

	XGBoost	LightGBM	CatBoost
Tree growth	level-wise (depth-first balanced)	**leaf-wise** (grows the highest-loss leaf)	symmetric / oblivious trees
Speed	fast	**fastest**, especially on large data	moderate
Memory	moderate	low (histogram binning, EFB)	higher
Categoricals	needs encoding (native support added recently)	native	**best in class: ordered target statistics**
Small data	good	**can overfit badly** (needs `num_leaves` / `min_data_in_leaf` control)	very good, strong defaults
Key innovation	regularised objective, second-order (Newton) boosting, sparsity-aware splits	GOSS + Exclusive Feature Bundling	**ordered boosting** to prevent target leakage
Overfit control	`lambda`, `alpha`, `gamma`, `subsample`, `colsample_bytree`	same family plus `num_leaves`	`l2_leaf_reg`, ordered boosting

KEY IDEA XGBoost's real innovation was putting the regularisation term into the objective itself and using a second-order Taylor expansion (gradients **and** Hessians) to choose splits, rather than only first-order gradients. LightGBM's is leaf-wise growth plus histogram binning, which is why it is so much faster but also more prone to overfitting on small data. CatBoost's is ordered boosting, which fixes the subtle target leakage that ordinary target encoding introduces. Naming the specific innovation for each is what a strong candidate does.

```python
import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=2000,
    learning_rate=0.03,
    max_depth=6,
    min_child_weight=1,
    subsample=0.8,            # row sampling per tree: adds bagging-style variance reduction
    colsample_bytree=0.8,     # feature sampling per tree
    reg_lambda=1.0,           # L2
    reg_alpha=0.0,            # L1
    gamma=0,                  # minimum loss reduction to make a split
    scale_pos_weight=neg/pos, # for imbalance
    eval_metric="aucpr",
    early_stopping_rounds=100,
    tree_method="hist",
    random_state=42,
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=100)
print("best iteration:", model.best_iteration)
```

```python
import lightgbm as lgb
model = lgb.LGBMClassifier(
    n_estimators=3000, learning_rate=0.03,
    num_leaves=31,            # THE key LightGBM knob; roughly 2^max_depth but tune it directly
    min_child_samples=20, subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0, random_state=42,
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)],
          callbacks=[lgb.early_stopping(100), lgb.log_evaluation(100)])
```

The tuning order that actually works

1. Set a low learning rate (0.03 to 0.05) and a high `n_estimators`, and use **early stopping** to find the right number of trees automatically.
2. Tune tree complexity: `max_depth` / `num_leaves`, then `min_child_weight` / `min_child_samples`.
3. Tune sampling: `subsample`, `colsample_bytree`.
4. Tune regularisation: `reg_lambda`, `reg_alpha`, `gamma`.

5. Finally, lower the learning rate further and raise the tree count for a last small gain.

5.10 Stacking and blending

Train several diverse base models, then train a **meta-model** on their out-of-fold predictions.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

stack = StackingClassifier(
    estimators=[("rf", rf), ("xgb", xgb_model), ("svm", svm)],
    final_estimator=LogisticRegression(),
    cv=5,                # base predictions generated out-of-fold: essential to avoid leakage
    passthrough=False,
)
```

The meta-model must be trained on **out-of-fold** predictions, otherwise the base models' training-set predictions are overfit and the meta-model learns to trust them wrongly. Keep the meta-model simple (logistic or ridge regression); a complex one just overfits the stack. Diversity of base models matters more than individual strength.

Voting is the simpler cousin: hard voting takes the majority label, soft voting averages the predicted probabilities (usually better, and requires calibrated models to be meaningful).

5.11 The model selection table

Model	Handles non-linearity	Needs scaling	Interpretable	Big data	Small data	Notes
Linear regression	no	if regularised	**very**	yes	yes	the baseline for regression
Logistic regression	no	if regularised	**very**	yes	yes	the baseline for classification; well calibrated
k-NN	yes	**yes**	somewhat	no	yes	no training, slow inference, curse of dimensionality
Naive Bayes	limited	no	yes	yes	**excellent**	text baseline; badly calibrated
SVM (RBF)	**yes**	**yes**	no	no ($O(n^2)$)	**yes**	strong on small/medium high-dimensional data
Decision tree	yes	no	**very**	yes	yes	high variance alone
Random Forest	yes	no	somewhat	yes	yes	robust, few knobs, hard to break
Gradient boosting	**yes**	no	somewhat	yes	yes	**usually the best on tabular data**
Neural network	**yes**	**yes**	no	**yes**	no	wins on images, text, audio; rarely on tabular

INTERVIEW QUESTION Q: You have a tabular dataset with 50,000 rows and 40 mixed-type features. Which model and why? I would start with two baselines on the same CV split: a majority-class or mean predictor to establish the floor, and a regularised logistic regression with proper preprocessing to establish a strong, interpretable reference. Then gradient boosting, LightGBM or XGBoost, because on tabular data of that size it is very reliably the best-performing family: it handles mixed types and non-linear interactions, is not sensitive to feature scaling, and handles missing values natively. I would tune it with early stopping on a validation fold. I would only reach for a neural network if I had a specific reason, such as high-cardinality categorical features I wanted to learn embeddings for, or a multi-modal input, because on plain tabular data of this size boosted trees typically match or beat deep nets at a fraction of the effort. And I would keep the logistic regression around, because if it is within one or two points of the boosted model, the interpretability may well be worth more to the business than the accuracy.

Part VI — Unsupervised Learning

6.1 K-Means

The algorithm

1. Choose k and initialise k centroids.
2. **Assign** each point to the nearest centroid.
3. **Update** each centroid to the mean of its assigned points.
4. Repeat 2 and 3 until assignments stop changing.

This is coordinate descent on the objective **WCSS** (within-cluster sum of squares, also called inertia): `Σ_clusters Σ_points ||x - μ_c||²`. It is guaranteed to converge, but only to a **local** optimum, which is why initialisation matters.

k-means++ initialisation spreads the initial centroids apart probabilistically and is the default in sklearn. `n_init=10` runs the whole thing ten times and keeps the best.

Choosing k

- **Elbow method**: plot inertia against k and look for the bend. Often ambiguous.
- **Silhouette score**: for each point, `(b - a) / max(a, b)` where a is the mean distance to its own cluster and b to the nearest other cluster. Pick the k with the highest mean silhouette. More reliable than the elbow.
- **Gap statistic**: compare inertia to that of uniformly random data.
- **Domain knowledge**: usually the real answer. "We need 5 customer segments because marketing can run 5 campaigns."

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

for k in range(2, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42).fit(X_scaled)
    print(k, km.inertia_, silhouette_score(X_scaled, km.labels_))
```

Assumptions and limits: K-Means assumes clusters are spherical, similar in size, and similar in density, because it uses Euclidean distance to a mean. It fails on elongated, nested, or crescent-shaped clusters, it is sensitive to outliers (a mean is not robust), and it forces every point into a cluster. It requires scaled features and it requires k up front.

Variants: MiniBatchKMeans (fast on big data), K-Medoids/PAM (uses actual data points as centres, robust to outliers), Kernel K-Means, Fuzzy C-Means (soft assignments).

6.2 Hierarchical clustering

Build a tree (dendrogram) of nested clusters. **Agglomerative** (bottom-up) is the common form: start with every point as its own cluster and repeatedly merge the two closest.

Linkage defines "closest":

Linkage	Distance between clusters	Behaviour
Single	closest pair	chaining, finds elongated shapes, sensitive to noise
Complete	furthest pair	compact, roughly equal-diameter clusters
Average	mean pairwise distance	a middle ground
Ward	the merge that minimises the increase in total within-cluster variance	the default; produces balanced, compact clusters

Advantages: no need to choose k in advance (cut the dendrogram wherever you like), and the dendrogram itself is informative. Disadvantage: $O(n^2)$ memory and $O(n^3)$ time in the naive form, so it does not scale beyond tens of thousands of points.

6.3 DBSCAN

Density-Based Spatial Clustering of Applications with Noise.

Two parameters: `eps` (the neighbourhood radius) and `min_samples` (how many points must be within eps to be dense).

- **Core point**: has at least `min_samples` neighbours within `eps`.
- **Border point**: within eps of a core point but not itself dense.
- **Noise**: neither. **DBSCAN labels these -1 and leaves them out**, which is its killer feature.

Clusters are connected components of core points plus their borders.

> **KEY IDEA — DBSCAN vs K-Means, a favourite comparison question** K-Means needs k in advance, forces every point into a cluster, and only finds spherical, similar-sized groups. DBSCAN infers the number of clusters from the data, finds arbitrarily shaped clusters, and explicitly labels outliers as noise. The trade-off is that DBSCAN struggles when clusters have very different densities (one eps cannot fit all) and it becomes hard to tune in high dimensions. **HDBSCAN** fixes the varying-density problem by building a hierarchy over densities, and is what you should reach for in practice.

Choosing eps: plot the sorted distance to each point's k-th nearest neighbour and look for the knee.

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5).fit(X_scaled)
print("clusters:", len(set(db.labels_)) - (1 if -1 in db.labels_ else 0))
print("noise points:", (db.labels_ == -1).sum())
```

6.4 Gaussian Mixture Models

Model the data as a weighted sum of k Gaussians. Each point gets a **soft** assignment: a probability of belonging to each component. Fitted with **Expectation-Maximisation**:

- **E-step**: given current parameters, compute each point's responsibility for each component.
- **M-step**: given responsibilities, update means, covariances, and mixing weights by weighted MLE.

Repeat until the log-likelihood converges.

GMM generalises K-Means: K-Means is the limit of a GMM with spherical, equal covariance and hard assignments. GMM's `covariance_type` ("full", "tied", "diag", "spherical") lets clusters be elliptical and differently shaped.

Choosing k here has a principled answer: minimise **BIC** or **AIC**, which trade log-likelihood against the number of parameters.

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=3, covariance_type="full", random_state=42).fit(X)
probs = gmm.predict_proba(X)
print(gmm.bic(X), gmm.aic(X))
```

6.5 PCA (Principal Component Analysis)

The idea

Find a new orthogonal coordinate system where the first axis captures the most variance, the second the most of what remains, and so on. Keep the first k axes.

The steps

1. **Standardise** the features (mandatory: PCA follows variance, and an unscaled large-magnitude feature will dominate).
2. Compute the covariance matrix.
3. Compute its eigenvectors (the principal components) and eigenvalues (the variance along each).
4. Sort by eigenvalue descending, keep the top k.
5. Project the data: `X_reduced = X_centred @ W_k`.

In practice, libraries do this via SVD on the centred data matrix rather than forming the covariance matrix, because it is more numerically stable.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)            # keep enough components for 95% of variance
X_pca = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_.cumsum())
print("components kept:", pca.n_components_)
```

Choosing k: the cumulative explained-variance threshold (90 to 99%), the scree-plot elbow, or Kaiser's rule (keep eigenvalues above 1 on standardised data).

Limits: PCA is linear, so it cannot unfold curved manifolds. Components are linear combinations of all original features, which destroys interpretability. It is unsupervised, so the directions of maximum variance are not necessarily the directions that predict your target: a low-variance component can be the one that matters. And it is sensitive to outliers.

> **INTERVIEW QUESTION Q: PCA vs LDA?** Both project data to a lower-dimensional space, but they optimise different things. PCA is **unsupervised** and finds the directions of maximum **variance**. LDA (Linear Discriminant Analysis) is **supervised** and finds the directions that maximise **between-class separation relative to within-class scatter**. So for a classification pipeline LDA often gives a better low-dimensional representation, but it is capped at (number of classes - 1) components and it assumes each class is Gaussian with a shared covariance. PCA has no such cap and no class assumption. In practice: PCA for compression, denoising, and visualisation; LDA when the goal is specifically to separate known classes.

6.6 The other dimensionality reduction methods

Method	Linear?	Supervised?	Preserves	Use for
PCA	yes	no	global variance	compression, denoising, decorrelation
SVD / Truncated SVD	yes	no	global structure	sparse data, LSA on TF-IDF (PCA needs centring which destroys sparsity)
LDA	yes	**yes**	class separation	supervised reduction before a classifier
t-SNE	no	no	**local** neighbourhoods	**visualisation only**
UMAP	no	optional	local **and** some global	visualisation, and as a preprocessing step
Autoencoder	no	no	whatever the loss demands	non-linear compression, learned features
ICA	yes	no	statistical independence	blind source separation (EEG, audio)
NMF	yes	no	non-negativity, parts-based	topic modelling, image parts

COMMON MISTAKE — t-SNE, and this is asked deliberately Reading meaning into t-SNE cluster **sizes** or the **distances between** clusters. t-SNE only preserves local neighbourhood structure; global distances and relative blob sizes are essentially arbitrary artefacts of the perplexity setting and the random seed. Also, t-SNE has no `transform` method for new points: it must be refit. Use it to look at your data, never as a feature-engineering step feeding a model. UMAP is faster, preserves more global structure, and does support transforming new points, which is why it has largely replaced t-SNE for anything beyond a single plot.

6.7 Anomaly detection

Method	Idea	Good for
Statistical (z-score, IQR, Grubbs)	far from the mean	univariate, roughly normal data
Isolation Forest	anomalies are easier to isolate with random splits, so they sit at shallow depths	the general-purpose default; fast, high-dimensional
Local Outlier Factor	compares a point's local density to its neighbours'	clusters of differing density
One-Class SVM	learn a boundary around the normal data	small, clean training data
Autoencoder reconstruction error	anomalies reconstruct badly	images, sequences, high-dimensional data
Elliptic Envelope	fits a robust Gaussian	roughly Gaussian data
DBSCAN noise labels	anything labelled -1	as a by-product of clustering

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.01, random_state=42).fit(X)
outliers = iso.predict(X) == -1        # -1 = anomaly, 1 = normal
scores = iso.score_samples(X)          # lower = more anomalous
```

SCENARIO *You need to detect fraud, but you have almost no labelled fraud examples. What do you do?* With essentially no positives, supervised learning is not viable, so I would frame it as anomaly detection: train an Isolation Forest or an autoencoder on the (mostly legitimate) traffic and score by how anomalous each transaction is. That gives a ranked queue for human reviewers. Crucially, I would treat those reviews as a labelling process: every case the analysts confirm or reject becomes ground truth, so within a few months I have enough labels to train a supervised model, which will substantially outperform the unsupervised one. That transition, unsupervised to human-in-the-loop labelling to supervised, is the standard path and describing it shows you have thought past the first model.

6.8 Association rule mining

For "customers who bought X also bought Y". Three quantities:

- **Support(X)** = fraction of transactions containing X. How common is it.
- **Confidence(X→Y)** = Support(X ∪ Y) / Support(X). Given X, how often does Y appear.
- **Lift(X→Y)** = Confidence(X→Y) / Support(Y). How much more likely Y is given X than at random. Lift > 1 means a positive association; lift = 1 means independence.

Apriori prunes the search using the fact that any superset of an infrequent itemset is also infrequent. **FP-Growth** is a faster tree-based alternative.

COMMON MISTAKE Quoting high confidence without lift. If 80% of all baskets contain bread, then "milk → bread" with 80% confidence is completely uninformative: lift is 1.0 and there is no association at all. Always report lift.

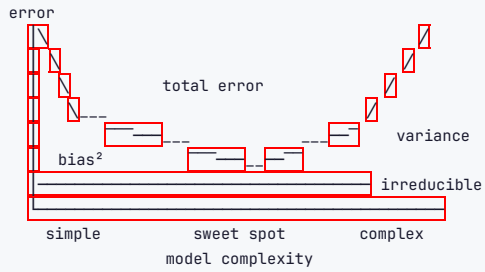
Part VII — The Theory That Gets Tested

7.1 The bias-variance decomposition

THE MATH For squared error, the expected test error at a point decomposes exactly:

$$E[(y - \hat{f}(x))^2] = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2$$

- **Bias**: error from wrong assumptions. The model is too simple to capture the truth. High bias = **underfitting**.
- **Variance**: error from sensitivity to the particular training sample. The model learns noise. High variance = **overfitting**.
- **Irreducible error σ^2**: noise in the data itself. No model can beat this. This term is why "100% accuracy" is usually a sign of leakage rather than brilliance.



	High bias (underfit)	High variance (overfit)
Train error	high	**low**
Validation error	high (and close to train)	**high** (and far from train)
Fixes	bigger model, more features, less regularisation, train longer, better features	**more data**, regularisation, simpler model, dropout, early stopping, bagging, data augmentation

KEY IDEA — the diagnostic that answers half of all debugging questions Look at the **gap** between training and validation error. Both high, small gap → **underfitting**, so increase capacity. Train low, validation high, big gap → **overfitting**, so regularise or get more data. Both low → done. Validation **lower** than train → you have a bug, or dropout/augmentation is active only during training (which is normal and expected in deep learning, so check before panicking), or the validation set is easier.

The modern caveat worth mentioning: the classic U-shaped curve is not the whole story for very large neural networks. **Double descent** is the observed phenomenon where test error goes down, then up around the interpolation threshold, then **down again** as the model becomes hugely over-parameterised. This is why models with billions of parameters trained on less data than parameters do not simply overfit into uselessness. Mentioning double descent signals that you read past the textbook.

7.2 Overfitting and underfitting: the full toolkit

Regularisation methods, all in one place:

Method	Applies to	Mechanism
L1 / L2 penalty	linear models, NNs (weight decay)	shrink or zero coefficients
Early stopping	any iterative learner	stop when validation stops improving
Dropout	neural networks	randomly zero activations, forcing redundancy
Data augmentation	images, audio, text	synthesise more training variety
Batch/layer norm	neural networks	stabilises training, mild regularisation effect
Pruning	trees, NNs	remove low-value structure
Ensembling / bagging	any	average away variance
Label smoothing	classification NNs	prevents over-confident logits
Noise injection	any	adds robustness
More data	any	the strongest fix there is

7.3 The curse of dimensionality

As dimensions grow: - The volume of the space grows exponentially, so any fixed dataset becomes hopelessly sparse. - All pairwise distances converge toward one another, so "nearest neighbour" stops being meaningful. - The number of samples needed to cover the space grows exponentially. - Almost all the volume of a high-dimensional ball sits near its surface, which breaks intuitions built in 2D.

Consequences: k-NN, K-Means, and RBF kernels degrade badly; you need dimensionality reduction, feature selection, or strong regularisation. It is also the reason vector databases need approximate nearest-neighbour indexes rather than exact search.

7.4 No Free Lunch

Averaged over **all** possible problems, every algorithm performs identically. There is no universally best model. This is why the honest answer to "which algorithm is best" is always "for what data, what metric, and what constraints," and why you should always try a few families rather than assuming.

7.5 Hyperparameter tuning

Parameters are learned from data (weights, coefficients, split thresholds). **Hyperparameters** are set by you before training (learning rate, depth, k, C, number of layers).

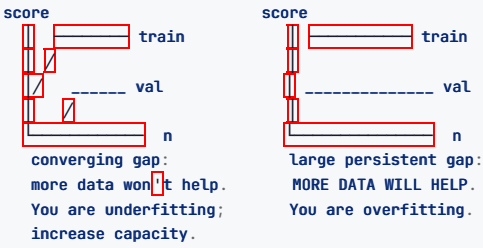
Strategy	How	When
Grid search	try every combination	few hyperparameters, small ranges
Random search	sample combinations randomly	**better than grid in almost all cases**: with a fixed budget it explores far more distinct values of each individual hyperparameter, and usually only a few of them matter
Bayesian optimisation (Optuna, Hyperopt)	build a surrogate model of the objective and sample where improvement is likely	expensive-to-train models; the professional default
Hyperband / Successive Halving	start many configs cheaply, kill the bad ones early	deep learning, large budgets
Population-based training	evolve configurations during training	RL and large-scale DL

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

search = RandomizedSearchCV(
    XGBClassifier(),
    {"max_depth": randint(3, 10),
     "learning_rate": loguniform(1e-3, 3e-1),
     "subsample": [0.6, 0.8, 1.0],
     "colsample_bytree": [0.6, 0.8, 1.0],
     "reg_lambda": loguniform(1e-2, 1e2)},
    n_iter=60, cv=5, scoring="average_precision", n_jobs=-1, random_state=42,
).fit(X_train, y_train)

# Optuna: the modern standard
import optuna
def objective(trial):
    params = {
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    }
    return cross_val_score(XGBClassifier(**params), X, y, cv=5,
                           scoring="average_precision").mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100)
print(study.best_params)
```

> **KEY IDEA — why random search beats grid search** Suppose you have 5 hyperparameters but only 2 actually matter. A 4-point grid over 5 dimensions costs 1,024 runs but only tries 4 distinct values of each important parameter. Random search with the same 1,024 runs tries 1,024 distinct values of each. Bergstra and Bengio's 2012 paper made exactly this argument, and it is a nice reference to name.

7.6 Learning and validation curves

Learning curve: performance versus training set size. It tells you whether more data would help.



Validation curve: performance versus a single hyperparameter. Shows you exactly where the bias-variance sweet spot is for that knob.

```
from sklearn.model_selection import learning_curve, validation_curve
sizes, train_sc, val_sc = learning_curve(model, X, y, cv=5,
                                         train_sizes=np.linspace(0.1, 1.0, 10))
train_sc, val_sc = validation_curve(model, X, y, param_name="max_depth",
                                    param_range=range(1, 21), cv=5)
```

INTERVIEW QUESTION Q: Your model gets 99% training accuracy and 70% validation accuracy. What do you do, in order? That gap is textbook overfitting, so I would work in order of cost. First, confirm it is real and not a split artefact by checking across CV folds and confirming the validation set is representative. Second, the cheapest fixes: turn up regularisation (L2 or the tree's `min_samples_leaf` / `max_depth`), and add early stopping. Third, reduce capacity or reduce the feature count, since with a big gap I suspect some features are effectively memorising, and I would check feature importances for anything suspiciously dominant, which would also be my leakage check. Fourth, more data or data augmentation, which is the most effective fix if it is available. Fifth, ensembling, since averaging reduces variance directly. Throughout I would plot the learning curve, because if the two curves are still converging as n grows, more data is the answer and I should not waste a week tuning regularisation instead.

Part VIII — Deep Learning Foundations

Everything from here builds toward the architectures you were asked about by name: ANN, CNN, RNN, LSTM, and the Transformer. This part is the machinery all of them share.

8.1 The perceptron

The 1958 ancestor of every neural network. It takes inputs, multiplies each by a weight, sums them, adds a bias, and passes the result through a step function.

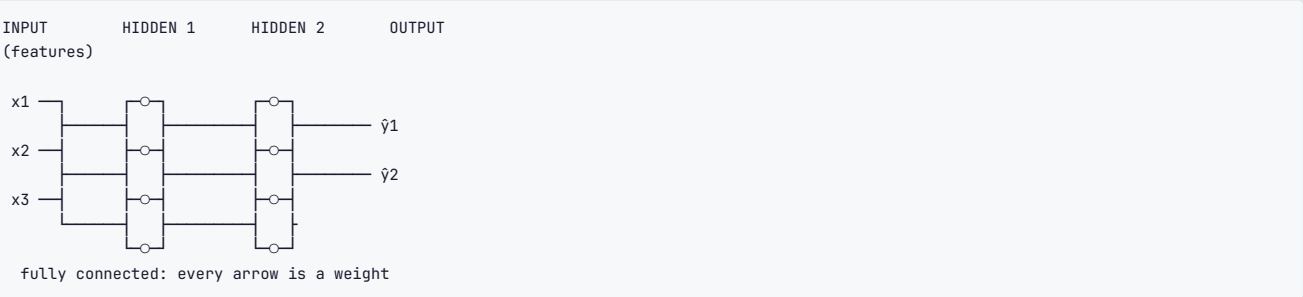
THE MATH `output = step(w·x + b)`, where `step(z) = 1 if z ≥ 0 else 0`.

Training rule: for each misclassified point, nudge the weights toward the correct answer: `w := w + n(y - ŷ)x`.

> **KEY IDEA — the limitation that caused the first AI winter** A single perceptron can only learn **linearly separable** functions. It cannot learn **XOR**, because no single straight line separates XOR's classes. Minsky and Papert proved this in 1969 and it froze neural network research for over a decade. The fix, discovered later, was to stack perceptrons into layers with a **non-linear** activation between them, which is the multi-layer perceptron. That historical arc is a great thing to be able to tell, because it explains *why* activation functions and hidden layers exist at all.

8.2 The Artificial Neural Network (ANN / MLP)

An **ANN**, also called a **Multi-Layer Perceptron (MLP)** or **feedforward network** or **fully-connected/dense network**, stacks layers of neurons. Each neuron in a layer connects to every neuron in the next.



One neuron computes `a = activation(w·x + b)`. **One layer** does this for all its neurons at once as a matrix multiply: `A = activation(XW + b)`. A **deep network** chains these:

$h_1 = f(XW_1 + b_1)$ → $h_2 = f(h_1W_2 + b_2)$ → … → $\hat{y} = \text{output\_activation}(h_nW_{n+1} + b_{n+1})$

> **KEY IDEA — the Universal Approximation Theorem** A feedforward network with a **single** hidden layer containing enough neurons can approximate any continuous function to arbitrary accuracy. So why go deep? Because "enough neurons" in one layer can mean an astronomically large number, whereas **depth** lets the network build features hierarchically (edges → shapes → objects), reusing lower-level features, which is exponentially more parameter-efficient. The theorem says a shallow net *can*; practice says a deep net does it with far fewer parameters. That is the standard answer to "why deep learning rather than one wide layer."

Output layer design, which interviewers check:

Task	Output neurons	Output activation	Loss
Regression	1	none (linear)	MSE / MAE / Huber
Binary classification	1	sigmoid	binary cross-entropy
Multi-class (single label)	k	softmax	categorical cross-entropy
Multi-label	k	sigmoid (per class)	binary cross-entropy per output

8.3 Activation functions

Without a non-linear activation, stacking layers is pointless: the composition of linear functions is still linear, so a 100-layer linear network has exactly the power of a single layer. **The activation is where the non-linearity, and therefore all the expressive power, comes from.**

Activation	Formula	Range	Pros	Cons
Sigmoid	$1/(1+e^{-z})$	(0,1)	probabilistic output	**vanishing gradient**, not zero-centred, saturates
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(-1,1)	zero-centred	still saturates and vanishes
ReLU	$\max(0, z)$	[0,∞)	fast, no saturation for z>0, sparse	**dying ReLU**, not zero-centred
Leaky ReLU	$\max(\alpha z, z)$, α≈0.01	(-∞,∞)	fixes dying ReLU	α is a hyperparameter
PReLU	leaky with learned α	(-∞,∞)	learns the slope	extra parameters
ELU	$z \text{ if } z>0 \text{ else } \alpha(e^{z}-1)$	(-α,∞)	smooth, closer to zero mean	slower (exp)
GELU	$z \cdot \Phi(z)$	≈[-0.17,∞)	smooth, **the transformer default**	slightly costlier
Swish/SiLU	$z \cdot \sigma(z)$	≈[-0.28,∞)	smooth, often beats ReLU	costlier
Softmax	$e^{z_i}/\Sigma e^{z}$	(0,1), sums to 1	multi-class output	output layer only

> **KEY IDEA — the vanishing gradient problem, asked constantly** Sigmoid and tanh saturate: for large positive or negative inputs their derivative approaches zero. During backpropagation, gradients are **multiplied** layer by layer via the chain rule, so many small derivatives multiply into a vanishingly tiny number, and the early layers receive almost no gradient and barely learn. **ReLU** largely solved this because its derivative is exactly 1 for all positive inputs, so it does not shrink the gradient. This single change, along with better initialisation, is a big part of why training deep networks became feasible around 2011.
>
> **COMMON MISTAKE — dying ReLU** If a neuron's weights update such that its input is always negative, ReLU outputs 0, its gradient is 0, and it never recovers: it is dead. This happens with too-high learning rates. Leaky ReLU, ELU, or GELU avoid it by allowing a small gradient for negative inputs. If a large fraction of your ReLU units are always zero, this is why.

8.4 Loss functions (deep learning)

Task	Loss	Formula (per sample)
Regression	MSE	$(y - \hat{y})^2$
Regression, robust	Huber	quadratic near 0, linear far out
Binary classification	Binary cross-entropy	$-(y \log p + (1-y) \log(1-p))$
Multi-class	Categorical cross-entropy	$-\Sigma\, y_k \log p_k$
Imbalanced classification	Focal loss	$-\alpha(1-p)^{\gamma} \log p$
Embeddings / similarity	Contrastive, Triplet, InfoNCE	pull positives together, push negatives apart
Segmentation	Dice loss	overlap-based
Ranking	Hinge, pairwise	margin-based

`nn.CrossEntropyLoss` in PyTorch combines `LogSoftmax` and `NLLLoss`, so you feed it **raw logits, not softmax probabilities**. Applying softmax yourself first is a very common bug that quietly hurts training.

8.5 Backpropagation, derived

This is the algorithm that makes training possible, and being able to explain it precisely is expected at every level.

> **KEY IDEA — one sentence** Backpropagation is the chain rule applied from the output back to the inputs, computing how much each weight contributed to the loss, so the optimiser can nudge each one to reduce it.

The four steps:

1. **Forward pass**: push the input through the network, computing and caching each layer's pre-activation z and activation a, ending in a prediction and a scalar loss L.
2. **Compute the output error**: $\partial L/\partial \hat{y}$. For softmax with cross-entropy this simplifies beautifully to $(\hat{y} - y)$.
3. **Backward pass**: propagate the error backward. For each layer, the chain rule gives the gradient of the loss with respect to that layer's weights, and the error to pass to the previous layer: - $\partial L/\partial W_l = \delta_l \cdot a_{l-1}^{T}$ - $\delta_{l-1} = (W_l^{T}\, \delta_l) \odot f'(z_{l-1})$ where ⊙ is element-wise multiply.
4. **Update**: $W := W - n \cdot \partial L/\partial W$ for every layer.

Notice the $f'(z)$ factor in step 3: that is exactly where the vanishing gradient comes from. If f' is small (saturated sigmoid), δ shrinks each layer.

```python
# Backprop by hand for a 2-layer network, to prove you understand it
import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))

# Forward
z1 = X @ W1 + b1;      a1 = sigmoid(z1)
z2 = a1 @ W2 + b2;     a2 = sigmoid(z2)          # prediction
loss = -np.mean(y*np.log(a2) + (1-y)*np.log(1-a2))

# Backward (chain rule, right to left)
dz2 = a2 - y                                     # dL/dz2 for BCE+sigmoid
dW2 = a1.T @ dz2 / n
db2 = dz2.mean(0)
da1 = dz2 @ W2.T
dz1 = da1 * a1 * (1 - a1)                         # * sigmoid'(z1)
dW1 = X.T @ dz1 / n
db1 = dz1.mean(0)

# Update
for p, g in [(W1,dW1),(b1,db1),(W2,dW2),(b2,db2)]:
    p -= lr * g
```

Modern frameworks build a **computation graph** during the forward pass and traverse it in reverse automatically (**reverse-mode automatic differentiation**). You never write the above; you write the forward pass and call `.backward()`.

8.6 Optimisers

All of them are variations on "step downhill", differing in how they set the step.

Optimiser	Idea	Notes
SGD	`w -= η·g`	simple, noisy, needs tuning
SGD + Momentum	accumulate a velocity of past gradients	rolls through small bumps and ravines; β≈0.9
Nesterov	look ahead before stepping	slightly better momentum
AdaGrad	per-parameter learning rate, scaled by accumulated squared gradients	learning rate decays to zero, so it stalls
RMSProp	AdaGrad with an exponential moving average instead of a sum	fixes the stalling; good for RNNs
Adam	momentum + RMSProp + bias correction	**the default**; robust, fast
AdamW	Adam with **decoupled** weight decay	the correct way to regularise Adam; the transformer standard
LAMB / LARS	layer-wise adaptive rates	very large batch training

THE MATH — Adam Maintains a first moment (mean of gradients, momentum) `m` and a second moment (uncentred variance) `v`, both exponential moving averages, bias-corrects them, and updates `w := w - η · m̂ / (√v̂ + ε)`. Typical: β_1=0.9, β_2=0.999, ε=1e-8.

> **KEY IDEA** Adam adapts the learning rate **per parameter** using the recent gradient magnitude, so parameters with large, noisy gradients take smaller steps and rarely-updated parameters take larger ones. That is why it "just works" with the default learning rate on most problems. Note AdamW versus Adam: plain Adam's L2 penalty interacts badly with the adaptive scaling, so AdamW **decouples** weight decay from the gradient update, which is why every modern transformer uses AdamW. Being able to state that distinction is a strong signal.

Learning rate schedules matter as much as the optimiser: step decay, cosine annealing, and especially **warmup** (start tiny, ramp up over the first few hundred steps, then decay), which is essential for training transformers stably.

8.7 Weight initialisation

If all weights start equal, every neuron in a layer computes the same thing and receives the same gradient forever: the **symmetry** never breaks. So we initialise randomly, but the *scale* is critical.

Scheme	Variance	Use with
Xavier/Glorot	`1/n_in` (or `2/(n_in+n_out)`)	tanh, sigmoid
He/Kaiming	`2/n_in`	**ReLU and its variants**
LeCun	`1/n_in`	SELU
Orthogonal	orthogonal matrix	RNNs

KEY IDEA Initialise too small and the signal shrinks toward zero as it passes through layers (vanishing); too large and it explodes. Xavier/He initialisation sets the variance so that the signal's variance is roughly preserved layer to layer. Use **He for ReLU networks**, Xavier for tanh. Getting this wrong is a common reason a deep network "won't train" while a shallow one does.

8.8 Normalisation layers

Layer	Normalises over	Best for	Notes
Batch Norm	the batch dimension, per feature	CNNs	needs a reasonably large batch; behaves differently in train vs eval; stores running stats
Layer Norm	the feature dimension, per sample	**transformers, RNNs**	batch-size independent; the transformer standard
Instance Norm	per sample, per channel	style transfer	
Group Norm	groups of channels	small-batch CNNs / detection	batch-independent alternative to BatchNorm
RMSNorm	like LayerNorm without centring	modern LLMs (LLaMA)	cheaper

KEY IDEA — why BatchNorm helps It keeps each layer's inputs in a stable distribution as training changes the earlier layers (the original "internal covariate shift" story), but the more accepted modern explanation is that it **smooths the loss landscape**, allowing higher learning rates and faster, more stable convergence. It also has a mild regularising effect because the batch statistics add noise. The catch, and a favourite question: BatchNorm behaves **differently in training** (uses batch statistics) **and inference** (uses stored running averages), which is exactly why you must call `model.eval()` before evaluating, and why it breaks with a batch size of 1.

8.9 Regularisation in deep networks

- **Dropout**: randomly zero a fraction p of activations each step, forcing the network not to rely on any single neuron. At test time, dropout is off and activations are scaled (or, in inverted dropout, scaled during training). Typical p: 0.2 to 0.5. This is why validation loss can be **lower** than training loss: dropout handicaps training but not evaluation.
- **Weight decay (L2)**: via AdamW.
- **Early stopping**: monitor validation loss, keep the best weights.
- **Data augmentation**: covered per-domain later.
- **Label smoothing**: replace hard 0/1 targets with 0.1/0.9, preventing over-confidence.
- **Gradient clipping**: cap the gradient norm to prevent exploding gradients (essential for RNNs).

8.10 The full training loop

```python
import torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, d_hidden), nn.BatchNorm1d(d_hidden), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(d_hidden, d_hidden), nn.BatchNorm1d(d_hidden), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(d_hidden, d_out),          # raw logits; no softmax here
        )
    def forward(self, x): return self.net(x)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = MLP(20, 128, 3).to(device)
criterion = nn.CrossEntropyLoss()               # expects logits + integer labels
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)

best_val, patience, bad = 1e9, 10, 0
for epoch in range(100):
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()                   # 1. clear old gradients
        logits = model(xb)                      # 2. forward
        loss = criterion(logits, yb)            # 3. loss
        loss.backward()                         # 4. backward
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # optional: clip
        optimizer.step()                        # 5. update
    scheduler.step()

    model.eval()                                # turns OFF dropout and BN updates
    val_loss = 0
    with torch.no_grad():                       # no gradient graph: faster, less memory
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            val_loss += criterion(model(xb), yb).item()
    if val_loss < best_val:
        best_val, bad = val_loss, 0
        torch.save(model.state_dict(), "best.pt")   # keep the best
    else:
        bad += 1
        if bad >= patience: break               # early stopping
```

COMMON MISTAKE — the top three PyTorch bugs, all in one place 1. Forgetting `optimizer.zero_grad()`. PyTorch **accumulates** gradients, so without clearing them each batch adds to the last and training silently produces nonsense (it does not crash). 2. Forgetting `model.eval()` at evaluation. Dropout stays on and BatchNorm keeps using batch stats, so your metrics are wrong. 3. Forgetting `torch.no_grad()` at evaluation. It still works but wastes memory building a graph you never use, and can cause out-of-memory errors on large validation sets.

Part IX — Convolutional Neural Networks

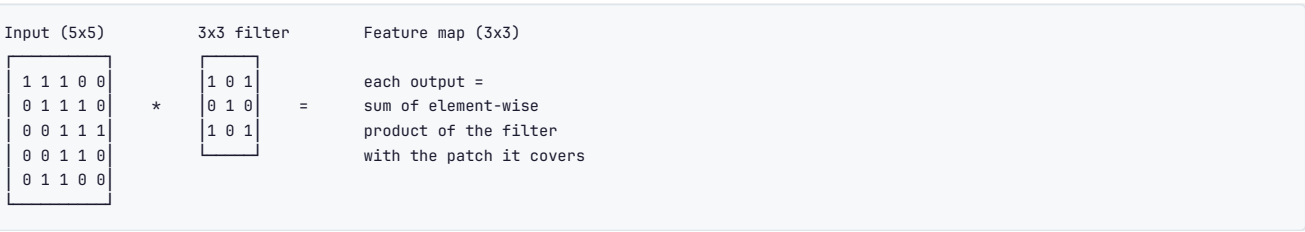
9.1 Why convolution

A dense layer on a 224×224×3 image would need over 150,000 weights **per neuron** in the first layer. That is absurd, it ignores spatial structure entirely, and it is not translation-invariant: a cat in the top-left and the same cat in the bottom-right would be completely unrelated inputs. CNNs fix all three problems with three ideas:

- **Local connectivity**: each neuron looks at a small patch, not the whole image.
- **Parameter sharing**: the same small filter slides across the whole image, so a feature detector learned in one place works everywhere. This is what gives **translation invariance** and slashes the parameter count.
- **Hierarchy**: early layers detect edges, later layers combine them into textures, then parts, then objects.

9.2 The convolution operation

A **filter** (kernel) is a small grid of learnable weights, for example 3×3. It slides across the input, and at each position computes the dot product of the filter with the patch beneath it, producing one number. The grid of all those numbers is a **feature map**.



Key terms:

- **Kernel size**: 3×3 is the modern default (two stacked 3×3s see the same region as one 5×5 but with fewer parameters and more non-linearity).
- **Stride**: how far the filter moves each step. Stride 2 halves the output size (downsampling).
- **Padding**: adding a border of zeros so the output stays the same size ("same" padding) rather than shrinking ("valid" padding).
- **Channels**: a colour image has 3 input channels; a filter spans all of them. A conv layer has many filters, each producing one output channel.

THE MATH — output size `out = floor((W - K + 2P) / S) + 1`, where W is input size, K kernel, P padding, S stride.

Parameter count of a conv layer: `(K × K × C_in + 1) × C_out`. Notice it does not depend on the image size, which is the whole point.

9.3 Pooling, padding, stride

Pooling downsamples a feature map, reducing computation and adding a little translation invariance.

- **Max pooling** (2×2): take the maximum in each window. The standard; keeps the strongest activation.
- **Average pooling**: take the mean. Smoother.
- **Global average pooling**: average each entire feature map to one number, replacing the giant flatten-then-dense layers of old architectures with far fewer parameters. Modern networks use this.

Pooling has no learnable parameters. Some modern architectures drop pooling in favour of strided convolutions.

9.4 CNN architectures, in historical order

Each one introduced an idea worth naming:

Architecture	Year	Key contribution
LeNet-5	1998	the original CNN; digit recognition
AlexNet	2012	ReLU, dropout, GPU training; **won ImageNet and started the deep learning era**
VGGNet	2014	depth with only 3×3 filters; simple and uniform
GoogLeNet/Inception	2014	the Inception module (parallel filters of several sizes); 1×1 convolutions for cheap channel mixing
ResNet	2015	**residual/skip connections**, enabling networks of 100+ layers
DenseNet	2017	connect every layer to every later layer
MobileNet	2017	depthwise separable convolutions for phones
EfficientNet	2019	compound scaling of depth, width, and resolution together
ConvNeXt	2022	a CNN redesigned with transformer-era tricks, competitive with ViT

KEY IDEA — ResNet and the skip connection, the single most important CNN idea to know Before ResNet, making networks deeper eventually made them **worse**, even on training data, because gradients degraded over many layers. The fix is the **residual connection**: instead of learning `H(x)`, a block learns the *residual* `F(x)` and outputs `F(x) + x`. The `+ x` gives the gradient a direct path (an identity shortcut) straight back through the block, so it cannot vanish, and if a layer is not useful the block can trivially learn `F(x)=0` and pass the input through unchanged. This is why 152-layer networks suddenly trained. The same skip-connection idea reappears inside every transformer block, so understanding it here pays off twice.

9.5 Transfer learning for vision

You almost never train a CNN from scratch. You take one pretrained on ImageNet (1.4M images) and adapt it.

```
import torch, torch.nn as nn
from torchvision import models

# 1. Load a pretrained backbone
model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

# 2. Freeze the feature extractor (keep what it learned)
for p in model.parameters():
    p.requires_grad = False

# 3. Replace the final layer with your own head (2 classes here)
model.fc = nn.Linear(model.fc.in_features, 2)   # only this trains at first

# 4. Train the head, then optionally UNFREEZE the top blocks and fine-tune
#    at a TINY learning rate (e.g. 1e-5) so you gently adjust, not bulldoze.
```

KEY IDEA The pretrained network already knows edges, textures, shapes, and object parts, learned from millions of images. You are not teaching it to see; you are teaching it that this particular combination means "your class." That takes hundreds of examples, not millions. **Feature extraction** (freeze everything, train a new head) suits small or similar datasets; **fine-tuning** (unfreeze some layers at a low learning rate) suits larger or more different datasets. The tiny learning rate in fine-tuning matters because the pretrained weights are already good and a large step would destroy them.

9.6 Object detection

Classification says *what*; detection says *what and where*, drawing bounding boxes.

Family	Members	Trade-off
Two-stage	R-CNN → Fast R-CNN → Faster R-CNN	propose regions, then classify them. Accurate, slower
One-stage	**YOLO** (v1–v11), SSD, RetinaNet	predict boxes and classes in a single pass. Fast, real-time
Transformer-based	DETR	detection as set prediction, no hand-designed anchors

Key concepts asked about: **IoU** (intersection over union, the overlap metric for boxes), **Non-Max Suppression** (removing duplicate overlapping boxes for the same object), **anchor boxes** (predefined box shapes), and **mAP** (mean average precision, the standard detection metric, averaging AP across classes and IoU thresholds).

KEY IDEA YOLO ("You Only Look Once") reframed detection from "run a classifier over many regions" to "one network predicts all boxes and classes in a single forward pass over a grid." That is why it runs in real time. RetinaNet's contribution was **focal loss**, which solved the extreme foreground-background imbalance that had held one-stage detectors back.

9.7 Segmentation

Per-pixel classification.

- **Semantic segmentation**: label every pixel by class (road, car, sky), but do not distinguish two cars. Architectures: **U-Net** (encoder-decoder with skip connections, dominant in medical imaging), FCN, DeepLab (atrous/dilated convolutions).
- **Instance segmentation**: distinguish individual objects (car 1 vs car 2). **Mask R-CNN** adds a mask-prediction branch to Faster R-CNN.
- **Panoptic**: both together.

U-Net's skip connections carry fine spatial detail from the encoder directly to the decoder, which is why its boundaries are sharp; this is the same skip-connection idea again.

9.8 Vision Transformers (ViT)

Split an image into fixed patches (say 16×16), flatten each patch, linearly embed it into a token, add positional encodings, and feed the sequence into a standard transformer encoder (Part XI). A special classification token's final embedding is used for the prediction.

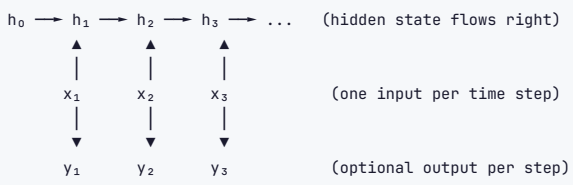
> **KEY IDEA — CNN vs ViT, a current and popular question** CNNs have strong **inductive biases** built in: locality and translation equivariance. That makes them **data-efficient**, working well on modest datasets. ViTs have almost no such bias, so they need either **huge** datasets or heavy augmentation and regularisation to match CNNs, but given that data they scale better and reach higher ceilings, because they can model long-range global relationships from layer one rather than building up a receptive field gradually. The practical summary: CNNs (or hybrids like ConvNeXt) for smaller data, ViTs when data and compute are abundant. Hybrid architectures now blur the line.
>
> **INTERVIEW QUESTION Q: Why do CNNs use parameter sharing, and what property does it give the model?** A single filter is slid across the entire image, so the same weights detect the same feature wherever it appears. This gives **translation equivariance**: shift the input and the feature map shifts identically. It also cuts the parameter count enormously compared to a dense layer, which both reduces overfitting and makes training on images tractable. Pooling then adds a degree of translation *invariance* on top. Without parameter sharing you would need to relearn "edge detector" separately for every location, which is both wasteful and would demand far more data.

Part X — Recurrent Networks and Sequences

This part covers the models you named specifically: RNN, LSTM, and GRU. They process sequences, one element at a time, carrying a memory forward.

10.1 The RNN

A feedforward network has no memory: each input is independent. For sequences (text, time series, audio, DNA) order matters, and the meaning of the current element depends on what came before. An **RNN (Recurrent Neural Network)** adds a loop: at each time step it combines the current input with a **hidden state** carried over from the previous step.



THE MATH $h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$ $y_t = W_{hy} \cdot h_t + b_y$

> **KEY IDEA — the crucial point about RNNs** The **same** weight matrices are reused at every time step. This is parameter sharing across time, the temporal analogue of the CNN's parameter sharing across space, and it is what lets an RNN handle sequences of any length with a fixed number of parameters. The hidden state h_t is a compressed summary of everything seen up to step t.

Input/output configurations:

Type	Example
one-to-one	standard classification (not really recurrent)
one-to-many	image captioning (one image → a sentence)
many-to-one	sentiment classification (a sentence → one label)
many-to-many (aligned)	part-of-speech tagging (one label per word)
many-to-many (seq2seq)	translation (a sentence → a differently-sized sentence)

10.2 Backpropagation Through Time, and the vanishing gradient

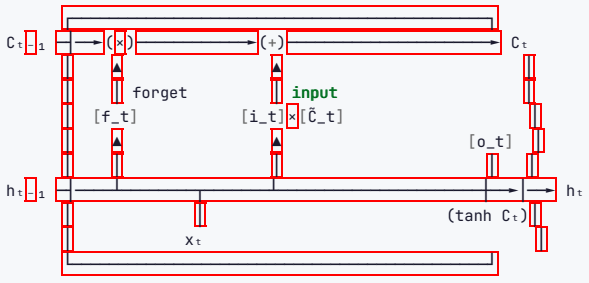
To train an RNN you **unroll** it into a deep feedforward network, one layer per time step, and backpropagate: this is **BPTT (Backpropagation Through Time)**.

> **KEY IDEA — why plain RNNs fail, the single most important thing in this part** Because the hidden state is multiplied by the same weight matrix at every step, the gradient over many steps involves that matrix raised to a high power. If its relevant values are below 1, the gradient shrinks exponentially toward zero (**vanishing gradient**), and the network cannot learn dependencies more than a handful of steps apart, so it forgets the start of a long sentence. If they are above 1, the gradient explodes to infinity (**exploding gradient**), and training diverges. Exploding gradients are patched with **gradient clipping**; vanishing gradients need an architectural fix, which is exactly what the LSTM provides.

10.3 The LSTM cell, gate by gate

LSTM (Long Short-Term Memory), 1997, solves the vanishing gradient with a separate **cell state** that runs straight through the sequence like a conveyor belt, modified only by carefully controlled **gates**. This gives gradients a near-uninterrupted highway backward through time, the same idea as a ResNet skip connection but for sequences.

Each cell maintains two things passed to the next step: the **cell state** `C` (long-term memory) and the **hidden state** `h` (short-term / working memory, and the output).



The three gates, each a sigmoid producing values in (0,1) that act as "how much to let through" dials:

Gate	Formula	Job
Forget f_t	$\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	what to erase from the cell state
Input i_t	$\sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	what new information to write
Candidate $\tilde{C}_t$	$\tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	the candidate values to write
Output o_t	$\sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	what part of the cell state to expose as output

The updates: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ (forget some old, add some new) $h_t = o_t \odot \tanh(C_t)$

> **KEY IDEA — why this solves vanishing gradients** The cell state update is **additive** ($C_t = f_t \cdot C_{t-1} + \ldots$), not repeatedly multiplicative through a squashing non-linearity. When the forget gate stays near 1, the gradient flows backward through the cell state almost unchanged over many steps, so long-range dependencies survive. The gates *learn* what to remember and for how long. This is the answer to "how does an LSTM fix the vanishing gradient problem," and it is asked in essentially every sequence-modelling interview.

10.4 The GRU

GRU (Gated Recurrent Unit), 2014, is a streamlined LSTM: it merges the cell and hidden state into one, and uses two gates instead of three.

Gate	Job
Reset r_t	how much past state to forget when computing the candidate
Update z_t	how much to blend old state with new candidate (does the job of both LSTM's forget and input gates)

$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

> **KEY IDEA — LSTM vs GRU, a standard question** The GRU has fewer parameters (two gates, one state), so it trains faster and needs less data, and it often matches the LSTM's performance. The LSTM's separate cell state and extra gate give it slightly more expressive control, which can help on very long or complex sequences. The honest interview answer: they are usually comparable, so try both; use GRU when data or compute is limited, LSTM when you need the extra capacity and can afford it. And note that in modern practice **transformers have largely replaced both** for text, because they parallelise and capture long-range dependencies better, which is the natural bridge to the next part.

10.5 Bidirectional and stacked RNNs

- **Bidirectional RNN**: run one RNN forward and another backward over the sequence, then concatenate their hidden states. Each position now has context from **both** directions. Great for tasks where the whole sequence is available (named-entity recognition, classification); impossible for real-time generation where the future is not yet known.
- **Stacked (deep) RNN**: feed one RNN's outputs as another's inputs, building a hierarchy of temporal features, exactly like stacking CNN layers.

10.6 Sequence-to-sequence (encoder-decoder)

For mapping one sequence to a different-length sequence (translation, summarisation): an **encoder** RNN reads the whole input and compresses it into a fixed **context vector**; a **decoder** RNN generates the output from that vector, one token at a time.

> **KEY IDEA — the bottleneck that led to attention** Cramming an entire input sentence into a single fixed-size context vector is a brutal bottleneck: for a long sentence, information from the start is lost by the time the decoder needs it. Performance drops sharply as input length grows. The fix, **attention**, lets the decoder look back at **all** of the encoder's hidden states and focus on the relevant ones at each output step. This is the innovation that made neural machine translation work, and it is the direct ancestor of the transformer.

10.7 Attention (the pre-transformer form)

At each decoding step, compute a relevance score between the decoder's current state and every encoder hidden state, turn those scores into weights with a softmax, and take the weighted sum of encoder states as a **context vector** tailored to this step.

$\text{attention weights} = \text{softmax}(\text{scores})$ → $\text{context} = \Sigma\ (\text{weight}_i \times \text{encoder\_hidden}_i)$

The decoder now dynamically attends to different input words as it generates each output word, aligning "chat" with "cat" and so on. This is **cross-attention**. The transformer's leap was to realise you could throw away the RNN entirely and build the whole model out of attention, which is Part XI.

INTERVIEW QUESTION Q: Why did transformers replace RNNs and LSTMs for most NLP? Three reasons. First, **parallelism**: an RNN must process a sequence step by step because each hidden state depends on the previous one, so it cannot use the GPU's parallelism over the sequence, whereas a transformer processes all positions simultaneously, making training dramatically faster. Second, **long-range dependencies**: in an RNN, information between two distant positions must survive many sequential steps and tends to decay, while self-attention connects any two positions directly in one step, a constant path length regardless of distance. Third, **scalability**: that parallelism and those direct connections let transformers scale to billions of parameters and enormous datasets, which is what produced modern LLMs. The trade-off is that self-attention is quadratic in sequence length, which is why very long contexts are expensive and an active research area.

Part XI — Transformers and Large Language Models

This connects directly to your first guide: that guide taught you to *build with* LLMs; this part is what they are inside. "Attention Is All You Need" (Vaswani et al., 2017) is the foundational paper.

11.1 Self-attention, derived

Self-attention lets every token in a sequence look at every other token and decide how much each one matters for its own representation. "The animal didn't cross the street because **it** was tired" — self-attention is what lets "it" attend strongly to "animal."

Each token's embedding is projected into three vectors by learned weight matrices:

- **Query (Q)**: what this token is looking for.
- **Key (K)**: what this token offers, for others to match against.
- **Value (V)**: the actual information this token passes on if attended to.

> **ANALOGY — the library/retrieval analogy interviewers like** Think of a search. Your **query** is what you type. Each document has a **key** (its title/index) that your query is matched against, and a **value** (its contents) that you actually retrieve. Attention scores every key against your query, softmaxes the scores into weights, and returns a weighted blend of the values. Self-attention does this with every token simultaneously acting as a query against all tokens' keys. This is also exactly why vector search in the RAG guide feels similar: both are query-key matching followed by value retrieval.

THE MATH — scaled dot-product attention, the one formula to memorise $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$

Step by step: QK^T gives every query-key similarity (a scores matrix); dividing by $\sqrt{d_k}$ keeps the scores from growing large with dimension (which would push softmax into saturation and kill gradients); softmax turns each row into attention weights that sum to 1; multiplying by V produces each token's output as a weighted blend of all values.

> **KEY IDEA — why divide by √d_k, a specific and common question** For large d_k, the dot products QK^T have large variance, pushing softmax into regions where it is nearly one-hot and its gradient is almost zero. Scaling by $\sqrt{d_k}$ normalises the variance back to roughly 1, keeping softmax in a well-conditioned range so gradients flow. Without it, deep transformers train poorly.

```python
import torch, torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / d_k**0.5     # (..., seq, seq)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))  # causal / padding mask
    weights = F.softmax(scores, dim=-1)
    return weights @ V, weights
```

11.2 Multi-head attention

Instead of one attention operation, run several ("heads") in parallel, each with its own Q/K/V projections, then concatenate and project the results. Each head can specialise: one might track syntactic subject-verb links, another coreference, another local phrasing.

$\text{MultiHead} = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$, where each head is scaled dot-product attention on its own projections.

> **KEY IDEA** Multiple heads let the model attend to information from different representation subspaces at once. It is the attention analogue of using many filters in a CNN layer: each learns to detect a different kind of relationship. The total dimension is split across heads, so it costs roughly the same as one full-width head but is far more expressive.

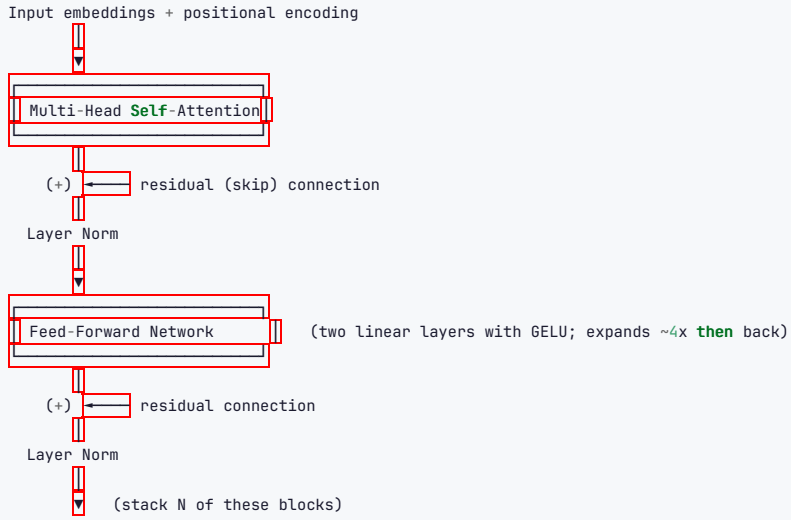
11.3 Positional encoding

Self-attention is **permutation-invariant**: it treats the input as a set, so on its own it has no idea of word order. We inject order by adding a **positional encoding** to each token embedding.

- **Sinusoidal** (original): fixed sine and cosine waves of different frequencies. No parameters, extrapolates to unseen lengths.
- **Learned**: a trainable embedding per position. Simple, but capped at the trained length.
- **Rotary (RoPE)** and **ALiBi**: modern relative schemes used in current LLMs (LLaMA, etc.) that generalise better to long contexts.

> **KEY IDEA** Removing positional encoding turns "dog bites man" and "man bites dog" into identical inputs, because attention alone sees an unordered set. This is a favourite gotcha question: *what happens if you remove positional encodings from a transformer?* The answer is that it loses all notion of order and can no longer distinguish sequences that differ only in arrangement.

11.4 The full transformer block



Note the two ideas reused from earlier parts: **residual connections** (the ResNet idea, giving gradients a highway through many blocks) and **layer normalisation** (chosen over batch norm because it is batch-size independent and works per token). The feed-forward network processes each position independently and is where much of the model's capacity and factual storage lives.

The original architecture had an **encoder** (bidirectional attention, sees the whole input) and a **decoder** (causal/masked attention, only sees previous tokens, plus cross-attention to the encoder). Modern models pick a side:

Type	Attention	Sees	Best at	Examples
Encoder-only	bidirectional	whole input	understanding: classification, NER, embeddings	BERT, RoBERTa
Decoder-only	causal (masked)	previous tokens only	generation	GPT, LLaMA, Mistral
Encoder-decoder	both	input fully, output causally	seq2seq: translation, summarisation	T5, BART

11.5 BERT, GPT, T5

- **BERT** (encoder-only): pretrained with **Masked Language Modelling** (predict 15% randomly masked tokens using both-side context) and Next Sentence Prediction. Bidirectional, so excellent for understanding tasks and for producing embeddings, but it cannot generate text left to right.
- **GPT** (decoder-only): pretrained with **next-token prediction** (causal language modelling). Predicts the next token given all previous ones. This simple objective, scaled up, is what powers modern generative LLMs.
- **T5** (encoder-decoder): frames **every** task as text-to-text ("translate English to German: ..." → the translation), unifying classification, translation, and summarisation under one objective.

> **KEY IDEA — the causal mask, and why it matters** A decoder uses a **causal (look-ahead) mask** that sets attention scores to future positions to negative infinity before the softmax, so each token can only attend to itself and earlier tokens. This is what makes autoregressive generation valid: during training the model predicts every next token in parallel without cheating by seeing the answer, and at inference it generates left to right. Explaining the causal mask crisply is a strong signal in an LLM interview.

11.6 Tokenisation

Models do not read characters or words; they read **tokens** (subword units), roughly 3-4 characters each. Subword tokenisation is a compromise between character-level (tiny vocabulary, very long sequences) and word-level (huge vocabulary, cannot handle unseen words).

Algorithm	Used by	Idea
BPE (Byte-Pair Encoding)	GPT	start from characters, greedily merge the most frequent adjacent pair, repeat
WordPiece	BERT	like BPE but merges by likelihood gain
Unigram / SentencePiece	T5, LLaMA	start big, prune tokens that hurt likelihood least; language-agnostic

Consequences you already met in the first guide: you pay per token, rare words split into more tokens, and non-English text is often less efficient.

11.7 Pretraining, SFT, RLHF, DPO

The modern LLM training pipeline has distinct stages, and knowing the order is expected:

1. **Pretraining**: self-supervised next-token prediction on trillions of tokens of internet text. Produces a "base model" that is knowledgeable but not helpful or aligned. This is the vastly expensive stage.
2. **Supervised Fine-Tuning (SFT)**: fine-tune on curated (instruction, good response) pairs so the model learns to follow instructions and adopt a helpful format.
3. **Preference alignment**, making outputs match human preferences: - **RLHF (Reinforcement Learning from Human Feedback)**: train a **reward model** on human comparisons of response pairs, then optimise the LLM against that reward with **PPO**, keeping a **KL penalty** to the SFT model so it does not drift into gibberish that games the reward. (This is the KL divergence from Part I, doing real work.) - **DPO (Direct Preference Optimisation)**: a newer, simpler method that skips the separate reward model and RL loop, optimising a classification-style loss directly on preference pairs. More stable and cheaper, increasingly the default.

> **KEY IDEA — RAG vs fine-tuning, tying back to the first guide** This is worth restating because it unifies both guides. **Pretraining and fine-tuning change the model's weights**; **RAG changes what you put in the prompt at inference**. Fine-tuning teaches *behaviour, format, and style*; RAG supplies *facts, freshness, and citations*. If you need the model to answer in your company's tone and JSON schema, fine-tune. If you need it to know today's inventory, use RAG. Most teams who think they need fine-tuning actually need RAG, and production systems often use both.

11.8 PEFT and LoRA

Full fine-tuning of a large model updates billions of parameters, needing enormous memory. **Parameter-Efficient Fine-Tuning (PEFT)** updates only a tiny fraction.

LoRA (Low-Rank Adaptation): freeze the original weights and inject small trainable low-rank matrices alongside them. Instead of updating a `d×d` weight matrix, learn two small matrices `A (d×r)` and `B (r×d)` with rank `r` far smaller than d, so the update is `BA`. This can train under 1% of the parameters with performance close to full fine-tuning, and the adapters are tiny to store and swap.

QLoRA adds 4-bit quantisation of the frozen base model, letting you fine-tune very large models on a single GPU.

> **KEY IDEA** LoRA works because the *update* needed to adapt a pretrained model to a task has low intrinsic rank: it lives in a small subspace, so two thin matrices capture it. You keep the base model frozen and shared, and each task is just a small adapter. This is why "fine-tune a 70B model on one GPU" is now possible, and naming the low-rank-update insight is what separates understanding from buzzword-dropping.

11.9 Inference optimisation

The concepts behind making LLMs fast and cheap to serve (the engineering half of the first guide's deployment story):

- **KV cache**: during generation, cache the keys and values of past tokens so each new token does not recompute attention over the whole history. The main memory consumer at inference.
- **Quantisation**: store weights in 8-bit or 4-bit instead of 16/32-bit. Big memory and speed win for a small quality cost.
- **Flash Attention**: an exact attention implementation that is IO-aware, avoiding materialising the full attention matrix in slow memory. Faster and far more memory-efficient.
- **Speculative decoding**: a small draft model proposes several tokens, the big model verifies them in one pass; accepted tokens come "for free."
- **Batching / continuous batching** (vLLM's PagedAttention): pack many requests together to keep the GPU busy, the key to cheap serving.
- **Mixture of Experts (MoE)**: only a subset of the network's "expert" subnetworks activates per token, so a model can have huge total capacity while only using a fraction per forward pass (Mixtral, and reportedly the largest models).

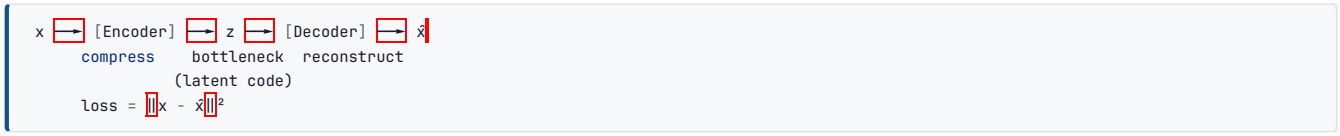
> **INTERVIEW QUESTION Q: Walk me through what happens, mechanically, when an LLM generates one token.** The input text is tokenised into subword IDs and embedded, with positional information added. The sequence passes through the stack of transformer blocks: in each, multi-head self-attention (with a causal mask so tokens only see earlier ones) lets every position gather relevant context, then a position-wise feed-forward network transforms it, each sublayer wrapped in a residual connection and layer norm. The final layer's output for the **last** position goes through a linear projection to vocabulary-sized logits, and a softmax turns those into a probability distribution over the next token. A decoding strategy picks one (greedy, or sampling with temperature and top-p), that token is appended, and the whole thing repeats, reusing the KV cache so only the new token is computed. Generation stops at an end-of-sequence token or a length limit.

Part XII — Generative Models

Discriminative models draw boundaries; generative models learn the data distribution so they can **produce new samples**. The families below are what "Generative AI" means beyond LLMs.

12.1 Autoencoders

An autoencoder learns to compress data and reconstruct it. An **encoder** maps input to a small **latent** (bottleneck) representation; a **decoder** reconstructs the input from it. Trained to minimise reconstruction error, with no labels: it is self-supervised.



The bottleneck forces the network to keep only the most informative features. Uses: **dimensionality reduction** (a non-linear PCA), **denoising** (train to reconstruct clean images from corrupted inputs), and **anomaly detection** (anomalies reconstruct poorly, so high reconstruction error flags them).

A plain autoencoder is **not** generative: its latent space has gaps, so sampling a random z usually decodes to garbage. Fixing that is the VAE.

12.2 Variational Autoencoders (VAE)

A VAE makes the latent space **continuous and well-structured** so you can sample from it. Instead of encoding to a single point, the encoder outputs a **distribution** (a mean and variance) per input; you sample z from it and decode.

THE MATH — the loss has two terms `Loss = Reconstruction term + KL divergence term`

- **Reconstruction**: decoded output should match the input (as before).
- **KL divergence**: the encoded distribution should stay close to a standard normal `N(0, I)`. This regularises the latent space into a smooth, gap-free region you can sample from.

The **reparameterisation trick** makes this trainable: instead of sampling z directly (which is not differentiable), write `z = μ + σ ⊙ ε` where `ε ~ N(0,I)`, moving the randomness outside the gradient path so backprop can flow through μ and σ.

> **KEY IDEA** The KL term is the difference between an autoencoder and a VAE. It pulls every input's latent distribution toward a common standard normal, so the latent space has no holes, which means a randomly sampled z decodes to something realistic. That is what makes a VAE generative. VAEs produce slightly blurry samples (a consequence of the reconstruction loss and the Gaussian assumptions) but have stable training and a meaningful, smooth latent space you can interpolate through.

12.3 GANs (Generative Adversarial Networks)

Two networks in a contest (Goodfellow et al., 2014):

- **Generator G**: takes random noise and produces fake samples, trying to fool the discriminator.
- **Discriminator D**: takes a sample and predicts real or fake, trying not to be fooled.

They train together in a **minimax game**: G improves at faking, D improves at detecting, and at equilibrium G's outputs are indistinguishable from real data.

> **KEY IDEA** GANs produce the **sharpest** images of the classic generative families because the discriminator provides a learned, adaptive loss rather than a fixed pixel-wise one, so it penalises exactly the artefacts that look fake. The price is **notoriously unstable training**: mode collapse (G produces only a few outputs that reliably fool D), vanishing discriminator gradients, and failure to converge. Much GAN research is stabilisation: Wasserstein GAN (a better loss with meaningful gradients), gradient penalty, spectral normalisation, progressive growing (StyleGAN). Naming mode collapse and WGAN as its fix is the expected depth.

12.4 Diffusion models

The technology behind Stable Diffusion, DALL-E, Midjourney, and Imagen, and now the dominant image generator.

- **Forward process**: take a real image and add a small amount of Gaussian noise, repeatedly, over many steps, until it is pure static. This is fixed, not learned.
- **Reverse process**: train a neural network (typically a **U-Net**) to **remove** a little noise at each step, i.e. to predict the noise that was added. Run it

starting from pure noise and it gradually **denoises** into a coherent image.

- **Text conditioning**: guide each denoising step with a text embedding (often from **CLIP**) so the image forms toward the prompt. **Classifier-free guidance** strengthens how closely the output follows the prompt.
- **Latent diffusion** (what makes Stable Diffusion efficient): run the whole process in a compressed latent space (from a VAE) rather than on full-resolution pixels, drastically cutting cost.

> **ANALOGY** A diffusion model is a sculptor working in reverse. Instead of a finished statue slowly being buried in sand (the forward noising), the model learns to brush sand away grain by grain (the reverse denoising) until a statue emerges from a featureless pile, with the text prompt telling it which statue to uncover.
>
> **KEY IDEA — the parallel to everything else in this guide** Diffusion is still "train a network to minimise a loss with gradient descent." The novelty is only *what* it is trained to predict: the noise to remove at each step. It beats GANs on training stability (no adversarial game, just a regression loss) and on diversity and mode coverage, at the cost of slow sampling because it needs many denoising steps, which is why speed-ups (DDIM, distillation, consistency models) are an active area.

12.5 Normalising flows

Learn an **invertible** transformation from a simple distribution (a Gaussian) to the data distribution. Because the mapping is invertible with a tractable Jacobian, flows can compute **exact** likelihoods, which VAEs (which only bound the likelihood) and GANs (which give no likelihood) cannot. The invertibility constraint limits architectural flexibility, so they are less common for images but valued where exact density matters.

12.6 Comparison

Model	How it generates	Sample quality	Training	Likelihood	Key weakness
Autoencoder	decode a latent	n/a (not generative)	stable	no	latent space has gaps
VAE	sample latent → decode	good, slightly blurry	**stable**	approximate (lower bound)	blurriness
GAN	noise → generator	**sharpest**	**unstable** (mode collapse)	no	training difficulty, mode collapse
Diffusion	iterative denoising	**excellent + diverse**	stable	approximate	**slow sampling** (many steps)
Normalising flow	invertible transform	good	stable	**exact**	architectural constraints

> **INTERVIEW QUESTION Q: GANs vs VAEs vs diffusion, when would you pick each?** VAE when I want a smooth, meaningful latent space to interpolate or manipulate, and I can tolerate some blur; training is stable and cheap. GAN when sample sharpness is paramount and I can invest in stabilising training; historically the best for high-resolution faces via StyleGAN. Diffusion when I want the best quality **and** diversity with stable training, which is why it now dominates text-to-image; the cost is slow, multi-step sampling, though distillation and consistency models are closing that gap. If I needed exact likelihoods, for instance for anomaly detection or density estimation, I would use a normalising flow. The current default for high-quality image generation is diffusion, in a latent space for efficiency.

Part XIII — NLP Beyond LLMs

The classical NLP that still underpins production systems and still gets asked about.

13.1 Text preprocessing

The traditional pipeline (much of which modern transformers skip, because subword tokenisation and learned representations handle it, but which you must still know):

- **Tokenisation**: split into words or subwords.
- **Lowercasing**: reduces vocabulary, but can lose meaning (US vs us, Apple vs apple).
- **Stop-word removal**: drop "the", "is", "at". Helpful for bag-of-words, but **harmful for transformers**, which use those words for structure.
- **Stemming**: crudely chop suffixes ("running" → "run", "studies" → "studi"). Fast, produces non-words.
- **Lemmatisation**: map to the dictionary base form using vocabulary and part-of-speech ("better" → "good", "was" → "be"). Slower, linguistically correct.
- **N-grams**: contiguous sequences of n tokens, to capture short phrases ("New York") that single words miss.

> **KEY IDEA** Stemming vs lemmatisation is a common quick question. Stemming is fast and rule-based and can output non-words; lemmatisation is slower, dictionary-based, and outputs real base forms using context. Also worth saying: with modern transformer models you generally do **not** stem, lemmatise, or remove stop words, because subword tokenisation plus contextual embeddings use that information. Those steps belong to the TF-IDF / bag-of-words era.

13.2 Bag-of-Words and TF-IDF

- **Bag-of-Words**: represent a document as a vector of word counts, ignoring order. Simple, but treats "not good" and "good" as sharing the positive word, and it is high-dimensional and sparse.
- **TF-IDF (Term Frequency-Inverse Document Frequency)**: weight each word by how often it appears in the document (TF) times how *rare* it is across all documents (IDF). Common words like "the" get low weight; distinctive words get high weight.

THE MATH `TF-IDF(t, d) = TF(t, d) × log(N / DF(t))`, where DF(t) is the number of documents containing term t and N is the total.

TF-IDF is still a strong, fast baseline for text classification and is the "keyword" half of **hybrid search** in the RAG guide (BM25 is a refined TF-IDF variant).

13.3 Word2Vec, GloVe, FastText

Static word embeddings: one fixed vector per word, learned so that similar words are nearby. The idea that "you shall know a word by the company it keeps" (the distributional hypothesis).

- **Word2Vec**: two training schemes: **CBOW** (predict a word from its context, faster) and **Skip-gram** (predict the context from a word, better for rare words). Famous for vector arithmetic: `king - man + woman ≈ queen`.
- **GloVe**: factorises a global word co-occurrence matrix. Combines global corpus statistics with local context.
- **FastText**: represents words as bags of character n-grams, so it can embed **out-of-vocabulary** words and handles morphologically rich languages and typos.

> **KEY IDEA — static vs contextual embeddings, a standard question** Word2Vec/GloVe/FastText give each word **one** vector regardless of context, so "bank" has a single embedding blending the river and the money senses. **Contextual** embeddings from BERT or similar give a **different** vector for each occurrence depending on the surrounding words, so "river bank" and "savings bank" get different representations. This is the fundamental advance that made modern NLP work, and it is why "the embedding of a word" is a slightly ill-posed phrase for a transformer.

13.4 Contextual embeddings

From transformer encoders (BERT and descendants): each token's representation depends on the whole sentence. For sentence-level tasks and semantic search you want one vector per sentence; **Sentence-BERT (SBERT)** fine-tunes BERT with a siamese structure to produce embeddings where cosine similarity is meaningful, which is exactly the embedding model layer in the RAG guide.

13.5 Classic NLP tasks

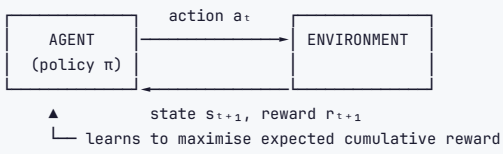
Task	What it does
Text classification	sentiment, topic, spam
Named Entity Recognition (NER)	tag people, places, organisations
Part-of-Speech tagging	label each word's grammatical role
Machine translation	one language to another
Summarisation	extractive (select sentences) or abstractive (generate new text)
Question answering	extractive (find the span) or generative
Coreference resolution	link "it"/"she" to what they refer to
Topic modelling	discover themes (LDA, the Latent Dirichlet Allocation kind, not the classifier)

Part XIV — Reinforcement Learning

RL matters for interviews both in its own right and because **RLHF** aligns the LLMs from Part XI.

14.1 The MDP framing

An **agent** interacts with an **environment** over time. At each step it observes a **state**, takes an **action**, and receives a **reward** and a new state. The goal is a **policy** (a mapping from states to actions) that maximises **cumulative** reward, not immediate reward.



Formalised as a **Markov Decision Process (MDP)**: states S, actions A, transition probabilities P, rewards R, and a **discount factor γ** (0 to 1) that trades immediate against future reward. The **Markov property** means the next state depends only on the current state and action, not the full history.

Key functions: - **Return** $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$: discounted future reward. - **State-value** $V(s)$: expected return from state s under the policy. - **Action-value** $Q(s,a)$: expected return from taking action a in state s, then following the policy. - **Bellman equation**: expresses a value in terms of the values of successor states, the recursion at the heart of RL.

14.2 The exploration-exploitation trade-off

> **KEY IDEA** An agent must **exploit** what it knows to earn reward, but also **explore** unknown actions that might be better. Pure exploitation gets stuck in a local optimum; pure exploration never cashes in. The classic scheme is **ε-greedy**: with probability ε take a random action, otherwise take the best known one, often decaying ε over time. This trade-off, illustrated by the **multi-armed bandit** problem, is the RL question most likely to come up in a general ML interview.

14.3 Q-learning and Deep Q-Networks

- **Q-learning**: learn the Q-table by iteratively updating $Q(s,a)$ toward the observed reward plus the discounted best next-state value. It is **off-policy** (learns the optimal policy while behaving with exploration) and **model-free** (needs no model of the environment). Works when states and actions are few enough to tabulate.
- **Deep Q-Network (DQN)**: when the state space is huge (raw pixels), replace the table with a neural network that approximates Q. DeepMind's Atari result. Two stabilising tricks worth naming: **experience replay** (store past transitions and train on random batches, breaking correlation between consecutive samples) and a **target network** (a periodically-updated copy used for the update target, preventing a moving-target instability).

14.4 Policy gradients and PPO

Instead of learning values and deriving a policy, **policy-gradient** methods optimise the policy directly, which handles continuous action spaces and stochastic policies naturally.

- **REINFORCE**: the basic policy gradient; high variance.
- **Actor-Critic**: an actor (policy) and a critic (value function) that reduces the variance of the gradient estimate. A2C/A3C are variants.
- **PPO (Proximal Policy Optimisation)**: constrains each policy update to stay close to the previous policy (via a clipped objective), preventing destructive large steps. Stable and robust, which is why it became the default, including for **RLHF** in LLM training.

14.5 RL in LLMs

This is the concrete payoff and a likely question given the LLM focus. In **RLHF** (Part XI), the language model is the **policy**, generating a response is a sequence of **actions** (tokens), a learned **reward model** scores the full response, and **PPO** updates the policy to earn higher reward, with a **KL-divergence penalty** keeping it near the original model so it stays fluent. **DPO** later removed the explicit RL loop, but the framing, aligning a generative model to human preferences via a reward signal, is pure RL and worth being able to describe in those terms.

Part XV — Time Series

15.1 Components and stationarity

A time series decomposes into **trend** (long-term direction), **seasonality** (fixed-period cycles: daily, weekly, yearly), **cyclic** behaviour (non-fixed-period swings), and **residual** noise. Decomposition can be **additive** (`y = trend + seasonal + residual`) or **multiplicative** (`y = trend × seasonal × residual` , when the seasonal swing grows with the level).

> **KEY IDEA — stationarity, the central time-series concept** A series is **stationary** if its statistical properties (mean, variance, autocorrelation) do not change over time. Most classical methods (ARIMA) **require** stationarity, because you cannot reliably learn from a moving target. You test for it with the **Augmented Dickey-Fuller (ADF)** test (null hypothesis: non-stationary), and you achieve it by **differencing** (subtracting the previous value to remove trend), by transforming (log to stabilise variance), or by removing seasonality. This is the first thing to discuss in any time-series interview.

15.2 The ARIMA family

ARIMA(p, d, q) combines three parts: - **AR(p)** AutoRegressive: the value depends on its own p previous values. - **I(d)** Integrated: the series is differenced d times to make it stationary. - **MA(q)** Moving Average: the value depends on the previous q forecast errors.

You choose p and q from the **ACF** (autocorrelation) and **PACF** (partial autocorrelation) plots, or let `auto_arima` search. **SARIMA** adds seasonal terms; **SARIMAX** adds external regressors.

15.3 ML and DL for time series

- **Classical**: ARIMA/SARIMA, Exponential Smoothing (Holt-Winters), and simple baselines (naive, seasonal naive) which are surprisingly hard to beat and must always be included.
- **ML**: turn forecasting into supervised learning by engineering **lag features** (value at t-1, t-7), **rolling statistics** (7-day mean/std), and date parts, then apply gradient boosting. This wins many real forecasting competitions.
- **DL**: LSTMs/GRUs, **Temporal Convolutional Networks**, and transformer variants (Informer, Temporal Fusion Transformer) for long or multivariate series. **Prophet** (from Meta) is a robust, easy decomposable model for business series with strong seasonality and holidays.

15.4 Validation without leakage

Restating the critical point from Part III because it is the number one time-series mistake: **never use random k-fold**. Use forward-chaining (`TimeSeriesSplit`): always train on the past and validate on the future. Never let a feature use information from after the prediction point (a rolling mean must not include the target period). Add a **gap** between train and validation if features use windows that could straddle the boundary.

Part XVI — Recommender Systems

16.1 Collaborative filtering

Recommend based on user-item interactions, ignoring content. Two flavours: - **User-based**: find users similar to you, recommend what they liked. - **Item-based**: find items similar to ones you liked (similarity computed from co-interaction patterns). More stable and scalable, since items change less than users.

The cold-start problem is the defining challenge: a brand-new user or item has no interactions, so pure collaborative filtering cannot place them. Solutions: fall back to content-based features, use popularity defaults, or ask onboarding questions.

16.2 Matrix factorisation

Represent the sparse user-item rating matrix as the product of two low-rank matrices: user factors and item factors, each a small **latent** vector. A predicted rating is the dot product of a user vector and an item vector. Learned latent factors capture themes (a "sci-fi" dimension, a "lightweight comedy" dimension) without anyone labelling them. **SVD**, **SVD++**, and **Alternating Least Squares (ALS)** are the standard methods; this is the direct application of the SVD from Part I.

16.3 Content-based and hybrid

- **Content-based**: recommend items whose **features** resemble ones the user liked (genre, tags, text embeddings). Handles new items well; tends to over-specialise (a filter bubble).
- **Hybrid**: combine collaborative and content signals, which is what every real system does, precisely because it covers each method's cold-start and diversity weaknesses.

16.4 Deep recommenders

Neural Collaborative Filtering replaces the dot product with a learned neural interaction. **Two-tower** models embed users and items in separate networks and match by similarity, which scales to retrieval over millions of items (and is architecturally identical to the retriever in the RAG guide). **Wide & Deep** and **DeepFM** combine memorisation of feature crosses with generalisation. **Sequential/session-based** recommenders (often transformer-based, like SASRec) model the order of interactions.

16.5 Evaluation

Offline ranking metrics from Part IV: **Precision@k, Recall@k, NDCG, MAP, MRR**, plus **coverage** and **diversity** and **novelty** (a recommender that only shows blockbusters is accurate but useless). Crucially, offline metrics correlate imperfectly with business impact, so the real evaluation is an **online A/B test** on engagement or revenue. Saying that offline metrics guide iteration but online A/B tests decide is exactly the production maturity interviewers want.

Part XVII — Explainability, Fairness, and Responsible AI

Increasingly asked, especially for senior and regulated-industry roles.

17.1 Feature importance

- **Model-specific**: linear-model coefficients (on standardised features), tree impurity importances (biased, as noted in Part V).
- **Permutation importance**: shuffle one feature and measure the drop in performance. Model-agnostic, computed on held-out data, far more trustworthy than impurity importance.

17.2 SHAP and LIME

> **KEY IDEA — global vs local explanations** **Global** explanations describe the whole model ("income is the most important feature overall"). **Local** explanations describe a single prediction ("*this* loan was denied because of debt-to-income and recent missed payments"). Regulated decisions usually need local explanations for the individual affected. Know which is which.

- **SHAP (SHapley Additive exPlanations)**: grounded in cooperative game theory. It fairly distributes a prediction's deviation from the average among the features, with strong consistency guarantees. Provides both local (per-prediction) and, aggregated, global explanations. The current standard, though it can be slow (TreeSHAP is fast for tree models).
- **LIME (Local Interpretable Model-agnostic Explanations)**: fits a simple, interpretable model (linear) locally around one prediction by perturbing the input and seeing how the output changes. Fast and intuitive, but less stable than SHAP.

17.3 Fairness

- **Sources of bias**: biased training data (historical discrimination baked in), biased labels, biased sampling, and feedback loops (a biased model shapes future data).
- **Fairness metrics**, and the key point that they **conflict**: **demographic parity** (equal positive rates across groups), **equal opportunity** (equal true-positive rates), **equalised odds** (equal TPR and FPR). It is mathematically impossible to satisfy all of them simultaneously except in trivial cases, so fairness is a value choice about which error matters, not a metric you simply maximise.
- **Mitigation**: at the data level (reweighting, resampling), in-processing (fairness constraints in the objective), or post-processing (group-specific thresholds).

17.4 Privacy

Differential privacy adds calibrated noise so that no individual's presence in the training data can be inferred from the model. **Federated learning** trains across devices without centralising raw data (the model comes to the data). **Membership inference attacks** try to determine whether a specific record was in the training set, which is the threat these defend against.

17.5 Governance

Model cards (document a model's intended use, performance across subgroups, and limitations) and **datasheets for datasets** (document how data was collected and its biases) are the standard documentation artefacts. Being able to say "I would ship a model card documenting subgroup performance and known limitations" is a strong responsible-AI signal.

Part XVIII — ML System Design

System design rounds separate senior candidates from junior ones. There is no single right answer; there is a right **process**.

18.1 The framework

Walk these steps out loud, in order, every time. The structure itself scores points.

1. **Clarify requirements.** What is the actual goal? Who is the user? What is the scale (queries per second, number of items, latency budget)? Is this a ranking, classification, or generation problem? What does success mean to the business?
2. **Frame it as an ML problem.** Define the input, the output, and the target. State whether ML is even needed, and what the non-ML baseline is.
3. **Define metrics.** Both **offline** (the metric you optimise: AUC, NDCG) and **online** (the business metric you actually care about: click-through, revenue, retention). Name the gap between them.
4. **Data.** What data exists? How are labels obtained (explicit, implicit, human annotation)? What is the volume? What are the privacy and freshness constraints?
5. **Features.** What signals matter? User, item, context, interaction, and historical aggregate features.
6. **Model.** Start simple (a baseline), justify the progression to something more complex. Discuss the trade-offs, not just the choice.
7. **Training and evaluation.** Split strategy (time-based for anything temporal), validation, offline evaluation, how you catch overfitting and leakage.
8. **Serving.** Batch vs real-time, latency, the two-stage retrieve-then-rank pattern for large candidate sets, caching, the model artefact.
9. **Monitoring and iteration.** Drift detection, feedback loops, retraining cadence, A/B testing, guardrails.

> **KEY IDEA** The most common failure in a system design round is jumping straight to "I'd use a neural network" at step 6 without doing steps 1 through 5. Interviewers are testing whether you can **scope a problem**, not whether you know model names. Spend real time on requirements, metrics, and data. The candidate who says "before I pick a model, let me understand the label, the scale, and the cost of each error type" has already outscored the one who names an architecture in the first sentence.

18.2 Ten worked designs (the sketch of a good answer)

Design 1 — Recommend videos on a home feed. Frame as ranking. Two-stage: a **retrieval** model (two-tower, embed user and videos, approximate nearest neighbour over millions of items to get ~hundreds of candidates) then a **ranking** model (a richer model, often gradient boosting or a deep net, scoring the few hundred on engagement probability). Offline metric NDCG; online metric watch time with guardrails on diversity and long-term retention. Features: watch history, video embeddings, freshness, context. Cold-start via popularity and content features. Retrain frequently; monitor for feedback loops. Note that this is architecturally the retrieve-then-rerank pattern from the RAG guide, applied to recommendations.

Design 2 — Detect fraudulent transactions. Frame as extreme-imbalance binary classification (Part IV and the imbalance section). Metric: PR-AUC and precision/recall at a threshold set by the review team's capacity, not accuracy. Real-time serving with a strict latency budget. Features: transaction, user history aggregates, velocity features, device. Start with gradient boosting plus class weighting; consider anomaly detection for the cold-start of never-seen fraud patterns. Human-in-the-loop review generates labels. Monitor drift closely, because fraud is adversarial and patterns shift deliberately.

Design 3 — A customer support chatbot over company docs. This is a **RAG** system (the entire first guide). Ingestion pipeline chunks and embeds the docs into a vector DB; at query time, embed the question, retrieve top-k, augment the prompt, generate with citations. Metric: retrieval hit rate and MRR offline, plus faithfulness and answer relevance (RAGAS); online, resolution rate and escalation rate. Add hybrid search for exact terms, reranking for precision, and query rewriting for follow-ups. Monitor retrieval quality, because RAG degrades when retrieval starts missing, not when the model breaks.

Design 4 — Predict customer churn. Frame as binary classification, but the subtlety is **temporal**: define the prediction window and the outcome window carefully, and use a time-based split to avoid leakage. The biggest risk is target leakage from features populated only after churn (Part III's scenario). Metric: AUC or PR-AUC, plus lift for the retention team ("target the top decile"). Gradient boosting, with SHAP for the per-customer explanation the retention team needs. Threshold set by retention-offer economics.

Design 5 — Image search ("find similar products"). Frame as embedding + nearest neighbour. A pretrained CNN or ViT (fine-tuned with a contrastive/triplet loss) embeds every product image; at query time embed the query image and do ANN search over the catalogue. Metric: Precision@k, Recall@k. This is the same embed-and-retrieve pattern as text RAG, on images, which is worth pointing out to show you see the unifying structure.

Design 6 — Search ranking (a search engine or in-app search). Frame as learning-to-rank. Two stages: retrieval (an inverted index plus vector retrieval to get candidates) then ranking (a learned model scoring relevance). Features: query-document match (BM25, semantic similarity), document quality/popularity, personalisation, freshness. Labels come from clicks (implicit, but biased toward higher positions, so correct for position bias) or human relevance judgements. Metric: **NDCG** offline (it handles graded relevance and position discounting), online engagement/success rate. Approaches: pointwise, pairwise (RankNet), or listwise (LambdaMART is the classic strong baseline; a neural ranker if data is large). Discuss the exploration problem (you only get clicks on what you show) and query understanding (spelling, intent, entities).

Design 7 — Ad click-through-rate (CTR) prediction. Frame as binary classification (will this ad be clicked) at massive scale and low latency. The signature challenges: enormous, sparse, high-cardinality categorical features (user IDs, ad IDs), extreme class imbalance (clicks are rare), and calibration (predicted probability feeds the bid, so it must be accurate, not just ranked). Metric: log loss and calibration, plus AUC. Models: logistic regression with feature hashing and crosses at first (scales, interpretable), then factorisation machines or a deep model (Wide & Deep, DeepFM) that learn feature interactions and embed high-cardinality features. Retrain very frequently because ad inventory and user behaviour shift fast, and serve under a strict latency budget.

Design 8 — A content moderation / safety classification system. Frame as multi-label classification (a piece of content can violate several policies) with an asymmetric, evolving, adversarial threat. Metric: per-class recall at a precision floor (a missed violation and a wrongly-removed post both cost, differently), plus fairness so it does not over-flag particular groups or dialects. Labels: human reviewers with strong guidelines and measured agreement (kappa); label quality is a first-order problem. Architecture: a cheap model screens all content and a heavier model plus human review handle the uncertain middle (two-stage). Production: adversaries adapt, so monitor drift, keep humans in the loop generating fresh labels, and provide an appeals path because false positives harm real users.

Design 9 — A demand / sales forecasting platform (many series). Frame as time-series forecasting across thousands or millions of series (per product, per store). Metric: an asymmetric business cost (stockout vs overstock), so quantile loss over symmetric error, evaluated with time-based validation. Approach: strong baselines (seasonal-naive) that are hard to beat, then a single global gradient-boosting model with lag and calendar features across all series (scales far better than one model per series and shares signal), and deep models (Temporal Fusion Transformer) only if warranted. Handle intermittent demand, promotions and holidays as regressors, and cold-start for new products via similar-item priors. The key design decision is global-model-with-features versus per-series models.

Design 10 — A real-time personalization / next-best-action system. Frame as contextual bandits or ranking under tight latency. The system chooses, in milliseconds, which offer/content/action to show given the user context, and must balance **exploration and exploitation** (it only learns from what it shows). Approach: a contextual bandit (Thompson sampling or LinUCB) or a ranking model with an exploration policy, features precomputed in a feature store to meet latency. Metric: online reward (conversion, engagement) via interleaving or A/B tests, with guardrails. Discuss the feedback loop, cold-start, and delayed rewards (a conversion may happen hours later).

18.3 Production concerns (the cross-cutting checklist)

- **Latency budget**: what is the p99 requirement, and does the model fit in it? Precompute embeddings offline; keep the online path light.
- **Two-stage retrieval**: never score millions of candidates with an expensive model. Cheaply retrieve a few hundred, then rank those. This single pattern appears in search, recommendations, and RAG.
- **Caching**: cache embeddings, frequent queries, and features.
- **Training-serving skew**: the feature computed at training time must match the one computed at serving time. A feature store exists to guarantee this.
- **Drift**: monitor input distributions (KS test) and prediction distributions; alert and retrain when they move.
- **A/B testing**: the only real proof a model helps. Offline metrics guide iteration; online experiments decide.
- **Rollback and shadow deployment**: ship new models in shadow mode (serving in parallel, not affecting users) before switching traffic, and keep the ability to roll back instantly.
- **Cost**: GPU inference, API tokens, storage. The reranker trick (retrieve many cheaply, send few to the expensive model) is a cost lever as much as a quality one.

Part XIX — The Interview

19.1 The question bank (with answers)

These are grouped by topic. Practise saying the answers out loud. Where an answer is fully covered earlier, the reference is given instead of repeating it.

Foundations and statistics

Q1. Explain the bias-variance trade-off. See Part VII. Key sentence: bias is error from wrong assumptions (underfitting), variance is error from sensitivity to the training sample (overfitting), and total error is bias² + variance + irreducible noise. Diagnose via the train-validation gap.

Q2. What is the difference between correlation and causation? Correlation measures statistical association; causation means one variable produces a change in the other. Confounders create correlation without causation (ice cream sales correlate with drownings; the confounder is summer heat). Establishing causation needs randomised experiments or careful causal inference, not just observational correlation.

Q3. What is a p-value? The probability of observing data at least as extreme as what you saw, assuming the null hypothesis is true. It is **not** the probability the null is true, nor the probability your result happened by chance. See Part I.

Q4. Explain the Central Limit Theorem and why it matters. The mean of a large enough i.i.d. sample is approximately normal regardless of the underlying distribution. It underpins confidence intervals, standard errors, and most hypothesis tests.

Q5. Type I vs Type II error. Type I is a false positive (rejecting a true null); Type II is a false negative (failing to reject a false null). They map directly onto FP and FN in the confusion matrix. Reducing one typically increases the other at a fixed sample size.

Q6. What is MLE? Choosing parameters that maximise the probability of the observed data. Most losses are negative log-likelihoods: MSE is MLE under Gaussian noise, cross-entropy under a categorical distribution. See Part I.

Q7. Explain the disease-testing / base rate problem. See Part I. The point: with a low base rate, even an accurate test yields mostly false positives, which is why precision collapses on rare classes.

Data and features

Q8. How do you handle missing data? Diagnose the mechanism (MCAR/MAR/MNAR) first, because it determines what is valid. Then choose: drop (only if tiny and MCAR), impute (median/mode, or model-based like KNN/MICE), add a missingness indicator, or use a model that handles missingness natively (LightGBM/XGBoost). Fit imputers on train only. See Part III.

Q9. How do you detect and handle outliers? IQR or z-score or Isolation Forest to detect; investigate before removing (an outlier may be the signal, as in fraud); then cap, transform (log), or use robust models/losses. See Part III.

Q10. What is data leakage? Give examples. Information available in training that would not exist at prediction time, causing optimistic offline results and production failure. Examples: scaling before splitting, target-derived features, random splits on temporal data, target encoding on the full set, the same entity in train and test. Fix with pipelines and careful temporal/group splits. See Part III.

Q11. When must you scale features, and when is it unnecessary? Scale for distance-based (k-NN, K-Means, SVM), gradient-based (neural nets), and variance-based (PCA) methods, and for regularised linear models. Unnecessary for tree-based methods, which split one feature at a time and are invariant to monotonic rescaling. See Part III.

Q12. One-hot vs label vs target encoding? One-hot for nominal inputs (no false ordering, but high dimensionality); ordinal only for genuinely ordered categories; target encoding for high cardinality but only computed inside CV folds with smoothing to avoid leakage; label encoding for the target variable only. See Part III.

Q13. How do you deal with imbalanced classes? Fix the metric first (PR-AUC, precision/recall, not accuracy), then class weights, then resampling **inside folds** (SMOTE), then threshold tuning, and consider anomaly detection at extreme ratios. See Part III.

Classical models

Q14. How does logistic regression work, and why not MSE? Linear score through a sigmoid gives P(y=1); trained with cross-entropy. Not MSE because MSE is non-convex with the sigmoid and its gradient vanishes when confidently wrong; cross-entropy is convex with a clean gradient. See Part V.

Q15. Explain regularisation, and L1 vs L2. A penalty on coefficient size that trades bias for variance. L1 (diamond constraint) drives coefficients to exactly zero, giving sparsity and feature selection; L2 (circular constraint) shrinks smoothly without zeroing. Elastic Net combines them. See Parts I and V.

Q16. How does a decision tree choose splits? It greedily picks the feature and threshold that most reduce impurity (Gini or entropy for classification, variance for regression), measured by information gain. See Part V.

Q17. Bagging vs boosting. Bagging trains models in parallel on bootstrap samples and averages, reducing variance (base learners are deep/low-bias). Boosting trains sequentially, each correcting the previous errors, reducing bias (base learners are shallow/weak). See Part V.

Q18. How does random forest differ from bagging? Random forest adds random feature subsampling at each split, which decorrelates the trees so averaging reduces variance more effectively. See Part V.

Q19. Explain gradient boosting. Each new tree fits the negative gradient of the loss with respect to current predictions (residuals for squared error), added with a learning-rate shrinkage. See Part V.

Q20. XGBoost vs LightGBM vs CatBoost. XGBoost: regularised objective, second-order splits, level-wise growth. LightGBM: leaf-wise growth plus histogram binning, fastest, overfits on small data. CatBoost: ordered boosting and native categorical handling, best on categorical-heavy data. See Part V.

Q21. Explain the kernel trick. The SVM dual depends only on inner products, so a kernel computes the inner product in a high-dimensional space without constructing it, giving non-linear boundaries cheaply. RBF corresponds to an infinite-dimensional space. See Part V.

Q22. What are support vectors? The points on the margin boundary that alone determine the SVM's solution; all other points could be removed without changing the decision boundary. See Part V.

Q23. Why is Naive Bayes "naive," and why does it work anyway? It assumes features are conditionally independent given the class, which is usually false. It works because classification only needs the correct class to score highest, not accurate probabilities. See Part V.

Q24. Why does k-NN struggle in high dimensions? The curse of dimensionality: distances converge, so "nearest" loses meaning; the space becomes sparse. See Part VII.

Unsupervised

Q25. How do you choose k in K-Means? Elbow on inertia, silhouette score (more reliable), gap statistic, or domain knowledge. See Part VI.

Q26. K-Means vs DBSCAN. K-Means needs k, forces all points into spherical clusters; DBSCAN infers cluster count, finds arbitrary shapes, and labels outliers as noise, but struggles with varying density (use HDBSCAN). See Part VI.

Q27. Explain PCA. Project onto orthogonal directions of maximum variance (eigenvectors of the covariance matrix / via SVD), keeping the top components. Unsupervised, linear, requires scaling. See Part VI.

Q28. PCA vs LDA? PCA is unsupervised (max variance); LDA is supervised (max class separation), capped at classes-minus-one components. See Part VI.

Q29. Why shouldn't you interpret t-SNE cluster sizes or distances? t-SNE preserves only local neighbourhoods; global distances and blob sizes are artefacts. Use it for visualisation only; prefer UMAP. See Part VI.

Deep learning

Q30. Why do we need activation functions? Without non-linearity, stacked layers collapse to a single linear layer; the activation is where all expressive power comes from. See Part VIII.

Q31. Explain the vanishing gradient problem and its solutions. Repeated multiplication of small derivatives (saturated sigmoid/tanh) shrinks gradients toward zero in early layers. Solutions: ReLU, good initialisation (He), batch/layer norm, residual connections, and for sequences the LSTM/GRU gating. See Parts VIII and X.

Q32. Explain backpropagation. The chain rule applied from output to input, computing each weight's contribution to the loss so the optimiser can update it. See Part VIII.

Q33. Compare optimisers: SGD, Momentum, Adam. SGD is plain gradient descent; momentum accumulates a velocity to roll through ravines; Adam adds per-parameter adaptive rates via first and second moment estimates. AdamW decouples weight decay and is the transformer standard. See Part VIII.

Q34. What does batch normalisation do, and how does it behave differently at train and test time? Normalises layer inputs to stabilise and speed training (and smooths the loss landscape). At training it uses batch statistics; at inference it uses stored running averages, which is why `model.eval()` is required. See Part VIII.

Q35. Dropout: what and why, and why can validation loss be lower than training loss? Randomly zeroes activations to prevent co-adaptation and overfitting; it is on during training and off during evaluation, so evaluation is un-handicapped, which can make validation loss appear lower. See Part VIII.

Q36. He vs Xavier initialisation? He (variance 2/n_in) for ReLU networks; Xavier (1/n_in) for tanh/sigmoid. Correct scaling preserves signal variance across layers. See Part VIII.

CNNs

Q37. Why convolution over dense layers for images? Local connectivity, parameter sharing (translation equivariance and far fewer parameters), and hierarchical feature learning. See Part IX.

Q38. What problem do residual connections solve? Degradation in very deep networks: the identity shortcut gives gradients a direct path backward and lets blocks learn zero if useless, enabling 100+ layer training. See Part IX.

Q39. Explain pooling and its purpose. Downsamples feature maps, reducing computation and adding slight translation invariance; max pooling keeps the strongest activation. See Part IX.

Q40. CNN vs Vision Transformer? CNNs have built-in locality/translation biases, making them data-efficient; ViTs lack these, needing more data but scaling higher and modelling global relations from the first layer. See Part IX.

Q41. How does transfer learning work for vision? Take an ImageNet-pretrained backbone, freeze the feature extractor, replace and train a new head, then optionally fine-tune top layers at a tiny learning rate. See Part IX.

RNNs and sequences

Q42. Why do vanilla RNNs fail on long sequences? BPTT multiplies the same matrix repeatedly, so gradients vanish or explode over many steps, preventing learning of long-range dependencies. See Part X.

Q43. How does an LSTM solve this? A separate cell state with additive updates gated by forget/input/output gates gives gradients a near-uninterrupted path backward, so long-range information survives. See Part X.

Q44. LSTM vs GRU? GRU merges cell and hidden state and uses two gates, so it is lighter and faster with often comparable performance; LSTM has more capacity via its extra gate and separate cell state. See Part X.

Q45. What is a bidirectional RNN and when can't you use one? Runs forward and backward and concatenates, giving both-direction context; impossible for real-time generation where the future is unknown. See Part X.

Transformers and LLMs

Q46. Explain self-attention. Each token forms a query, key, and value; attention scores queries against keys (scaled by √d_k), softmaxes into weights, and returns a weighted sum of values, so every token gathers relevant context from all others. See Part XI.

Q47. Why divide by √d_k? To normalise the variance of the dot products so softmax stays in a well-conditioned range and gradients do not vanish. See Part XI.

Q48. What is multi-head attention for? Attending to different representation subspaces simultaneously, like multiple filters in a CNN. See Part XI.

Q49. Why do transformers need positional encoding? Self-attention is permutation-invariant, so without positional information it cannot distinguish word order. See Part XI.

Q50. Why did transformers replace RNNs? Parallelism over the sequence, direct constant-length paths between distant tokens, and scalability to huge models. See Part X.

Q51. BERT vs GPT? BERT is an encoder trained with masked language modelling (bidirectional, for understanding/embeddings); GPT is a decoder trained with next-token prediction (causal, for generation). See Part XI.

Q52. Explain the causal mask. It sets attention to future positions to negative infinity so each token sees only itself and earlier tokens, making autoregressive training and generation valid. See Part XI.

Q53. RAG vs fine-tuning? RAG supplies facts at inference via retrieval (fresh, citable, cheap to update); fine-tuning changes weights to alter behaviour/format/style. Most "fine-tuning" needs are actually RAG. See Part XI and the first guide.

Q54. What is LoRA? Freeze base weights and learn small low-rank update matrices, training under 1% of parameters because the task-adaptation update has low intrinsic rank. QLoRA adds 4-bit quantisation. See Part XI.

Q55. What is RLHF? Train a reward model on human preference comparisons, then optimise the LLM against it with PPO under a KL penalty to the reference model. DPO achieves similar alignment without the RL loop. See Part XI.

Generative

Q56. GAN vs VAE vs diffusion? GAN: sharpest, unstable (mode collapse). VAE: stable, smooth latent, blurry. Diffusion: best quality and diversity, stable, but slow sampling. See Part XII.

Q57. Explain how diffusion models work. Add noise forward over many steps; train a network to reverse it step by step (predict the noise); generate by denoising from pure noise, guided by a text embedding. Latent diffusion runs it in a compressed space. See Part XII.

Q58. What is mode collapse? A GAN failure where the generator produces only a few outputs that reliably fool the discriminator, losing diversity; Wasserstein GAN and other fixes address it. See Part XII.

Q59. What is the reparameterisation trick? Writing $z = \mu + \sigma \cdot \varepsilon$ with ε from a fixed normal, so randomness is outside the gradient path and backprop can flow through μ and σ in a VAE. See Part XII.

Evaluation (expect several)

Q60. Draw and explain the confusion matrix. See Part IV. TP/TN/FP/FN, with FP as Type I and FN as Type II.

Q61. Precision vs recall, and when to prioritise each. Precision = TP/(TP+FP), "when it says yes is it right," prioritise when false positives are costly. Recall = TP/(TP+FN), "did it catch them all," prioritise when false negatives are costly. See Part IV.

Q62. What is F1 and why the harmonic mean? The harmonic mean of precision and recall; it stays low unless both are high, unlike the arithmetic mean. See Part IV.

Q63. Give three names for recall. Recall, sensitivity, true positive rate. See Part IV.

Q64. What is specificity? TN/(TN+FP), the true negative rate; 1 minus specificity is the false positive rate. See Part IV.

Q65. Why is precision affected by class balance but recall is not? Recall is computed within actual positives; precision mixes both classes down the predicted-positive column, so prevalence changes it. See Part IV.

Q66. Explain ROC-AUC and its probabilistic interpretation. TPR vs FPR across thresholds; AUC is the probability a random positive is ranked above a random negative. See Part IV.

Q67. ROC-AUC vs PR-AUC on imbalanced data. Prefer PR-AUC: FPR's huge denominator makes ROC look good despite terrible precision; PR-AUC stays sensitive to false positives. See Part IV.

Q68. Accuracy paradox. With heavy imbalance, predicting the majority class gives high accuracy and zero utility. See Part IV.

Q69. Macro vs weighted vs micro F1. Macro treats classes equally (surfaces minority failure); weighted lets the majority dominate; micro equals accuracy in single-label multi-class. See Part IV.

Q70. RMSE vs MAE. RMSE squares errors (penalises large ones, outlier-sensitive, minimised by the mean); MAE is robust (minimised by the median). See Part IV.

Q71. What does R^2 mean, and can it be negative? Fraction of variance explained; yes, negative means worse than predicting the mean. See Part IV.

Q72. What is model calibration and how do you fix it? Predicted probabilities match observed frequencies; fix with Platt scaling or isotonic regression (or temperature scaling for NNs). See Part IV.

System design and production

Q73. How would you design a recommendation system? See Part XVIII, Design 1: two-stage retrieval then ranking, NDCG offline, engagement online, cold-start handling.

Q74. How do you detect model drift in production? Monitor input and prediction distributions (KS test), track the online metric, alert on divergence, retrain. See Parts III and XVIII.

Q75. How do you decide when to retrain? On a schedule, on drift detection, or on a performance-drop trigger; balanced against retraining cost. See Part XVIII.

Q76. Offline metrics look great but the business metric didn't move. What happened? Offline-online gap: the offline metric is a proxy, there may be a train-serve skew, a feedback loop, or the metric doesn't capture the true objective. Only an A/B test proves impact. See Part XVIII.

Q77. How would you reduce inference cost of an LLM feature? Quantisation, KV caching, a smaller or distilled model, retrieval to shrink prompts, caching frequent queries, batching, and the retrieve-many-rerank-few pattern. See Part XI and the first guide.

19.2 Coding problems: implement from scratch

Interviewers ask you to code core algorithms in NumPy to prove you understand them, not to test library recall.

```
import numpy as np

# 1. K-Means from scratch
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    centroids = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        d = np.linalg.norm(X[:, None] - centroids[None], axis=2)  # (n, k)
        labels = d.argmin(1)
        new = np.array([X[labels == j].mean(0) if (labels == j).any()
                        else centroids[j] for j in range(k)])
        if np.allclose(new, centroids): break
        centroids = new
    return labels, centroids

# 2. Linear regression via the normal equation
def linreg_normal(X, y):
    X = np.c_[np.ones(len(X)), X]              # add bias column
    return np.linalg.solve(X.T @ X, X.T @ y)   # solve, don't invert

# 3. Logistic regression via gradient descent
def logreg(X, y, lr=0.1, epochs=1000):
    X = np.c_[np.ones(len(X)), X]
    w = np.zeros(X.shape[1])
    for _ in range(epochs):
        p = 1 / (1 + np.exp(-X @ w))
        w -= lr * X.T @ (p - y) / len(y)       # the clean cross-entropy gradient
    return w

# 4. k-Nearest Neighbours
def knn_predict(X_train, y_train, X_test, k=5):
    preds = []
    for x in X_test:
        d = np.linalg.norm(X_train - x, axis=1)
        idx = d.argsort()[:k]
        preds.append(np.bincount(y_train[idx]).argmax())
    return np.array(preds)

# 5. PCA
def pca(X, n_components):
    Xc = X - X.mean(0)
    cov = np.cov(Xc.T)
    vals, vecs = np.linalg.eigh(cov)           # eigh for symmetric matrices
    order = vals.argsort()[::-1]
    W = vecs[:, order[:n_components]]
    return Xc @ W

# 6. Cross-entropy and softmax (numerically stable)
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)       # subtract max: prevents overflow
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def cross_entropy(probs, y_true):
```

```
    n = len(y_true)
    return -np.log(probs[np.arange(n), y_true] + 1e-12).mean()

# 7. Compute all confusion-matrix metrics from scratch
def metrics_from_scratch(y_true, y_pred):
    tp = int(((y_pred == 1) & (y_true == 1)).sum())
    tn = int(((y_pred == 0) & (y_true == 0)).sum())
    fp = int(((y_pred == 1) & (y_true == 0)).sum())
    fn = int(((y_pred == 0) & (y_true == 1)).sum())
    precision = tp / (tp + fp) if tp + fp else 0
    recall    = tp / (tp + fn) if tp + fn else 0        # = sensitivity = TPR
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0
    return dict(tp=tp, tn=tn, fp=fp, fn=fn, precision=precision,
                recall=recall, specificity=tn/(tn+fp) if tn+fp else 0, f1=f1)

# 8. Train/test split from scratch
def train_test_split(X, y, test_size=0.2, seed=0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X))
    cut = int(len(X) * (1 - test_size))
    tr, te = idx[:cut], idx[cut:]
    return X[tr], X[te], y[tr], y[te]

# 9. A single self-attention head (the LLM-era coding question)
def attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    weights /= weights.sum(axis=-1, keepdims=True)
    return weights @ V

# 10. Gradient descent for any differentiable f
def gradient_descent(grad_fn, x0, lr=0.01, steps=1000):
    x = x0
    for _ in range(steps):
        x = x - lr * grad_fn(x)
    return x
```

Other classics to be ready for: implement a decision-tree split by maximising information gain; implement mini-batch SGD; implement BPE merges; compute IoU for two boxes; implement the sigmoid and its derivative; vectorise a slow loop; implement bootstrap sampling.

19.3 SQL and data manipulation

Data-heavy roles test SQL. Be fluent with: `GROUP BY` with `HAVING`, all `JOIN` types, **window functions** (`ROW_NUMBER`, `RANK`, `LAG` / `LEAD`, running totals with `SUM() OVER`), CTEs (`WITH`), self-joins, and finding duplicates or the top-N per group.

```
-- Top 3 products by revenue per category (window function pattern)
WITH ranked AS (
  SELECT category, product,
         SUM(revenue) AS rev,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY SUM(revenue) DESC) AS rn
  FROM sales
  GROUP BY category, product
)
SELECT category, product, rev FROM ranked WHERE rn <= 3;
```

Pandas equivalents to know cold: `groupby().agg()`, `merge()`, `pivot_table()`, `apply()` vs vectorised operations (prefer vectorised), `value_counts()`, handling NaN, `rolling()` windows, and `.loc` / `.iloc` indexing.

19.4 Case studies and take-homes

A typical take-home: given a dataset, build a model and present findings. What graders look for, in order of weight: correct **validation** (no leakage, right split), a **baseline** before complexity, appropriate **metric** for the problem, clean reproducible **code** (a pipeline, a fixed seed), honest discussion of **limitations**, and clear **communication** of the business implication. A modest model with rigorous methodology beats a fancy model with a leaky split every time. Do not skip the EDA, and do not present a single number without context.

19.5 Behavioural and project storytelling

Use **STAR** (Situation, Task, Action, Result) and quantify the result. For an ML project, have ready: the business problem and metric, why you chose the approach, what the **baseline** was, one hard technical problem you debugged (leakage, drift, imbalance, an unstable model), what the impact was in real terms, and what you would do differently. The single most valuable interview asset is one project you can discuss for fifteen minutes at increasing

depth, including its failures. As the first guide put it: interviewers want trade-offs and war stories, not definitions.

Part XXI — Gaps, Frontier Topics, and Modern LLM Evaluation

A short part filling in things a frontier-lab interview (OpenAI, Anthropic, Google DeepMind, Meta) will probe that the earlier parts touched only lightly.

21.1 Decoding strategies (how an LLM actually chooses each token)

The model outputs a probability distribution over the vocabulary; a **decoding strategy** turns that into a chosen token. This is asked constantly and people fumble it.

Strategy	How	Trade-off
Greedy	pick the single highest-probability token	deterministic, but repetitive and often bland; can miss a better overall sequence
Beam search	keep the top-b partial sequences at each step, expand all, prune to b	better for closed-ended tasks (translation, summarisation); tends to produce safe, generic text and is poor for open-ended generation
Temperature	divide logits by T before softmax; T>1 flattens (more random), T<1 sharpens	the creativity dial (see Part I of the first guide)
Top-k	sample only from the k most likely tokens	cuts the long tail of garbage; fixed k can be too rigid
Top-p (nucleus)	sample from the smallest set of tokens whose cumulative probability exceeds p	adapts the candidate set to the distribution's shape; the modern default, usually with p≈0.9
Min-p, typical, repetition penalty	refinements	reduce repetition and degenerate loops

> **KEY IDEA** Greedy and beam search are **deterministic** and good for tasks with a single correct answer; sampling with temperature and top-p is **stochastic** and good for open-ended generation. A common gotcha: beam search, despite sounding more sophisticated, produces *worse*, blander text for open-ended chat because it optimises for high-probability sequences, which are generic. Knowing *why* beam search hurts creative generation is the depth interviewers want.

21.2 Scaling laws and compute-optimal training

Scaling laws (Kaplan et al. 2020; Hoffmann et al. 2022, "Chinchilla") describe how loss falls predictably as a power law in three quantities: model **parameters**, **data** (tokens), and **compute**. The Chinchilla result corrected an earlier bias toward ever-larger models: for a fixed compute budget, models were *undertrained*, and you get lower loss by training a **smaller** model on **more** tokens, roughly **20 tokens per parameter** as a compute-optimal ratio.

> **KEY IDEA** "Chinchilla-optimal" means balancing model size and data for a fixed compute budget rather than just scaling parameters. It is why a well-trained 7B–70B model can beat a poorly-trained larger one, and why data quantity and quality became as important as parameter count. This is a favourite conceptual question at frontier labs because it shaped how everyone now trains models.

Emergent abilities (capabilities that appear only past a certain scale) and **test-time / inference-time scaling** (spending more compute at inference to "think longer," as in reasoning models) are the natural follow-ups. Test-time scaling means quality can improve by generating and evaluating more reasoning at inference rather than by making the model bigger.

21.3 Attention and efficiency variants

- **Multi-Query Attention (MQA)** and **Grouped-Query Attention (GQA)**: share key/value projections across heads (all heads, or groups of heads) to shrink the KV cache, the main memory cost at inference. GQA is the modern default (LLaMA 2/3) because it nearly matches full multi-head quality at a fraction of the memory.
- **Flash Attention**: an exact, IO-aware attention kernel that never materialises the full attention matrix in slow memory. Faster and far more memory-efficient; enables longer contexts.
- **Sliding-window / sparse attention** (Longformer, Mistral): each token attends only to a local window plus a few global tokens, making attention sub-quadratic for long documents.
- **Mixture of Experts (MoE)**: replace the feed-forward block with many "expert" sub-networks and a router that activates only a few per token, so total capacity is huge but compute per token stays modest (Mixtral, and reportedly the largest frontier models). The catch: routing balance and memory to hold all experts.
- **KV cache**: caches past keys/values so each new token is O(1) in history rather than recomputing attention over the whole sequence. Understanding the KV cache is essential for any inference-optimisation question.

21.4 Evaluating LLMs, RAG, and agents (the modern eval stack)

Classic metrics do not work for open-ended generation, and interviewers now test this directly.

Why BLEU and ROUGE are not enough. BLEU (precision of n-gram overlap, from translation) and ROUGE (recall of n-gram overlap, from summarisation) measure **word overlap against a reference**. They miss paraphrase ("press the reset button" vs "click the reset button" is penalised) and, critically, **they do not detect hallucination**: a fluent answer with completely wrong facts scores fine if it reuses the right words. Use them only when you have gold references and care about surface form; never as your only signal for a generative system.

The reference-based vs reference-free split. Reference-based metrics compare to a gold answer (works for factual QA, needs an expensive labelled set). Reference-free metrics judge the output on its own or against retrieved context (the only option in production, where no live labels exist).

LLM-as-a-judge has become the default for open-ended evaluation: another (usually stronger) model scores outputs against a natural-language rubric. Techniques like **G-Eval** (chain-of-thought scoring) improve reliability. Watch for its biases: position bias (favouring the first option), verbosity bias (favouring longer answers), and self-preference (favouring its own family's style). Mitigate by randomising order, using rubrics, and calibrating against human labels.

RAG evaluation (tying back to the first guide) splits by component, because the three failure modes look identical from outside but need different fixes: - **Retrieval**: hit rate, MRR, NDCG, and **context precision** (how much of what you retrieved was actually relevant; low context precision is the root cause of most RAG hallucination) and **context recall** (did retrieval get all the needed evidence). - **Generation**: **faithfulness** (is every claim supported by the retrieved context, i.e. did it hallucinate), **answer relevance** (does it address the question), and answer correctness. The **RAGAS** framework packages these, mostly reference-free via an LLM judge.

Agent evaluation adds trajectory-level metrics: **task completion / success rate**, **tool-call correctness** (did it call the right tool with the right arguments), step efficiency, and cost/latency per task. You evaluate the whole trace, not just the final answer.

> **KEY IDEA — the one-liner for "how do you evaluate a generative system"** Decompose it. For RAG, measure retrieval and generation separately, because an end-to-end score tells you nothing about which half failed. Use reference-free metrics (faithfulness, context precision, answer relevance) since production has no labels, and use an LLM-as-a-judge with a rubric for open-ended quality, calibrated against a small human-labelled set. Traditional word-overlap metrics like BLEU/ROUGE are a weak last resort because they miss paraphrase and hallucination entirely.

21.5 Graph Neural Networks (the family the earlier parts skipped)

For data that is naturally a **graph** (social networks, molecules, knowledge graphs, recommendation bipartite graphs), where relationships matter as much as node features.

The core idea is **message passing**: each node updates its representation by aggregating (sum/mean/max/attention) the representations of its neighbours, repeated over several layers so information propagates further across the graph. Variants: **GCN** (Graph Convolutional Network, normalised neighbour averaging), **GraphSAGE** (samples and aggregates neighbours, scales to huge graphs), and **GAT** (Graph Attention Network, learns per-neighbour attention weights, the same attention idea from Part XI applied to graph edges).

Uses: node classification (is this account fraudulent given its connections), link prediction (will these two users connect, which is recommendation), and graph classification (is this molecule toxic). A common gotcha is **over-smoothing**: too many message-passing layers make all node embeddings converge to the same value, so GNNs are usually shallow.

21.6 Distributed and efficient training (the systems half)

Frontier roles probe how models train at scale: - **Data parallelism**: replicate the model across GPUs, split the batch, average gradients (all-reduce). The default; limited by model fitting on one device. - **Model / tensor parallelism**: split a single layer's matrices across GPUs when the model is too big for one. Needs fast interconnect. - **Pipeline parallelism**: put different layers on different GPUs and pipeline micro-batches through them. - **ZeRO / FSDP** (Fully Sharded Data Parallel): shard optimiser states, gradients, and parameters across GPUs to fit enormous models without full replication. - **Mixed precision** (fp16/bf16 with fp32 master weights): roughly halves memory and speeds training; bf16 is preferred for its dynamic range. - **Gradient accumulation** (simulate a big batch on small memory), **gradient checkpointing** (recompute activations in the backward pass to save memory), and **quantisation** (int8/int4) for inference.

> **KEY IDEA** Distinguish the parallelisms by *what* they split: data parallelism splits the **batch**, tensor parallelism splits **within a layer**, pipeline parallelism splits **across layers**, and FSDP/ZeRO shards the **optimiser state and parameters**. Real large-scale training combines several (e.g. FSDP + tensor + pipeline, "3D parallelism"). Being able to say which to reach for, and why memory rather than compute is usually the binding constraint, is exactly the systems maturity a frontier lab checks.

21.7 Drift, monitoring, and the feedback loop (production ML)

- **Data drift (covariate shift)**: the input distribution P(x) changes (a new user demographic), even if the input-output relationship holds. Detect with distribution tests (KS test, population stability index) on features.
- **Concept drift**: the relationship P(y|x) itself changes (what counted as fraud last year no longer does). Detect via a drop in the live metric against labels; harder because you need ground truth.
- **Label drift / prior shift**: the class balance changes over time.
- **Training-serving skew**: features computed differently offline and online, the classic silent production bug, which a **feature store** exists to prevent.

Mitigations: monitor input and prediction distributions and the online metric; alert on divergence; retrain on a schedule, on a drift trigger, or on a performance-drop trigger; use shadow deployment and A/B tests before switching traffic; keep instant rollback. This is the loop that makes ML a lifecycle, not a one-off, and it connects directly to the MLOps companion.

Part XXII — Scenario and Applied Interview Questions

This is the part that gets you the offer. The earlier parts gave you concepts; this part gives you the **applied reasoning** that separates candidates who can *do* the job from those who can only define terms. Every question below is the kind actually asked at large companies and frontier labs, drawn from the patterns those interviews are known for: real-world simulations, "diagnose this failure," "design this under constraints," and safety/ethics reasoning.

How to use this part. For each question, read the scenario, then cover the answer and reason out loud yourself before checking. The written answers model the *structure* of a strong response, not a script to memorise. The single most important habit: **start by clarifying and framing, name the metric and the trade-off, and only then talk models.**

22.1 The universal framework for any applied question

Before the specific scenarios, internalise this skeleton. Almost every applied ML question can be answered by walking it:

1. **Clarify.** What exactly is the goal, who is the user, what is the scale, what is the latency budget, what does "good" mean to the business? Ask before assuming.
2. **Frame.** Input, output, target. Is it classification, regression, ranking, generation, or not ML at all? What is the non-ML baseline?
3. **Metric.** Offline (what you optimise) and online (what the business cares about), and the gap between them. Which error costs more, FP or FN?
4. **Data.** What exists, how labels are obtained, volume, freshness, privacy, bias.
5. **Approach.** Baseline first, then justify added complexity by its trade-off. Never lead with the fanciest model.
6. **Evaluate and iterate.** Split strategy (time-based if temporal), validation, leakage checks.
7. **Serve, monitor, and close the loop.** Latency, drift, A/B test, retraining, guardrails.

> **KEY IDEA** The number-one reason strong-on-paper candidates fail applied rounds is jumping to "I'd use a transformer" before doing steps 1–4. Interviewers are testing whether you can **scope an ambiguous problem**. Spend the first third of your answer on clarifying, framing, and metrics. The candidate who asks "what's the cost of a false positive here?" has already outscored the one who names an architecture in sentence one.

22.2 Diagnostic scenarios ("something is broken, fix it")

These test debugging instinct, the skill frontier labs prize most. The pattern is always: **isolate the failure, form hypotheses, test the cheapest one first.**

S1. "Your model has 99.8% accuracy on a medical diagnosis task. Ship it?" No, and the accuracy is a red flag, not a triumph. First, what is the base rate? If the disease affects 0.2% of patients, a model that always predicts "healthy" scores 99.8% and catches nothing, so accuracy is meaningless here. I would demand precision, recall, and especially recall/sensitivity (missing a sick patient is the costly error), on a PR curve, and I would look at the confusion matrix directly. Second, 99.8% on a genuinely hard task screams **data leakage**, so I would audit every feature for information unavailable at diagnosis time and check that no patient appears in both train and test. Third, I would check subgroup performance (does it work across ages, sexes, ethnicities?) because an aggregate number can hide a group where it fails badly. Only after all that, with an operating point chosen from the clinical cost of each error, would I discuss deployment, and even then behind human review.

S2. "Training loss is going down but validation loss is going up. What's happening and what do you do?" That is the textbook signature of **overfitting**: the model is memorising the training set rather than learning generalisable patterns. In order of cost: add or strengthen regularisation (L2/weight decay, dropout), add early stopping (which directly catches this), reduce model capacity or feature count, and, most effective if available, get more data or add augmentation. I would also plot the learning curve, because if train and val were still converging with more data, more data is the fix and I should not waste time tuning regularisation. If instead validation loss is *lower* than training loss, that is usually dropout or augmentation being active only at train time, which is normal, so I would check before panicking.

S3. "Your churn model scored 0.95 AUC in development and 0.62 in production. Diagnose it." (a classic FAANG question) That gap is the signature of leakage or shift. First, **leakage**: I list every feature and ask "would this value exist, with this value, at the moment we predict?" Anything populated after the outcome (a cancellation reason, a final invoice, a field with `_at_churn`) is an immediate suspect. Second, the **split**: a random split on time-ordered data lets the model train on the future, so I redo it as a temporal split and re-measure. Third, **duplicates** and the same customer in both splits. Fourth, if none of that explains it, **distribution shift** between the training window and live traffic, checked with a KS test per feature, plus training-serving skew from features computed differently offline and online. In practice it is leakage the large majority of the time.

S4. "A fraud model has 0.94 ROC-AUC and the business says it's useless. What happened?" The model probably ranks well but the operating point is wrong, and ROC-AUC is hiding it. At 0.2% fraud prevalence, a 5% false-positive rate looks fine on a ROC curve but means thousands of false alarms per hundred thousand transactions, drowning the few hundred real cases, so precision is a few percent. I would switch the headline to **PR-AUC**, plot precision and recall against the threshold, and set the operating point from the review team's real capacity ("we can review 500 alerts a day, so what recall does that buy?"). I would also check calibration, because if scores feed an expected-loss decision the probabilities must be honest, not just correctly ordered.

S5. "Your RAG chatbot gives a confidently wrong answer, but the correct document was retrieved. Where's the bug?" Retrieval succeeded, so this is a **generation** failure, not a retrieval one, and separating those is the whole skill. The model had the right context and still hallucinated or answered a different question. Fixes: strengthen the "answer only from the provided context, and say you don't know if it's absent" instruction; lower temperature to 0; check whether too many chunks were passed so the relevant one got lost in the middle (top-k of 4–8 is usually best, and faithfulness degrades as attention dilutes over more chunks); and add a faithfulness check (an LLM-as-a-judge verifying every claim traces to a retrieved chunk). If instead the right document had *not* been retrieved, it would be a retrieval bug and I would fix top-k, hybrid search, or chunking. Naming which half broke, and proving it by inspecting the retrieved chunks, is the answer.

S6. "Your image classifier works in the lab at 95% but drops to 70% when deployed on phone camera photos. Why?" Almost certainly **distribution shift**: the training images and real phone photos differ in lighting, angle, resolution, backgrounds, and compression. The model overfit to lab conditions. I would compare the two distributions, then fix it primarily with **data**: collect and label real in-the-wild photos, and heavily augment training (random crops, rotations, brightness/contrast, blur, JPEG artefacts) to simulate deployment conditions. I would also check for a subtler skew: preprocessing that differs between training and the phone app (resize method, normalisation constants), which is a training-serving skew and a very common silent culprit.

S7. "An LLM feature works great in testing and then degrades over weeks in production. What's going on?" Several candidates, and I would instrument to tell them apart. **Concept drift**: the world changed (new products, new slang, new question types) so the same prompts now underperform, which I catch by monitoring answer quality against sampled human labels. **Input drift**: users started asking different kinds of questions than the eval set covered. **Silent dependency changes**: if it calls a hosted model that was updated, behaviour shifts underneath you, so I pin versions and keep a regression eval suite. **Data/KB staleness** for RAG: the knowledge base drifted from reality, or retrieval quality decayed as documents were added. The through-line is that generative systems need continuous evaluation, not a one-time launch check, and I would have a standing eval set plus production monitoring from day one.

22.3 Design scenarios ("build this system")

These test whether you can scope and architect. Use the framework in 22.1. The answers are sketches; a real answer is a 10–20 minute conversation.

S8. "Design a system to detect toxic comments on a large platform." (Meta/Google style) Clarify: what counts as toxic, what languages, what scale (comments per second), what latency, and what is the cost of a false positive (censoring a legitimate user) versus a false negative (letting abuse through)? Frame as multi-label text classification (toxicity has types: harassment, hate, threats). Metric: because the cost is asymmetric and classes are imbalanced, PR-AUC and per-class recall at a precision floor, plus fairness checks across groups so the model does not disproportionately flag dialects or reclaimed terms. Data: human-labelled comments with strong annotation guidelines and inter-annotator agreement (Cohen's kappa); the label noise here is a first-order problem. Approach: a strong baseline (TF-IDF + linear, or a fine-tuned small transformer), then a larger fine-tuned model if justified. Serving: real-time with a latency budget; a two-stage design where a cheap model screens everything and a heavier model or human reviews the uncertain middle. Production: monitor drift (abuse evolves adversarially), keep humans in the loop to generate fresh labels, and expose an appeals path because false positives harm real users.

S9. "Design a recommendation system for a video platform." (the most common design question) Frame as ranking. Two-stage architecture, which is the key insight: a **retrieval** stage (a two-tower model embedding users and videos, then approximate-nearest-neighbour search over millions of items to get a few hundred candidates) and a **ranking** stage (a richer model, gradient boosting or a deep net, scoring those few hundred on predicted engagement). Offline metric NDCG; online metric watch time or a composite of engagement and long-term retention, decided by A/B test, with guardrails on diversity so it does not collapse into clickbait. Features: watch history, video and creator embeddings, freshness, context (time, device). Cold-start via popularity and content features for new users and items. Retrain frequently and watch for feedback loops, where the model shapes future data and narrows what users see. Note that this retrieve-then-rank pattern is the same one used in search and in RAG.

S10. "Design an inference batching system for serving an LLM on a single GPU." (reported at Anthropic) The goal is to maximise GPU throughput and utilisation under a latency SLA. The core idea is **batching**: process many requests together so the GPU's parallelism is used, but naive static batching wastes time waiting for a full batch and stalls on the longest sequence. I would use **continuous (in-flight) batching**, where new requests join the batch as soon as slots free up rather than waiting for the whole batch to finish, which is what vLLM's PagedAttention enables. Key components: a request queue, a scheduler that forms batches respecting the latency budget, a **KV-cache** manager (the main memory constraint, so paged/blocked allocation to avoid fragmentation and to fit more concurrent sequences), and prefill-vs-decode handling (the prompt is compute-bound, each generated token is memory-bandwidth-bound, so they batch differently). Trade-offs to name: throughput versus tail latency, memory for KV cache versus batch size, and fairness across requests of different lengths. I would measure tokens/second, p50/p99 latency, and GPU utilisation.

S11. "Design an evaluation framework for a new LLM feature before launch." (frontier-lab favourite) Start from what "good" means for this feature and who is harmed by each failure. Build a **layered eval**: (1) an offline benchmark set of representative and adversarial inputs with rubrics, scored by LLM-as-a-judge (with G-Eval-style reasoning) calibrated against a human-labelled subset, covering correctness, helpfulness, and format; (2) **safety and red-team** evals for harmful, biased, or policy-violating outputs, including jailbreak attempts, because for a customer-facing feature a single bad output erodes trust; (3) if it is RAG, component evals for retrieval (context precision/recall) and generation (faithfulness, answer relevance); (4) if it is agentic, trajectory evals for task success and tool-call correctness. Cap the metric set (a handful, not fifty) so it forces prioritisation. Gate launch on thresholds (stricter for customer-facing than internal), then run an **online A/B test** with guardrail metrics and staged rollout, because offline evals guide iteration but only production traffic proves impact. Keep the eval suite as a regression gate for every future model or prompt change.

S12. "Design a system to predict whether a loan applicant will default." (finance, and a fairness minefield) Frame as binary classification with a strong regulatory overlay. Clarify the outcome window and the decision it drives (approve, deny, price). Metric: AUC or PR-AUC for ranking, but **calibration is essential** because probabilities feed an expected-loss and pricing decision, so I would calibrate and check reliability curves. Crucially, this is a domain where **interpretability is often legally required**: I would favour a model I can explain per-decision (logistic regression, or gradient boosting with SHAP) so I can tell an applicant why they were declined, and I would run **fairness** analysis across protected groups, knowing the fairness metrics conflict and choosing which error to equalise is a value and compliance decision, not a pure optimisation. I would guard hard against leakage and against proxies for protected attributes (a zip code can encode race). The biggest risks here are legal and ethical, not accuracy, and saying so scores well.

S13. "Design a semantic search / 'find similar products' feature." Frame as embedding plus nearest-neighbour retrieval. Offline: embed every item (a fine-tuned encoder, or a CNN/ViT for images, trained with a contrastive or triplet loss so similar items are close). Online: embed the query and do approximate-nearest-neighbour search (HNSW in a vector DB) over the catalogue, then optionally rerank the top candidates with a heavier cross-encoder for precision. Metric: Precision@k and Recall@k against human relevance judgements, plus click-through online. This is architecturally identical to text RAG retrieval, which is worth pointing out to show you see the unifying pattern, and the reranking step is the same highest-ROI upgrade as in RAG.

22.4 Understanding scenarios ("do you really get it?")

These probe conceptual depth with a twist that rote memorisation cannot answer.

S14. "You add a feature that is highly correlated with your target, and performance gets worse. How is that possible?" Several ways. Most likely **leakage**: the feature is highly correlated because it encodes the target itself or something computed after the prediction moment, so it helps offline and vanishes or misleads in production, and if you already split, its correlation is spurious. Or **multicollinearity**: if it duplicates an existing feature, it can destabilise a linear model's coefficients. Or it added **noise/overfitting** capacity that hurt generalisation on a small dataset. Or the correlation is with the *training* labels but the feature is unavailable at inference. The instinct to suspect leakage first, and to verify by checking when the feature becomes known, is the point.

S15. "Why might a simpler model outperform a deep neural network on your tabular dataset?" On tabular data of typical size, gradient-boosted trees very reliably beat deep nets: they handle mixed feature types and non-linear interactions natively, need no scaling, are robust to irrelevant features and outliers, and do not need the huge data volumes deep nets require to shine. Neural nets excel where structure can be exploited by architecture (spatial in images, sequential in text) and where data is abundant; a plain tabular problem offers neither, so the deep net overfits or underperforms while a boosted model wins at a fraction of the effort. The mature answer also notes: try the simple model first, and if it is within a point or two, its interpretability and lower serving cost may be worth more to the business than marginal accuracy.

S16. "Your two models have identical accuracy. How do you choose between them?" Accuracy alone is nearly never the deciding factor. I would compare: the full metric picture (precision/recall trade-off, calibration, performance on the minority class and across subgroups); robustness (how they degrade under drift or adversarial input); interpretability (can I explain a decision, which may be legally required); latency and serving cost; and maintainability. A model that is equally accurate but calibrated, explainable, faster, and cheaper to serve is the better choice. If it is genuinely a coin-flip on everything that matters, I would prefer the simpler, more interpretable one, per Occam.

S17. "Explain why a model with high bias won't benefit from more data, but a high-variance model will." A high-bias model is too simple to capture the true relationship, so it is already making systematic errors that more of the same data will not fix; the fix is more capacity or better features. A high-variance model is capturing noise specific to its training sample, and more data averages that noise out and constrains the model toward the true signal, so it generalises better. This is exactly what the learning curve shows: a converged small gap means more data will not help (increase capacity), while a large persistent gap means more data will (reduce variance). Reading the learning curve to decide which lever to pull is the practical skill behind the theory.

S18. "When would you deliberately choose a less accurate model?" When something other than accuracy dominates: a hard latency budget (a slower model misses the SLA), a regulatory requirement for explainability (a black box is not deployable regardless of accuracy), fairness constraints, limited serving budget, the need for calibrated probabilities, or robustness and maintainability. Also when the accurate model's gains do not translate to the business metric, since offline accuracy is only a proxy. Being willing to trade accuracy for the constraint that actually matters is a maturity signal, not a weakness.

22.5 Safety, ethics, and responsible-AI scenarios (essential for frontier labs)

Frontier labs (and increasingly everyone) run explicit safety/ethics rounds. The pattern: identify harms, reason about trade-offs, and propose concrete mitigations rather than platitudes.

S19. "Your conversational model gives overconfident but factually wrong answers in high-risk contexts (medical, legal, financial). How do you address it?" (mirrors a reported Anthropic panel) This is a calibration-and-safety problem with several layers. First, the model has no internal "I don't know" signal and produces plausible text regardless of correctness (hallucination), so I would (1) ground it with **RAG** and citations so answers trace to sources and are checkable, (2) train or prompt it to **express uncertainty and defer** in high-risk domains rather than answer confidently, (3) add **guardrails** that detect high-risk topics and route to a safe response, a disclaimer, or a human, (4) build a **safety eval and red-team suite** specifically for these domains and gate on it, and (5) monitor production for confident-wrong outputs and feed failures back. The framing that matters: in high-risk contexts the cost of a confident wrong answer is severe and asymmetric, so the right behaviour is often to refuse or defer, and the system should be designed to make that the default, not an afterthought.

S20. "You're asked to build a model that could be misused. How do you think about it?" I would reason about the harm explicitly rather than refuse reflexively or comply blindly. Who could be harmed, how likely and how severe, and what is the legitimate use? Concrete steps: scope the capability to the legitimate use and design against the misuse (access controls, rate limits, monitoring, refusal behaviours for harmful requests), do a risk assessment, and escalate genuinely dangerous capabilities rather than deciding alone. Document intended use and limitations (a model card). The interviewer is testing structured ethical reasoning and a willingness to say "I would not ship this as specified, and here is a safer version," which is stronger than either extreme.

S21. "How would you detect and mitigate bias in a hiring model?" Detect: measure performance and error rates across protected groups (not just aggregate accuracy), check for disparate impact, and audit features for proxies (a school, a zip code, or a gap in employment can encode protected attributes). Understand the source: historical hiring data encodes past discrimination, so a model trained to reproduce it will perpetuate it, and the labels themselves may be biased. Mitigate at three points: data (reweight or rebalance, remove proxies), in-processing (fairness constraints in the objective), and post-processing (group-calibrated thresholds), while knowing the fairness definitions (demographic parity, equal opportunity, equalised odds) conflict and choosing among them is a value decision to make with stakeholders, not silently. Add human oversight, document with a model card, and question whether ML should make this decision at all versus assisting a human. The key insight: a model can be perfectly accurate at predicting a biased past and still be unfair, so fairness is not a subset of accuracy.

S22. "A stakeholder wants to deploy a model you think isn't ready. How do you handle it?" I would make the risk concrete and evidence-based rather than just object: show the failure modes (subgroup gaps, calibration issues, drift risk, the specific bad outputs), quantify the cost of those failures in business terms, and propose a lower-risk path, such as a limited rollout behind human review, a shadow deployment to gather real-world evidence, or an A/B test with tight guardrails and instant rollback. I would document the risks and my recommendation. The goal is to protect users and the business while respecting that shipping is a judgement call involving trade-offs I do not solely own, and to give the decision-maker the information to choose well.

22.6 Coding-under-pressure scenarios

Frontier labs (Anthropic among them) often ban AI tools in the screen and expect clean code against test cases, sometimes with an ethics discussion attached. Beyond the from-scratch implementations in Part XIX, be ready to: - **Implement a metric from scratch** (precision/recall/F1 from a confusion matrix, AUC via ranking, NDCG) and explain it. - **Write a data-processing task in standard library only** (no pandas/NLP packages): parse and extract from messy text, deduplicate, aggregate. Reported at Anthropic. - **Diagnose and fix a broken training loop** (the missing `zero_grad()`, the wrong loss input, the leaking split). - **Reason about a reliability error message** and propose a fix that improves model reliability on long-running tasks, walking through prompting techniques and their trade-offs. Reported at Anthropic's practical screen. - **Vectorise a slow loop** and explain the speedup.

> **KEY IDEA — what the coding round actually tests** At the top labs the algorithm itself is usually straightforward; they are watching *how* you work: do you clarify the spec, handle edge cases, write readable code, test your own work, reason out loud, and recover gracefully when stuck. Practise without AI assistance and narrate your thinking. And when an ethics or trade-off question is attached to the coding task, treat it with the same rigour as the code, because they are scoring both.

22.7 Domain-specific scenarios (ML applied across industries)

These test whether you can map a messy business problem onto ML, choose the right framing, and anticipate the domain's specific traps. The interviewer often does not care about the exact model; they care that you ask the right questions first.

S23. Healthcare — "A hospital wants to predict which patients will be readmitted within 30 days." Clarify the decision this drives: is it to allocate follow-up nurse calls, or to flag discharge risk? That changes the operating point. Frame as binary classification. Metric: recall/sensitivity is paramount (missing a high-risk patient is the costly error), but at a precision the care team can actually action, so PR-AUC and a threshold set by nursing capacity. Data traps specific to healthcare: severe class imbalance, missing values that are informative (a test not ordered is a signal, so add missingness flags), and heavy leakage risk from fields populated during or after the readmission. Interpretability is often required so clinicians trust and act on it, so gradient boosting with SHAP over a black box. Fairness across demographics is a regulatory and ethical must. And I would frame it as decision support for a clinician, not an autonomous decision.

S24. E-commerce — "Predict customer lifetime value (CLV) for a new user after their first purchase." Frame as regression (or as a two-part model: probability of repeat purchase times expected spend). Metric: because CLV is right-skewed with a long tail, RMSE is dominated by whales, so I would consider RMSLE or modelling log-CLV, and report MAE for interpretability. The core challenge is the tiny signal from a single purchase, so feature engineering is everything: first-order value, category, acquisition channel, time-of-day, device. Cold-start is inherent (one data point), so I would lean on population priors and segment-level averages early. A key trap: survivorship and censoring, since recent users have not had time to accumulate value, which biases the target if not handled with a proper observation window.

S25. Logistics — "A warehouse wants to forecast daily demand per SKU to optimise inventory." Frame as time-series forecasting, per SKU, which means thousands of series. Metric: because over- and under-forecasting cost differently (stockout vs holding cost), a quantile/pinball loss beats symmetric error, and I would evaluate with a business cost function. Approach: always start with strong baselines (seasonal-naive, moving average) because they are famously hard to beat, then gradient boosting with lag and calendar features (which wins many forecasting competitions and handles many SKUs well), and only then deep models if warranted. Critical: time-based validation with no leakage, handle intermittent demand (many zeros) for slow-moving SKUs differently, and account for promotions and holidays as external regressors.

S26. Finance — "Build a credit-risk model where regulators require you to explain every rejection." The regulatory constraint dominates the design (this is the loan scenario deepened). Interpretability is not optional, so I favour logistic regression (with monotonic constraints and reason codes) or gradient boosting with SHAP, never an unexplainable black box. Calibration is essential because outputs feed pricing and expected-loss decisions. I would guard hard against proxies for protected attributes (zip code can encode race), run fairness analysis, and document with a model card. The mature point: here a 2% accuracy gain from a black box is worthless if it is not deployable under regulation, so the "best" model is the best *explainable* one.

S27. Manufacturing — "Detect defective parts on a production line from camera images, but you have very few defect examples." Two intertwined problems: computer vision plus extreme imbalance. With few defects, I would not train a defect classifier from scratch; I would frame it as **anomaly detection** (train an autoencoder or use a pretrained feature extractor to model "normal" parts and flag high reconstruction error / distance), which needs only normal examples. As defects accumulate through inspection, transition to supervised. Heavy augmentation to simulate defect variety. Metric: recall is critical (a shipped defect is costly), balanced against false-alarm rate that would halt the line. Deploy at the edge with a latency budget, and expect distribution shift as lighting and camera conditions drift.

S28. Marketing — "Which users should we target with a retention campaign, given a limited budget?" The subtlety: this is really an **uplift / causal** problem, not a plain churn-prediction one. Targeting the users most likely to churn wastes budget on lost causes and on people who would have stayed anyway. What you want is the users whose behaviour the campaign will *change* (the "persuadables"), which requires uplift modelling (two-model or a causal forest) ideally trained on a randomised holdout. Metric: uplift/Qini curve, not AUC. If uplift modelling is not feasible, at minimum target by predicted churn probability and measure the campaign with a proper A/B test. Recognising that "who will churn" and "who should we target" are different questions is exactly the insight that scores.

S29. Social / content — "Rank a user's feed to maximise engagement without creating harmful filter bubbles." Frame as ranking (the recommendation two-stage design), but the twist is the **objective and its guardrails**. Optimising pure engagement drives clickbait, outrage, and filter bubbles, so the objective must be a composite: engagement plus diversity, plus long-term retention (not just next-session clicks), plus integrity signals that demote harmful content. Metric: NDCG offline, but online A/B on a balanced scorecard with guardrails on diversity, well-being proxies, and policy violations. The important point to raise unprompted: metric design is an ethical decision here, because the model will ruthlessly optimise exactly what you tell it to, so a naive engagement target produces predictable social harm.

S30. Autonomous systems / safety-critical — "Your model controls a physical system; a wrong prediction can hurt someone. How does that change your approach?" Everything gets more conservative. The cost of a false negative (or any error) can be catastrophic and irreversible, so: I would demand extremely high recall on the danger class, calibrated uncertainty so the system can say "I'm not sure" and fail safe, extensive testing on edge cases and adversarial/out-of-distribution inputs, and a human or hard-coded safety layer that can override the model. I would prefer interpretable or verifiable components where possible, monitor relentlessly, and design the system so the *default* action is safe. The framing that matters: in safety-critical ML, the right behaviour under uncertainty is to defer or fail safe, and that must be designed in, not bolted on.

22.8 Data and experimentation scenarios

S31. "How would you design an A/B test to decide whether your new model is better?" Define the hypothesis and the single primary metric up front (with guardrail metrics), and decide the minimum effect size worth detecting. Compute the required sample size and run duration from that effect size, the baseline rate, and the desired power (usually 80%) and significance (usually 5%), so I do not peek and stop early. Randomise at the right unit (user, not request, to avoid contamination) and check the groups are balanced. Run for full business cycles (at least a week to cover weekly seasonality). Analyse with the pre-registered test, correct for multiple comparisons if I look at many metrics, and watch for novelty effects. Only ship if the primary metric moves significantly and guardrails do not regress. The disciplined "define metric and sample size before running, don't peek" answer is the whole point.

S32. "Your A/B test shows the new model wins on click-through but loses on revenue. What do you do?" Do not ship on the secondary metric alone. First, this is a classic case where the proxy (CTR) and the true objective (revenue) diverge, so I would trust the metric that reflects business value. I would investigate *why*: perhaps the model surfaces cheaper or more clickbait items that get clicks but not purchases, or it shifts the mix of what users buy. I would segment the effect (is it all users or a subgroup?), check for novelty effects, and look at longer-term metrics if the revenue drop might be short-term. The decision depends on the strategic goal, but the instinct to prioritise the metric tied to actual value over a vanity metric, and to explain the divergence, is what scores.

S33. "You have 10 million rows but can only label 10,000. How do you choose which to label?" This is active learning. Rather than labelling randomly, I would label the examples the model is most **uncertain** about (near the decision boundary), or the most **diverse/representative** to cover the space, or a mix, in iterative rounds: train, query the most informative unlabelled points, label them, retrain. I would also stratify to ensure rare classes are represented, since random sampling would barely touch a 1% class. And I would keep a small randomly-sampled labelled set as an unbiased test set, because an actively-sampled set is biased and cannot be used to estimate true performance.

S34. "Two data scientists get different results on the 'same' analysis. How do you reconcile it?" I would treat it as a reproducibility investigation: are they using the same data snapshot, the same preprocessing, the same train/test split and seed, the same metric definition, the same library versions? Differences usually come from an unfixed seed, a different split, a subtly different metric (macro vs weighted F1), a leakage difference, or a data version mismatch. The fix and the lesson: pin seeds, version the data and code, share a single pipeline, and log everything (MLflow), so results are reproducible by construction. This is a maturity and collaboration signal as much as a technical one.

22.9 LLM-specific applied scenarios

S35. "A user reports the LLM leaked another user's data in its response. What happened and how do you prevent it?" This is a serious security and privacy incident, so first: contain, assess scope, and follow the incident process. Likely causes: cross-user context bleeding (a shared cache or a prompt that mingled sessions), the model regurgitating training data (memorisation of PII), or a RAG system retrieving documents the user should not access (missing access control on the vector store, the multi-tenancy problem). Prevention: strict per-user isolation and namespacing in retrieval, access-control checks *before* retrieval, output filtering / PII detection, not training on sensitive data (or using differential privacy), and eval/red-team suites specifically for data-leakage. The framing: in multi-tenant LLM systems, access control belongs at the retrieval layer and isolation must be designed in.

S36. "Your LLM app works for English but fails for other languages. Why, and what do you do?" Likely causes: the base model saw far less non-English data (weaker capability), tokenisation is less efficient for other scripts (more tokens, higher cost, truncated context), the RAG corpus and embeddings are English-centric so retrieval fails cross-lingual, and your eval set is English-only so you never measured the gap. Fixes: choose a strongly multilingual base model, use multilingual embeddings for retrieval, build eval sets per language, and consider translation as a fallback. The key insight: "works in English" is a biased evaluation, and the first fix is to measure the other languages before assuming the model is the problem.

S37. "How would you reduce hallucinations in a customer-facing LLM assistant?" Layered defence. Ground it with **RAG** and require answers to cite retrieved sources so claims are checkable, and add a **faithfulness check** (an LLM-as-judge verifying every claim traces to a source). Prompt and, if needed, train it to say "I don't know" rather than guess, and set temperature low for factual tasks. Add **guardrails** that detect out-of-scope or high-risk questions and defer to a human or a safe response. Constrain the output format where possible. Continuously evaluate hallucination rate in production and feed failures back. And manage expectations: hallucination cannot be driven to zero, so for high-stakes answers the system should defer rather than risk a confident error.

S38. "You need to choose between prompting, RAG, and fine-tuning for a task. Walk me through the decision." I decide by what the task actually needs. If it needs the model to **know facts** that are large, changing, or requiring citations, use **RAG** (cheap to update, grounded, citable). If it needs to **behave** a certain way, adopt a format, tone, or a narrow skill, use **fine-tuning** (it changes behaviour, not knowledge). If the need is small, a few examples or a style demonstration, **prompting / few-shot** is fastest and free. These compose: a production system often uses RAG for facts and light fine-tuning for format. The common trap I would call out: most teams reach for fine-tuning when they actually need RAG, because they conflate "the model doesn't know X" (a knowledge problem, RAG) with "the model doesn't do X the way I want" (a behaviour problem, fine-tuning).

S39. "Your RAG system's answers got worse over three months even though you didn't change the code. Why?" The code is static but the *data* is not. Likely: the knowledge base grew, and more documents mean retrieval now surfaces more near-duplicates or off-topic chunks (context precision dropped), diluting the model's attention. Or the corpus drifted from what users ask about (concept drift in the queries). Or documents became stale/contradictory as new ones were added without removing old ones. Or an upstream embedding/model version changed under you. Fixes: monitor

retrieval quality (context precision, hit rate) over time, not just at launch; add reranking to keep precision high as the corpus grows; deduplicate and lifecycle-manage documents; pin model versions and keep a regression eval. The lesson: RAG systems rot through data drift, so retrieval quality needs continuous monitoring.

S40. "How would you build guardrails for an agent that can take real actions (send emails, make purchases)?" Because actions are consequential and irreversible, I would design defence in depth: **human-in-the-loop approval** before any consequential action (the interrupt-before-tools pattern), a hard **recursion/iteration limit** and spending caps so a stuck loop cannot run up cost, **allow-lists** for tools and destinations, validation of tool arguments before execution, a dry-run/preview mode, comprehensive **logging and traceability** of every decision, and rollback where possible. I would also add input filtering against prompt injection (a document telling the agent to do something malicious) and evaluate the agent on adversarial trajectories. The framing: the more powerful and irreversible the action, the more the system must require explicit human confirmation and constrain what is even possible.

22.10 Rapid-fire applied judgment (short scenarios)

Interviewers sometimes fire a series of quick "what would you do" prompts to test instincts. One-to-two-sentence answers:

- **"Model works in dev, fails in prod."** → Suspect leakage or train-serving skew first; check that features are computed identically and the split respected time.
- **"Accuracy is 95% but the model is useless."** → Check the base rate; on imbalanced data accuracy is meaningless, switch to precision/recall/PR-AUC.
- **"Should you remove outliers?"** → Investigate first; an outlier is either an error to fix or the signal itself (fraud, defects). Never blindly drop.
- **"Your feature importance says a nonsensical feature is most important."** → Likely leakage; that feature probably encodes the target. Also check it's permutation importance on held-out data, not biased impurity importance.
- **"Training is very slow."** → Profile first; check batch size, data loading (often the bottleneck, not compute), mixed precision, and whether the GPU is actually being used.
- **"Loss is NaN."** → Exploding gradients or a bad learning rate; lower the LR, add gradient clipping, check for log(0) or division by zero, normalise inputs.
- **"Your model is biased against a group."** → Measure per-group error rates, audit features for proxies, and mitigate at data/training/threshold level; recognise fairness definitions conflict.
- **"You have almost no labelled data."** → Consider self-/semi-supervised methods, transfer learning, active learning, weak supervision, or an unsupervised framing.
- **"Which is better, model A or B, if accuracy is tied?"** → Compare calibration, subgroup performance, robustness, latency, interpretability, and cost; prefer the simpler one if truly tied.
- **"Stakeholder wants a model for something with no signal in the data."** → Say so honestly; propose collecting the right data or a non-ML solution rather than shipping noise.

22.11 The meta-questions ("how do you think")

Frontier labs probe your judgment and self-awareness, not just knowledge.

S41. "Tell me about a time your model failed in production. What did you learn?" Have a real STAR story ready: the situation, what broke, how you diagnosed it (this is where you show debugging instinct), the fix, and the lesson that changed how you work. The best stories are about leakage, drift, an unstable model, or a metric that did not match the business goal. Interviewers value the honesty and the diagnostic process far more than a flawless record, and a candidate who cannot name a single failure reads as inexperienced.

S42. "How do you stay current in a field that changes this fast?" Concrete habits: reading key papers and lab blogs (and being able to name recent ones relevant to the role), reproducing results, following practitioners, and, most convincingly, *building* with new techniques rather than just reading about them. Tie it to the specific company's work. The signal is genuine, applied curiosity, not a list of newsletters.

S43. "When would you tell a stakeholder that machine learning is the wrong tool?" When a deterministic rule covers the case (cheaper, testable, explainable), when there is too little data, when errors are catastrophic and unexplainable decisions are unacceptable, when a simple heuristic captures most of the value, or when the relationship is genuinely random. Being willing to argue *against* ML when it is not warranted is a strong maturity signal and shows you optimise for the business outcome, not for using fancy tools.

S44. "How would you explain [your model / a false positive / why it can't be 100% accurate] to a non-technical executive?" Drop the jargon and anchor to their decision and to cost. For a false positive: "sometimes it flags a good customer as risky; that costs us X in annoyance, versus missing a real risk which costs Y, so we tuned it to trade those off this way." For "why not 100%": "there's inherent noise and ambiguity in the data, so no model, or human, can be perfect; our job is to be reliably better than the current process and to fail safely." The skill being tested is translating technical reality into business terms, which is a large part of a senior ML role.

Part XXIII — The Project Portfolio: What to Build to Get Hired

> **KEY IDEA** Reading this guide makes you able to *answer* questions. Building the projects below makes you able to *say "yes, I built that"* to the follow-up, which is what actually converts an interview into an offer. As both companion guides stressed: interviewers want trade-offs and war stories, not definitions, and the only way to have a war story is to have shipped something and watched it break. A candidate with three deployed projects they can discuss for fifteen minutes at increasing depth beats a candidate who has memorised twice as many definitions.

How to use this part. You do not need all of these. Pick a spread that covers classical ML, deep learning, and the LLM/production stack, and for each one, do the thing that most candidates skip: **deploy it, document it, and be able to talk about what went wrong.** For every project, three deliverables matter more than the model: (1) a clean, reproducible repo with a README that states the problem, the metric, and the result; (2) a short write-up of one hard problem you hit and how you diagnosed it; (3) ideally a live demo link. A modest model with rigorous methodology and a clear story beats a fancy model with a leaky split, every single time.

23.1 The rules that make a project count

Before the list, the discipline that separates a portfolio project from a tutorial:

- **Always build a baseline first.** A majority-class predictor or a linear model. If your deep model does not beat the baseline, that is the finding, and knowing it is a strength.
- **No leakage.** Fit every transformer inside the CV fold, use a `Pipeline`, split time-ordered data by time. This is the first thing a good interviewer probes.
- **Pick the right metric and justify it.** Never report accuracy on imbalanced data. State the metric and why.
- **Reproducibility.** Fixed seeds, pinned dependencies, a README anyone can run. Use MLflow to track experiments.
- **Deploy it.** Even a Streamlit or FastAPI demo. "I put it behind an API and containerised it" is worth more than another 1% of accuracy.
- **Write the post-mortem.** The one paragraph on what broke (leakage, drift, imbalance, an unstable model, a bad chunk size) and how you found it is your best interview material.

23.2 Tier 1 — Foundational projects (classical ML, prove the fundamentals)

P1. End-to-end tabular prediction with a leak-free pipeline. Take a real tabular dataset (churn, credit default, housing prices) and build the full workflow: EDA, a `Pipeline` doing imputation/encoding/scaling, a logistic-regression baseline, then gradient boosting (XGBoost/LightGBM) tuned with early stopping and Optuna, evaluated with the correct metric and stratified CV. Add SHAP for interpretation. *What it proves:* the whole classical workflow, leakage avoidance, metric choice, tuning, interpretability. This is the single most important foundational project because nearly every applied interview implies it. *The war story to earn:* deliberately introduce a leaky feature, watch the CV score jump, then catch it. Now you can answer the 0.95-to-0.62 scenario from experience. *Stretch:* handle class imbalance properly (class weights vs SMOTE-in-fold, compared honestly), calibrate the probabilities, and tune the decision threshold on a cost function.

P2. Imbalanced fraud/anomaly detection. On a highly imbalanced dataset (credit-card fraud is the classic), compare approaches: class weighting, SMOTE inside folds, and an Isolation Forest / autoencoder anomaly-detection framing for the near-no-labels case. Evaluate with PR-AUC and precision/recall at a chosen operating point, not accuracy. *What it proves:* you understand the accuracy paradox, PR-AUC, threshold selection, and the unsupervised-to-supervised transition (scenario S from Part VI).

P3. Customer segmentation with clustering. Cluster customers (RFM features work well) with K-Means and DBSCAN/HDBSCAN, choose k with the silhouette score, reduce dimensions with PCA/UMAP for visualisation, and translate the clusters into a business narrative ("these are five actionable segments"). *What it proves:* unsupervised methods, cluster evaluation, and, crucially, communicating a model to non-technical stakeholders.

23.3 Tier 2 — Deep learning projects (prove you understand the architectures)

P4. Image classifier via transfer learning, in both frameworks. Fine-tune a pretrained ResNet (or EfficientNet/ViT) on a custom image dataset in *both* PyTorch and Keras, with data augmentation, early stopping, and a proper train/val/test split. Compare the training-loop code and the results. *What it proves:* CNNs, transfer learning, the training loop, and framework fluency (the PyTorch-vs-Keras trade-off from experience, not a blog post). *The war story:* delete `zero_grad()` and watch training silently fail; leave augmentation on at eval and watch metrics look wrong. Now the top-three-PyTorch-bugs question is a story. *Stretch:* deploy it as a phone-photo classifier and watch accuracy drop on in-the-wild images (scenario S6), then fix it with augmentation, giving you a real distribution-shift war story.

P5. Text classifier, three ways. Classify text (sentiment, topic, toxicity) three ways and compare: TF-IDF + logistic regression (the strong classical baseline), an LSTM, and a fine-tuned transformer (DistilBERT). Report the accuracy/effort/latency trade-off across all three. *What it proves:* the NLP progression, why a simple baseline is often competitive, RNNs, and transformers. Directly supports the "simpler model wins" understanding questions.

P6. A from-scratch neural network (no framework). Implement a multi-layer perceptron in pure NumPy: forward pass, backprop by hand, an optimiser, trained on MNIST. Then reproduce it in PyTorch. *What it proves:* you actually understand backpropagation, gradients, and the training loop, not just `.fit()`. This is the project that lets you answer "explain backprop" and "implement it" without hesitation, which frontier labs love.

P7. A sequence model for time series or text generation. Build an LSTM/GRU for a real sequence task (forecasting with proper time-based validation, or character-level text generation) and, for forecasting, compare against a classical baseline (ARIMA, seasonal-naive) and a gradient-boosting-with-lag-features approach. *What it proves:* RNN/LSTM understanding, and (for forecasting) time-series validation without leakage, plus the humbling fact that

simple baselines are hard to beat.

23.4 Tier 3 — LLM and production projects (prove you can ship modern systems)

These are the highest-signal projects for 2025-era roles and connect directly to the first two guides. Build at least one Tier-3 project end to end and deploy it.

P8. A production-minded RAG system with evaluation. Beyond the first guide's "chat with your PDF," build RAG that you actually *evaluate*: chunk and embed a document corpus into a vector DB, retrieve-augment-generate with citations, then build the **eval harness** (RAGAS-style faithfulness, context precision/recall, answer relevance, plus retrieval hit rate and MRR). Add hybrid search and a reranker and measure the improvement. *What it proves:* the whole RAG stack *and* that you can evaluate a generative system, which is the skill interviewers now test hardest. It lets you answer every RAG-debugging scenario (S5) from experience. *The war story:* set top-k too high and watch faithfulness degrade; break retrieval and watch the "confidently wrong but right doc retrieved vs wrong doc retrieved" distinction become concrete.

P9. A tool-using agent with guardrails and evaluation. Build a LangGraph agent (calculator, web search, a RAG retriever as a tool) with a recursion limit, human-in-the-loop approval before consequential actions, and an **agent eval** measuring task success rate and tool-call correctness across a test set. *What it proves:* agentic patterns, guardrail design (exactly what DeepMind's applied scenarios probe), and agent evaluation. Pairs with the agentic-AI guide.

P10. An LLM inference / serving project. Serve an open-weight model efficiently: use vLLM (or TGI) with continuous batching, measure tokens/second and p50/p99 latency under load, experiment with quantisation (int8/int4) and its quality-vs-speed trade-off, and reason about KV-cache memory. *What it proves:* the inference-optimisation and serving knowledge frontier labs test directly (scenario S10, the single-GPU batching design). This is a strong differentiator because most candidates have never touched serving internals.

P11. Fine-tune a model with LoRA/QLoRA. Fine-tune a small open model on a specific task or style with LoRA/QLoRA on a single GPU (Colab works), and compare it honestly against few-shot prompting and against RAG for the same goal, so you can speak to *when* fine-tuning is actually the right tool versus RAG or prompting. *What it proves:* PEFT, the RAG-vs-fine-tuning-vs-prompting decision (a guaranteed question), and efficient training.

P12. An LLM-as-a-judge evaluation pipeline. Build an evaluation system that scores model outputs against a rubric using an LLM judge (G-Eval style), calibrate it against a small human-labelled set, and measure and mitigate its biases (position, verbosity, self-preference). *What it proves:* the modern eval methodology, awareness of judge biases, and evaluation rigour, which is increasingly its own interview topic.

23.5 The capstone (the one for your CV headline)

P13. A full-stack, deployed, evaluated AI application. Combine the pieces into one shipped system: a RAG agent over your own documents *and* live web, with a FastAPI backend, a real frontend (React/Next, or Streamlit as a bridge), a vector DB, conversation memory, human-in-the-loop approval before consequential actions, citations on every answer, an evaluation suite gating changes, containerised with Docker and deployed to a cloud with basic monitoring on latency, cost, and retrieval quality. *What it proves:* essentially every line of a modern AI/ML job description, end to end, shipped. When an interviewer says "tell me about a project you've built," this is a fifteen-minute answer that touches classical rigour, deep learning, the LLM stack, evaluation, and production engineering. *Deploy it and put the link on your CV.* This single deliverable, done well and discussed honestly including its failures, does more for you than any number of un-deployed notebooks.

23.6 How to talk about your projects in the interview

Use STAR (Situation, Task, Action, Result) and **quantify**. For each project have ready: the problem and the metric, why you chose the approach, what the **baseline** was, one hard thing you **debugged** (leakage, drift, imbalance, an unstable model, a serving bottleneck), the **result** in real terms, and what you would **do differently**. Rehearse the deep-dive at three depths: a one-minute summary, a five-minute walkthrough, and a fifteen-minute technical deep dive where you can defend every decision. Be honest about limitations and failures, because "I understand X deeply, I understand Y conceptually and here is how I am closing that gap" earns far more trust than bluffing, and top-lab interviewers are specifically trained to catch the bluff.

> **THE FINAL WORD, FOR REAL THIS TIME** Three guides now sit behind you: the applied LLM/RAG/agent guide, the production and MLOps companion, and this foundations-and-interview guide. Together they cover what an international-level AI/ML interview probes, from the gradient of the logistic loss to the design of a single-GPU inference server to the ethics of shipping a model that could be misused. But no guide can do the last two things for you: **say the answers out loud until they are automatic**, and **build things, break them, and learn to tell the story**. That is why every checkpoint says "aloud" and this entire part is about shipping. Do the projects. Break them. Deploy them. Then walk into OpenAI, Anthropic, Google, or anywhere else, and talk about what happened. That is what gets the offer.

23.7 Tier 4 — Understanding projects (build to *learn*, not to show off)

These are small, focused builds whose entire purpose is to make a concept click by watching it behave. They take an afternoon each and produce the deep, from-experience answers that separate strong candidates. Do not deploy these; the learning *is* the deliverable.

P14. Visualise the bias-variance trade-off yourself. Fit polynomial regression of increasing degree to noisy data and plot training vs validation error against degree until you see the U-curve appear with your own eyes. Then plot a learning curve (error vs training-set size) for an underfit and an overfit model. *What it teaches:* bias-variance and how to read a learning curve to decide whether more data or more capacity is the fix, which underpins half the diagnostic scenarios.

P15. Break every metric on purpose. Build an imbalanced dataset, train a trivial majority-class classifier, and compute accuracy (watch it hit 99%), then precision, recall, F1, ROC-AUC, and PR-AUC, and watch which ones expose the uselessness. Sweep the decision threshold and plot precision and recall crossing. *What it teaches:* the accuracy paradox, why PR-AUC beats ROC-AUC on imbalance, and threshold selection, all from watching the numbers, so the metrics chapter becomes muscle memory.

P16. Reproduce a leakage disaster. Take a clean dataset, deliberately scale/impute before splitting (or add a target-derived feature), watch CV accuracy jump, then fix it with a proper `Pipeline` and watch it fall to honest levels. *What it teaches:* why leakage produces the 0.99-AUC red flag, and how a pipeline prevents it, so scenario S3 is a lived experience.

P17. Feel the curse of dimensionality. Empirically measure how the ratio of nearest to farthest pairwise distance approaches 1 as you add dimensions, and watch k-NN accuracy degrade as you pad a dataset with noise features. *What it teaches:* why distance-based methods fail in high dimensions and why dimensionality reduction matters.

P18. Implement and compare optimisers. On a 2D loss surface (a ravine), implement vanilla SGD, momentum, and Adam and animate their paths to the minimum. Then compare their convergence on a real training task. *What it teaches:* what momentum and adaptive learning rates actually do, so the optimiser question is visual and intuitive.

P19. Watch a network overfit and every regularizer fight it. Train a deep net with no regularisation until validation loss rises, then add dropout, weight decay, early stopping, and data augmentation one at a time and watch each flatten the gap. *What it teaches:* the concrete effect of each regularisation method, and why validation loss can dip below training loss when dropout is on.

P20. Break a training loop the three classic ways. Take a working PyTorch loop and, one at a time, remove `zero_grad()`, feed softmaxed values into `CrossEntropyLoss`, and skip `model.eval()` at test time, observing each silent failure. *What it teaches:* the top three PyTorch bugs from experience, a guaranteed coding-round topic.

P21. Demonstrate the vanishing gradient. Build a deep network with sigmoid activations, log the gradient magnitude per layer, and watch it shrink toward zero in early layers, then swap to ReLU and watch it survive. *What it teaches:* the single most important sequence/deep-learning concept, made concrete.

P22. Visualise word and sentence embeddings. Embed a vocabulary with Word2Vec (or a small transformer), reduce with PCA/UMAP, and plot it; verify king − man + woman ≈ queen; then show that a contextual model gives "bank" two different vectors in two sentences. *What it teaches:* embeddings, static vs contextual, and why semantic search works, connecting directly to the RAG guide.

P23. Build attention from scratch and visualise the weights. Implement scaled dot-product attention in NumPy, then run a small transformer and plot the attention matrix to see which tokens attend to which (e.g. pronoun to its antecedent). *What it teaches:* self-attention mechanically, and makes the "explain attention" and "why divide by √d_k" questions second nature.

P24. Calibrate a model and see the reliability curve move. Train a model, plot its reliability diagram (predicted vs actual probability), then apply Platt scaling and isotonic regression and watch the curve straighten. *What it teaches:* calibration, why AUC and calibration are independent, and when probabilities (not just rankings) matter.

23.8 Tier 5 — Extended portfolio projects (breadth across domains)

Deployable projects that broaden your portfolio into areas the earlier tiers do not cover. Pick ones matching the roles you target.

P25. A time-series forecasting system with proper validation. Forecast a real series (energy, sales, traffic) comparing seasonal-naive, ARIMA/SARIMA, Prophet, and gradient boosting with lag features, using time-based cross-validation and a business-relevant metric. *What it proves:* time-series fundamentals, leakage-free temporal validation, and that simple baselines are hard to beat. Supports the logistics/demand scenarios.

P26. A recommender system end to end. Build collaborative filtering and matrix factorisation on a ratings dataset, handle cold-start with content features, and evaluate with ranking metrics (NDCG, Precision@k, MAP), then discuss how you'd A/B test it. *What it proves:* the recommendation stack and ranking evaluation, the most common design-round topic.

P27. An object detection or segmentation project. Fine-tune YOLO for detection or U-Net for segmentation on a custom dataset, and report mAP or IoU. Deploy a demo that draws boxes/masks on uploaded images. *What it proves:* CV beyond classification, and the detection/segmentation vocabulary (IoU, NMS, mAP).

P28. An uplift / causal modelling project. On a dataset with a treatment and control (or a marketing dataset), build an uplift model to find "persuadables" rather than just likely-responders, and evaluate with a Qini curve. *What it proves:* the causal-vs-predictive distinction (scenario S28), which impresses because most candidates conflate them.

P29. An anomaly detection system for a real stream. Detect anomalies in server metrics, transactions, or sensor data with Isolation Forest and an autoencoder, tuning for the false-alarm rate a real ops team would tolerate. *What it proves:* the near-no-labels framing and the unsupervised-to-supervised path (manufacturing/fraud scenarios).

P30. A speech or audio project. Build audio classification (speech commands, genre, or environmental sounds) using spectrograms and a CNN, or use a pretrained model like Whisper for transcription and build something on top. *What it proves:* multi-modal breadth and that CNNs generalise beyond photos (spectrograms are images).

P31. A multi-modal application. Combine vision and text: image captioning, visual question answering, or a CLIP-based image search where users query images with text. *What it proves:* multi-modal understanding, increasingly central as frontier models go multi-modal.

P32. A graph neural network project. Node classification (fraud in a transaction graph) or link prediction (friend/product recommendation) with a GCN or GraphSAGE, discussing over-smoothing. *What it proves:* GNNs, a family most candidates have never touched, which differentiates you.

P33. A reinforcement learning project. Train an agent on a classic control task (CartPole, LunarLander) with DQN and then PPO, and explain the exploration-exploitation trade-off and why PPO is stable. *What it proves:* RL fundamentals, which connect to RLHF and are rare in portfolios.

P34. A fairness and explainability audit. Take a model on a sensitive dataset (lending, hiring) and produce a full audit: per-group performance, fairness metrics (showing they conflict), SHAP explanations, and a model card documenting limitations. *What it proves:* responsible-AI skills that frontier labs and regulated industries explicitly test (scenarios S21, S26).

P35. A model-compression / efficiency project. Take a trained model and apply quantisation, pruning, and knowledge distillation, measuring the size/latency/accuracy trade-off at each step. *What it proves:* the efficiency and serving knowledge that top labs probe, and awareness that production is about latency and cost, not just accuracy.

23.9 Tier 6 — Frontier / LLM-engineering projects (highest signal for top labs)

The projects most likely to impress OpenAI/Anthropic/DeepMind-style interviewers, because they touch the systems and safety concerns those roles centre on. Depth on one or two beats shallow coverage of all.

P36. A prompt-injection and jailbreak red-team suite. Build a systematic test set of adversarial inputs (prompt injections hidden in documents, jailbreak attempts) against an LLM or agent, measure the attack success rate, and implement and evaluate defences. *What it proves:* the safety and adversarial-robustness mindset frontier labs are built around (scenarios S19, S40).

P37. An LLM evaluation harness with an LLM judge. Build a reusable eval framework: a rubric-based LLM-as-judge (G-Eval style), calibrated against human labels, with measured and mitigated judge biases (position, verbosity, self-preference), packaged so any prompt/model change is gated by it. *What it proves:* modern evaluation methodology, now its own interview topic, and the rigour to ship generative systems safely.

P38. A reasoning / test-time-compute experiment. Compare a model answering directly vs with chain-of-thought vs with self-consistency (sample many reasoning paths and vote) vs a reflection loop, on a reasoning benchmark, measuring the accuracy-vs-compute trade-off. *What it proves:* understanding of test-time scaling and reasoning systems, a hot frontier-lab topic.

P39. A distributed / large-scale training demo. Train a model with data-parallel and FSDP/mixed-precision on multiple GPUs (cloud or Colab), measuring throughput and memory, and write up which parallelism to use when. *What it proves:* the training-infrastructure knowledge DeepMind/OpenAI system-design rounds probe, which almost no candidate has hands-on.

P40. A high-throughput inference server. Serve an open-weight model with vLLM/TGI using continuous batching, benchmark tokens/second and p50/p99 latency under concurrent load, and experiment with quantisation and KV-cache behaviour. *What it proves:* inference optimisation and serving internals (scenario S10, the single-GPU batching design), a strong differentiator.

P41. A guarded, evaluated multi-agent system. Build a supervisor-plus-specialists agent system with tool use, human-in-the-loop approval, recursion limits, injection defences, and a trajectory-level eval (task success, tool-call correctness). *What it proves:* agentic architecture plus the guardrail and evaluation discipline that separates a demo from a shippable system.

P42. A domain-specialised assistant (RAG + light fine-tune + eval), deployed. Pick a real domain (legal, medical, code), build a RAG assistant with citations, optionally LoRA-fine-tune for the domain's format, evaluate faithfulness and correctness against a held-out set, add safety guardrails for the domain's high-risk cases, and deploy it full-stack. *What it proves:* essentially the whole modern stack applied to a real, high-stakes domain, and the judgment to handle its safety implications, arguably the single best portfolio piece you can build.

> **PICKING YOUR SET** You cannot build all 42. A strong, hireable portfolio is roughly: two or three Tier 1–2 projects (proving classical and deep-learning rigour), three or four Tier 4 understanding builds (for the from-experience answers), one or two Tier 5 projects in your target domain, and one or two deployed Tier 3/6 projects plus the capstone (proving you ship modern systems). Depth and a deployed demo with an honest post-mortem beat a long list of notebooks every time.

Part XXV — The Complete Per-Concept Question Index

Parts XIX and XXII gave you the most-asked questions and the applied scenarios. This part guarantees **coverage**: for every concept explained anywhere in this guide, there is at least one direct question here, organised to mirror the guide's structure. Where a full answer lives earlier, the section reference is given so you can revise it. Treat this as a checklist: if you cannot answer any question below in two or three clear sentences, return to the referenced section.

How to drill this part. Cover the answers. Go concept by concept. For each question, say your answer aloud, then check. The goal is not recognition ("I've seen this") but fluent production under pressure. Mark anything shaky and revisit its section.

25.1 Mathematical foundations (Part I)

- **What is the shape of the product of a (m×n) and an (n×p) matrix, and why does matrix multiplication run deep learning?** (§1.1: (m×p); a layer is a matrix multiply, a network is a chain of them.)
- **Explain L1 vs L2 norms and why L1 induces sparsity.** (§1.1: diamond vs circle constraint; corners land on axes.)
- **What are eigenvectors and eigenvalues, and where do they appear in ML?** (§1.1: $Av=\lambda v$; PCA uses covariance eigenvectors.)
- **What is SVD and what is it used for?** (§1.1: $A=U\Sigma V^T$; best low-rank approximation; PCA, LSA, recommenders.)
- **What does the gradient represent and why do we step against it?** (§1.2: steepest-increase direction; negate to minimise.)
- **State and explain the chain rule's role in ML.** (§1.2: it is backpropagation.)
- **Batch vs stochastic vs mini-batch gradient descent, trade-offs.** (§1.2.)
- **Why don't we use second-order (Newton) methods in deep learning?** (§1.2: the Hessian is parameters^2, far too large.)
- **State Bayes' theorem and name every term.** (§1.3: posterior = likelihood × prior / evidence.)
- **Explain the low-base-rate testing paradox.** (§1.3: even accurate tests yield mostly false positives when positives are rare.)
- **Name the distributions a Bernoulli, binomial, Poisson, and normal describe.** (§1.3.)
- **State the Central Limit Theorem and why it matters.** (§1.3.)
- **What is MLE, and which losses are negative log-likelihoods?** (§1.3: MSE under Gaussian noise, cross-entropy under categorical.)
- **What is MAP, and how does it relate to regularisation?** (§1.3: L2 = Gaussian prior, L1 = Laplace prior.)
- **What exactly is a p-value, and what is it not?** (§1.4.)
- **Type I vs Type II error, and their link to the confusion matrix.** (§1.4: FP vs FN.)
- **What is statistical power?** (§1.4: $1-\beta$.)
- **When would you use a chi-square test vs a t-test vs ANOVA?** (§1.4.)
- **What is Simpson's paradox?** (§1.4.)
- **What is the multiple-comparisons problem and how do you correct for it?** (§1.4: Bonferroni, Benjamini-Hochberg.)
- **Define entropy, cross-entropy, and KL divergence, and relate them.** (§1.5: minimising cross-entropy = minimising KL.)
- **Why is KL divergence asymmetric, and where does that matter?** (§1.5: VAEs, RLHF penalty.)
- **What is mutual information used for?** (§1.5: non-linear feature selection.)
- **Convex vs non-convex optimisation, and why are neural nets fine despite being non-convex?** (§1.6: high-D critical points are mostly saddles.)

25.2 ML fundamentals and data (Parts II–III)

- **Give Mitchell's definition of learning (task, experience, performance).** (§2.1.)
- **Distinguish AI, ML, deep learning, and data science.** (§2.1.)
- **When would you NOT use machine learning?** (§2.1.)
- **Compare supervised, unsupervised, semi-supervised, self-supervised, and reinforcement learning.** (§2.2.)
- **What is self-supervised learning and why did it enable foundation models?** (§2.2.)
- **Parametric vs non-parametric models, with examples.** (§2.3.)
- **Discriminative vs generative models; is an LLM which?** (§2.4: generative.)
- **Walk through the end-to-end ML workflow.** (§2.5.)
- **One-hot vs ordinal vs target vs label encoding, and the danger of each.** (§3.1.)
- **How do you encode cyclical features like hour-of-day?** (§3.1: sine/cosine.)
- **Explain MCAR, MAR, MNAR and how the mechanism changes your imputation.** (§3.2.)
- **Why add a "was missing" indicator column?** (§3.2.)
- **How do you detect and handle outliers, and when is an outlier the signal?** (§3.3.)
- **Which models need feature scaling and which don't, and why?** (§3.4: trees are scale-invariant; distance/gradient/variance methods aren't.)
- **Name five feature-engineering techniques.** (§3.5.)
- **Filter vs wrapper vs embedded feature selection.** (§3.5.)
- **Why does random k-fold fail on time series, and what do you use instead?** (§3.6: forward-chaining split.)
- **When is stratified k-fold or group k-fold mandatory?** (§3.6.)

- **What is nested cross-validation for?** (§3.6.)
- **Define data leakage and give four examples.** (§3.7.)
- **Why must SMOTE be applied inside the CV fold?** (§3.8.)
- **Give data-level, algorithm-level, and evaluation-level fixes for class imbalance.** (§3.8.)

25.3 Evaluation metrics (Part IV) — every metric gets a question

- **Draw the confusion matrix and label all four cells plus Type I/II.** (§4.1.)
- **Define accuracy and explain the accuracy paradox.** (§4.2.)
- **Define precision; when do you optimise for it?** (§4.2: FP-costly.)
- **Give three names for recall and its formula.** (§4.2: recall/sensitivity/TPR = TP/(TP+FN).)
- **Define specificity and relate it to FPR.** (§4.2: TNR; 1−specificity = FPR.)
- **What is NPV?** (§4.2.)
- **Why is precision affected by class prevalence but recall is not?** (§4.2: column vs row of the matrix.)
- **Define F1 and explain why the harmonic mean.** (§4.3.)
- **What is F-beta and when do you use β=2 vs β=0.5?** (§4.3.)
- **What is MCC and why might it be better than F1?** (§4.3: uses all four cells.)
- **What is Cohen's kappa used for?** (§4.3: annotator agreement.)
- **What is balanced accuracy?** (§4.3.)
- **Explain the ROC curve and the probabilistic interpretation of AUC.** (§4.4: P(random positive ranked above random negative).)
- **What is the PR curve, and when do you prefer PR-AUC to ROC-AUC?** (§4.4: imbalanced data.)
- **What is a random classifier's PR-AUC?** (§4.4: the positive prevalence.)
- **What is a lift / gain chart?** (§4.4.)
- **How do you choose a decision threshold?** (§4.5: F1, recall-floor, or cost-minimisation on validation.)
- **What is the Youden J statistic?** (§4.5.)
- **Micro vs macro vs weighted averaging; which hides a failing minority class?** (§4.6: weighted.)
- **How is multi-class ROC-AUC computed (OvR vs OvO)?** (§4.6.)
- **Name three multi-label metrics.** (§4.6: exact-match, Hamming loss, micro/macro F1.)
- **RMSE vs MAE, and which conditional statistic does each minimise?** (§4.7: mean vs median.)
- **When would you use RMSLE or MAPE, and what breaks MAPE?** (§4.7: y near zero.)
- **What does R^2 mean and can it be negative?** (§4.7: yes, worse than the mean.)
- **What is adjusted R^2 for?** (§4.7.)
- **Name the ranking metrics and what MRR and NDCG capture.** (§4.8.)
- **Name three clustering metrics for when you have no labels.** (§4.8: silhouette, Davies-Bouldin, Calinski-Harabasz.)
- **What is log loss and what is the Brier score?** (§4.8.)
- **What is perplexity?** (§4.8: exp of cross-entropy.)
- **What does it mean for a model to be calibrated, and how do you fix miscalibration?** (§4.9: Platt/isotonic/temperature scaling.)
- **Which common models are well vs poorly calibrated?** (§4.9: logistic good; SVM/RF/boosting/deep nets not.)

25.4 Classical supervised learning (Part V) — one question per model

- **State the five assumptions of linear regression and how to check each.** (§5.1.)
- **Normal equation vs gradient descent; when each?** (§5.1.)
- **What is multicollinearity, how do you detect it (VIF), and when does it matter?** (§5.1: inference, not prediction.)
- **Ridge vs Lasso vs Elastic Net, and the geometric reason L1 zeros coefficients.** (§5.2.)
- **Why must you scale features before regularising?** (§5.2.)
- **How does logistic regression work, and why cross-entropy not MSE?** (§5.3: non-convex + vanishing gradient.)
- **Derive or state the logistic-regression gradient.** (§5.3: $X^T(\sigma(Xw)-y)/n$.)
- **What does a logistic coefficient mean in terms of odds?** (§5.3: multiplies odds by e^w.)
- **Is logistic regression linear, and why can't it solve XOR?** (§5.3: linear in log-odds.)
- **How does softmax generalise logistic regression?** (§5.3.)
- **Why is the C parameter the inverse of regularisation strength?** (§5.3.)
- **How does k-NN work, why is it "lazy," and why must you scale?** (§5.4.)
- **How does k affect the bias-variance trade-off in k-NN?** (§5.4.)
- **Why is Naive Bayes "naive," and why does it work despite that?** (§5.5.)
- **When would you use Gaussian vs Multinomial vs Bernoulli NB?** (§5.5.)
- **What is Laplace smoothing and what problem does it solve?** (§5.5: zero-frequency.)
- **What is the SVM margin, and what are support vectors?** (§5.6.)

- **Explain the kernel trick and what the RBF kernel corresponds to.** (§5.6: infinite-dimensional space.)
- **What do C and gamma control in an RBF SVM?** (§5.6.)
- **Why don't SVMs scale to millions of rows?** (§5.6: $O(n^2–n^3)$.)
- **How does a decision tree choose splits (Gini vs entropy vs information gain)?** (§5.7.)
- **Name five hyperparameters that fight tree overfitting, and pre- vs post-pruning.** (§5.7.)
- **Why is a single deep tree high-variance?** (§5.7.)
- **Explain bagging and out-of-bag error.** (§5.8.)
- **How does random forest differ from plain bagging, and why does feature subsampling help?** (§5.8: decorrelation.)
- **Why prefer permutation importance over the built-in impurity importance?** (§5.8: impurity is biased and computed on train.)
- **Bagging vs boosting: what does each attack and what base learners suit each?** (§5.9.)
- **How does AdaBoost work?** (§5.9: reweight misclassified samples.)
- **Explain gradient boosting (fit trees to the loss gradient / residuals).** (§5.9.)
- **XGBoost vs LightGBM vs CatBoost: name each one's key innovation.** (§5.9.)
- **Why does boosting overfit with too many trees but bagging doesn't?** (§5.9.)
- **What order do you tune gradient-boosting hyperparameters in?** (§5.9.)
- **What is stacking, and why must the meta-model train on out-of-fold predictions?** (§5.10.)
- **Hard vs soft voting.** (§5.10.)
- **Given 50k rows and 40 mixed features, which model and why?** (§5.11: gradient boosting.)

25.5 Unsupervised learning (Part VI)

- **Walk through the K-Means algorithm and its objective (WCSS).** (§6.1.)
- **How do you choose k (elbow, silhouette, gap)?** (§6.1.)
- **What are K-Means' assumptions and failure modes?** (§6.1: spherical, equal-size clusters.)
- **What does k-means++ fix?** (§6.1: initialisation.)
- **Explain hierarchical clustering and the linkage types.** (§6.2.)
- **How does DBSCAN work (core/border/noise), and how does it compare to K-Means?** (§6.3.)
- **What does HDBSCAN fix over DBSCAN?** (§6.3: varying density.)
- **Explain GMMs and the EM algorithm.** (§6.4.)
- **How does GMM generalise K-Means?** (§6.4.)
- **How do you choose the number of GMM components?** (§6.4: BIC/AIC.)
- **Walk through PCA step by step.** (§6.5.)
- **How do you choose the number of principal components?** (§6.5.)
- **What are PCA's limitations?** (§6.5: linear, destroys interpretability, unsupervised.)
- **PCA vs LDA?** (§6.6: unsupervised max-variance vs supervised max-separation.)
- **Why must you never interpret t-SNE cluster sizes or distances, and why prefer UMAP?** (§6.6.)
- **Name four anomaly-detection methods and when to use each.** (§6.7.)
- **How does Isolation Forest work?** (§6.7.)
- **Define support, confidence, and lift; why is confidence alone misleading?** (§6.8.)

25.6 Theory (Part VII)

- **State the bias-variance decomposition and define each term.** (§7.1.)
- **What is irreducible error and why does 100% accuracy suggest leakage?** (§7.1.)
- **How do you diagnose overfitting vs underfitting from the train-val gap?** (§7.1.)
- **What is double descent?** (§7.1.)
- **List regularisation methods for deep networks.** (§7.2.)
- **Explain the curse of dimensionality and its consequences.** (§7.3.)
- **State the No Free Lunch theorem and its practical implication.** (§7.4.)
- **Parameters vs hyperparameters.** (§7.5.)
- **Why does random search usually beat grid search?** (§7.5.)
- **What is Bayesian optimisation for hyperparameters?** (§7.5.)
- **How do you read a learning curve to decide if more data will help?** (§7.6.)
- **99% train accuracy, 70% validation: what do you do, in order?** (§7.6.)

25.7 Deep learning foundations (Part VIII)

- **What could and couldn't a single perceptron learn, and why did that cause an AI winter?** (§8.1: XOR.)

- **What is an MLP, and state the Universal Approximation Theorem; why go deep if one layer suffices?** (§8.2.)
- **Design the output layer (neurons, activation, loss) for regression, binary, multi-class, and multi-label.** (§8.2.)
- **Why do we need activation functions at all?** (§8.3.)
- **Compare sigmoid, tanh, ReLU, Leaky ReLU, and GELU.** (§8.3.)
- **Explain the vanishing gradient problem and how ReLU helps.** (§8.3.)
- **What is the dying ReLU problem and how do you avoid it?** (§8.3.)
- **Which loss for which task, and why feed logits (not softmax) to CrossEntropyLoss?** (§8.4.)
- **Explain backpropagation and the four steps.** (§8.5.)
- **Compare SGD, momentum, RMSProp, and Adam; what does Adam adapt?** (§8.6.)
- **Adam vs AdamW: what's the difference and why does it matter for transformers?** (§8.6: decoupled weight decay.)
- **Why is weight initialisation critical, and He vs Xavier?** (§8.7.)
- **What does batch normalisation do, and how does it behave differently at train vs test time?** (§8.8.)
- **Batch norm vs layer norm, and why do transformers use layer norm?** (§8.8.)
- **Explain dropout and why validation loss can be below training loss.** (§8.9.)
- **What is label smoothing / gradient clipping for?** (§8.9.)
- **What are the top three PyTorch training-loop bugs?** (§8.10: zero_grad, eval, no_grad.)

25.8 CNNs (Part IX)

- **Why convolution over dense layers for images (three reasons)?** (§9.1.)
- **Explain the convolution operation, and define kernel, stride, padding, channels.** (§9.2.)
- **Compute a conv layer's output size and parameter count.** (§9.2.)
- **What is pooling for, and max vs average vs global pooling?** (§9.3.)
- **Name the key innovation of AlexNet, VGG, Inception, and ResNet.** (§9.4.)
- **What problem do residual connections solve and how?** (§9.4.)
- **How does transfer learning work for vision (feature extraction vs fine-tuning)?** (§9.5.)
- **Why use a tiny learning rate when fine-tuning?** (§9.5.)
- **Classification vs detection vs segmentation; define IoU, NMS, mAP.** (§9.6.)
- **One-stage vs two-stage detectors; what did YOLO and focal loss contribute?** (§9.6.)
- **Semantic vs instance segmentation; what is U-Net?** (§9.7.)
- **CNN vs Vision Transformer: inductive biases and data efficiency.** (§9.8.)
- **What property does parameter sharing give a CNN?** (§9.8: translation equivariance.)

25.9 RNNs and sequences (Part X)

- **What is an RNN and what does the hidden state represent?** (§10.1.)
- **Name the RNN input/output configurations (one-to-many, etc.).** (§10.1.)
- **What is BPTT, and why do vanilla RNNs suffer vanishing/exploding gradients?** (§10.2.)
- **How is exploding gradient patched?** (§10.2: clipping.)
- **Explain the LSTM's three gates and cell state.** (§10.3.)
- **Why does the additive cell-state update solve the vanishing gradient?** (§10.3.)
- **How does a GRU differ from an LSTM, and when use each?** (§10.4.)
- **What is a bidirectional RNN, and when can't you use one?** (§10.5: real-time generation.)
- **Explain the seq2seq encoder-decoder and its bottleneck.** (§10.6.)
- **What is attention (pre-transformer) and what problem did it solve?** (§10.7.)
- **Why did transformers replace RNNs (three reasons)?** (§10.7.)

25.10 Transformers and LLMs (Part XI)

- **Explain self-attention via query, key, value.** (§11.1.)
- **State the scaled dot-product attention formula.** (§11.1.)
- **Why divide by √d_k?** (§11.1: variance control for softmax.)
- **What is multi-head attention for?** (§11.2.)
- **Why do transformers need positional encoding, and what happens without it?** (§11.3.)
- **Sinusoidal vs learned vs rotary positional encodings.** (§11.3.)
- **Describe the full transformer block; which earlier ideas reappear?** (§11.4: residuals, layer norm.)
- **Encoder-only vs decoder-only vs encoder-decoder; give a model for each.** (§11.4.)
- **BERT vs GPT vs T5: architecture and training objective.** (§11.5.)

- **Explain the causal mask and why it makes autoregressive training valid.** (§11.5.)
- **What is subword tokenisation; compare BPE, WordPiece, and Unigram.** (§11.6.)
- **Describe the LLM training pipeline: pretraining, SFT, preference alignment.** (§11.7.)
- **Explain RLHF (reward model, PPO, KL penalty) and how DPO differs.** (§11.7.)
- **RAG vs fine-tuning vs prompting: when each?** (§11.7 and §22.9.)
- **What is LoRA, why does it work, and what does QLoRA add?** (§11.8.)
- **Name the main LLM inference optimisations (KV cache, quantisation, Flash Attention, batching, speculative decoding, MoE).** (§11.9.)
- **Walk through what happens mechanically when an LLM generates one token.** (§11.9.)

25.11 Generative, NLP, RL, time series, recommenders, responsible AI (Parts XII–XVII)

- **What is an autoencoder, and why is a plain one not generative?** (§12.1.)
- **How does a VAE make the latent space samplable, and what is the reparameterisation trick?** (§12.2.)
- **Explain GANs and the minimax game; what is mode collapse and its fix?** (§12.3.)
- **How do diffusion models work (forward/reverse/conditioning), and what is latent diffusion?** (§12.4.)
- **GAN vs VAE vs diffusion: sample quality, training stability, speed.** (§12.6.)
- **Stemming vs lemmatisation, and why skip them for transformers?** (§13.1.)
- **What is TF-IDF, and how does it relate to BM25 and hybrid search?** (§13.2.)
- **Word2Vec (CBOW vs skip-gram), GloVe, FastText; static vs contextual embeddings.** (§13.3–13.4.)
- **Frame an RL problem as an MDP; define state, action, reward, policy, discount, value, Q.** (§14.1.)
- **Explain the exploration-exploitation trade-off and ε-greedy.** (§14.2.)
- **Q-learning and DQN; what do experience replay and the target network fix?** (§14.3.)
- **Policy gradients and PPO; why is PPO stable?** (§14.4.)
- **How is RLHF an RL problem?** (§14.5.)
- **Time-series components and stationarity; how do you test and achieve it?** (§15.1.)
- **Explain ARIMA(p,d,q) and how you choose the orders.** (§15.2.)
- **Why is time-series validation different, and how do you avoid leakage?** (§15.4.)
- **Collaborative vs content-based filtering, and the cold-start problem.** (§16.1, §16.3.)
- **What is matrix factorisation and how does it relate to SVD?** (§16.2.)
- **Why do offline recommender metrics need an online A/B test?** (§16.5.)
- **Global vs local explanations; what are SHAP and LIME?** (§17.1–17.2.)
- **Name three fairness metrics and explain why they conflict.** (§17.3.)
- **What are differential privacy and federated learning?** (§17.4.)
- **What is a model card?** (§17.5.)

25.12 Frontier topics and production (Parts XVIII, XXI)

- **Walk the ML system design framework.** (§18.1.)
- **Explain the two-stage retrieval-then-rank pattern and where it appears.** (§18.3.)
- **What is training-serving skew and what prevents it?** (§18.3: feature store.)
- **Greedy vs beam search vs top-k vs top-p decoding; why is beam search bad for open-ended text?** (§21.1.)
- **What are scaling laws and what did Chinchilla show?** (§21.2: ~20 tokens/param compute-optimal.)
- **What are emergent abilities and test-time scaling?** (§21.2.)
- **What is Multi-Query / Grouped-Query Attention for?** (§21.3: KV-cache size.)
- **What is a Mixture of Experts and its trade-off?** (§21.3.)
- **What is the KV cache and why is it the main inference memory cost?** (§21.3, §11.9.)
- **Why are BLEU and ROUGE insufficient for evaluating generative systems?** (§21.4.)
- **What is LLM-as-a-judge, and what biases must you mitigate?** (§21.4.)
- **How do you evaluate a RAG system (retrieval vs generation metrics)?** (§21.4.)
- **How do you evaluate an agent?** (§21.4: task success, tool-call correctness.)
- **What is message passing in a GNN; name GCN, GraphSAGE, GAT; what is over-smoothing?** (§21.5.)
- **Distinguish data, tensor, and pipeline parallelism, and FSDP/ZeRO.** (§21.6.)
- **Data drift vs concept drift vs label drift; how do you detect each?** (§21.7.)

Part XXVI — LLM and GenAI System Design

Modern AI/ML interviews, especially at frontier labs and any company shipping LLM features, include a design round specifically about **generative** systems. The classic ML-system-design framework (Part XVIII) still applies, but with new components (retrieval, prompts, guardrails, evals) and new failure modes. This part gives the framework and worked designs, complementing the RAG and agentic guides.

26.1 The GenAI design framework

Extend the Part XVIII framework with these LLM-specific steps:

1. **Clarify the task and the failure cost.** Is it Q&A, summarisation, extraction, an agent taking actions? How bad is a wrong or harmful answer? This sets how conservative the design must be.
2. **Decide the knowledge strategy.** Prompting, RAG, fine-tuning, or a combination (§22.9's decision). Facts that change or need citations → RAG; behaviour/format → fine-tune; small/simple → prompt.
3. **Choose the model.** Hosted API vs open-weight self-hosted (cost, latency, data residency, control), and size vs capability. Start with the smallest model that passes evals.
4. **Design retrieval (if RAG).** Chunking, embedding model, vector store, hybrid search, reranking, top-k. Retrieval quality dominates RAG quality.
5. **Design the prompt and output contract.** System prompt, few-shot examples, structured output, "answer only from context, else say you don't know."
6. **Guardrails.** Input filtering (prompt injection), output filtering (PII, safety), refusal/deferral in high-risk cases, human-in-the-loop for consequential actions, rate/spend limits.
7. **Evaluation.** An offline eval set with an LLM judge, safety/red-team evals, RAG component evals, agent trajectory evals, gated before launch and run as a regression suite.
8. **Serving and cost.** Latency budget, batching, KV cache, quantisation, caching frequent queries, the retrieve-many-rerank-few cost lever.
9. **Monitoring and iteration.** Track retrieval quality, answer quality, hallucination rate, cost, and latency in production; A/B test changes; watch for data drift in the corpus and query distribution.

> **KEY IDEA** The through-line of every GenAI design answer: generative systems fail differently from classifiers. They hallucinate, they can be adversarially manipulated, they have no built-in "I don't know," and their quality drifts as data changes. So a strong design is not just "call the model" but "ground it, guard it, evaluate it continuously, and make deferral the safe default in high-risk cases." Saying that framing explicitly is what separates a senior GenAI answer from a naive one.

26.2 Worked GenAI designs

G1. A customer-support assistant over company documentation. RAG with citations (the first guide's core system, designed for production). Ingestion pipeline chunks and embeds docs into a vector DB, refreshed as docs change. Query path: embed, retrieve top-k (4–8), rerank, augment a prompt that requires grounding and permits "I don't know," generate with citations. Add query rewriting for follow-ups and hybrid search for exact terms (error codes, product names). Guardrails: refuse or escalate on out-of-scope or high-risk questions. Eval: retrieval hit rate/MRR and context precision, generation faithfulness and answer relevance (RAGAS), gated before launch; online, resolution rate and escalation rate. Monitor retrieval quality because RAG rots through corpus drift, not code changes.

G2. A code assistant / copilot. Frame as retrieval-augmented generation over a codebase plus a strong base (or fine-tuned) code model. Retrieve relevant files/functions (embeddings over code, plus symbol/AST-aware retrieval) into context. Challenges: large context (whole files), correctness (the output must run, so add execution/lint feedback loops), and latency (developers expect fast completions). Eval: pass@k on held-out tasks (does generated code pass tests), plus human preference. Guardrails: never leak secrets from the codebase, sandbox any execution. Serving: aggressive caching and low-latency inference.

G3. A document-processing / extraction pipeline (invoices, contracts, forms). Frame as structured extraction. For each document: OCR if scanned, then an LLM (or a fine-tuned extractor) with a strict output schema (JSON) and validation. Because hallucinated fields are dangerous, require the model to ground each field in the source and flag low-confidence extractions for human review. Eval: field-level precision/recall against labelled documents (this is where classic metrics still apply). Design for the long tail of document formats and for confidence-based routing to humans. This mirrors the standard-library extraction task reported at Anthropic.

G4. A conversational agent that can take actions (book, buy, email, update records). The high-stakes agentic design (the agentic guide's territory). A LangGraph-style agent with tools, but the design centres on **guardrails** because actions are consequential and irreversible: human-in-the-loop approval before any consequential action, hard recursion/spend limits, tool allow-lists, argument validation, dry-run previews, full trace logging, and prompt-injection defences (a retrieved document must not be able to command the agent). Eval: trajectory-level task success and tool-call correctness on a test set including adversarial trajectories. The framing: the more powerful the action, the more the system must require explicit confirmation and constrain what is even possible.

G5. A content-generation system at scale (marketing copy, personalised messages). Frame as generation with brand-safety and quality controls. Prompt (and optionally fine-tune) for brand voice; retrieve product facts to avoid hallucinated claims; enforce structure and length. Guardrails: a safety/brand filter on every output, a factuality check for claims, and human review for high-visibility content. Eval: an LLM judge on brand-fit and correctness calibrated to human ratings, plus A/B tests on the business metric (engagement, conversion). Cost matters at scale, so cache and batch. The key risk to name is hallucinated claims about the product, which grounding and a factuality check address.

G6. An evaluation / grading system (e.g. scoring essays or model outputs). Frame as LLM-as-a-judge with rigour. A rubric-based judge (G-Eval style) that reasons before scoring, calibrated against a human-labelled set, with measured and mitigated biases (position, verbosity, self-preference, leniency). For high-stakes grading, use multiple judges or judge-plus-human. Eval of the evaluator: correlation with human scores, and bias audits. The insight that impresses: an LLM judge is itself a model that must be validated and monitored, not trusted blindly, and its biases are known and correctable.

> **THE FRAMING THAT WINS GENAI DESIGN ROUNDS** Always answer in this order: clarify the task and the cost of a bad output; choose the knowledge strategy (prompt/RAG/fine-tune) and justify it; design retrieval and the prompt contract; then spend real time on guardrails and evaluation, because that is where generative systems are won or lost and where junior answers fall silent. Close with serving cost and production monitoring. An interviewer wants to hear that you treat an LLM feature as a system to be grounded, guarded, and continuously evaluated, not a single API call.

Part XXVII — Reference

27.1 The one-line cheat sheet

Concept	The one-line answer
Bias-variance	Underfit = high bias; overfit = high variance; watch the train-val gap
L1 vs L2	L1 (diamond) zeros coefficients (selection); L2 (circle) shrinks them
Precision	When it says yes, is it right: TP/(TP+FP)
Recall / Sensitivity / TPR	Did it catch them all: TP/(TP+FN)
Specificity / TNR	Did it clear the negatives: TN/(TN+FP)
F1	Harmonic mean of precision and recall; high only if both are
AUC	P(random positive ranked above random negative)
PR-AUC	Use this, not ROC-AUC, on imbalanced data
Type I / II	FP / FN
Bagging	Parallel, averages, cuts variance (deep trees)
Boosting	Sequential, corrects errors, cuts bias (shallow trees)
Random Forest	Bagging + feature subsampling to decorrelate trees
Gradient boosting	Each tree fits the previous ensemble's residuals
Kernel trick	Inner products in high-D space without building it
SVM margin	Widest street; only support vectors matter
Naive Bayes	Bayes + conditional independence; works despite being wrong
K-Means	Assign to nearest centroid, update to mean; spherical clusters
DBSCAN	Density clusters, any shape, labels noise
PCA	Max-variance orthogonal directions (eigenvectors of covariance)
Curse of dimensionality	Distances converge, space is sparse
Activation function	Where all non-linearity/power comes from
Vanishing gradient	Small derivatives multiply toward zero; fix with ReLU/norm/residual/LSTM
Backprop	Chain rule from output to input
Adam	Momentum + per-parameter adaptive rate
BatchNorm	Stabilises training; differs train vs eval
Dropout	Randomly zero activations; off at test time
CNN	Local connectivity + parameter sharing + hierarchy
Residual connection	Identity shortcut so gradients survive depth
RNN	Shared weights across time; a hidden-state memory
LSTM	Additive gated cell state defeats vanishing gradients
GRU	Lighter LSTM: two gates, one state
Self-attention	$\text{softmax}(QK^T/\sqrt{d\_k})V$: every token attends to all others
Multi-head	Attend to several subspaces at once
Positional encoding	Injects word order into permutation-invariant attention
BERT vs GPT	Encoder/masked-LM vs decoder/next-token
RAG vs fine-tuning	Facts at inference vs behaviour in the weights
LoRA	Low-rank weight updates; train <1% of parameters
RLHF	Reward model + PPO + KL penalty to align to preferences
GAN vs VAE vs Diffusion	Sharp/unstable vs stable/blurry vs best/slow
Diffusion	Learn to reverse gradual noising
Cross-validation	Rotate held-out folds; stratify for classes, time-split for series
Data leakage	Train-time info absent at prediction; the 0.99-AUC red flag
Imbalance	Fix metric + class weights + resample in-fold + threshold
Calibration	Predicted probabilities match observed frequencies

27.2 The formula sheet

```
Linear regression       ŷ = Xw ;   w = (XᵀX)⁻¹Xᵀy
MSE                     (1/n) Σ(y - ŷ)²
Sigmoid                 σ(z) = 1/(1 + e⁻ᶻ)
Softmax                 eᶻⁱ / Σ eᶻ
Binary cross-entropy    -(1/n) Σ [y log p + (1-y) log(1-p)]
Logistic gradient       (1/n) Xᵀ(σ(Xw) - y)
Ridge / Lasso penalty   λΣwⱼ²   /   λΣ|wⱼ|
Bayes                   P(A|B) = P(B|A)P(A)/P(B)
Entropy                 -Σ p log p
Cross-entropy           -Σ p log q
KL divergence           Σ p log(p/q) = H(p,q) - H(p)
Gini impurity           1 - Σ pᵢ²
Cosine similarity       (a·b)/(‖a‖‖b‖)
Precision / Recall      TP/(TP+FP) ;  TP/(TP+FN)
Specificity / FPR       TN/(TN+FP) ;  FP/(FP+TN)
F1                      2PR/(P+R)
F-beta                  (1+β²)PR/(β²P+R)
MCC                     (TP·TN - FP·FN)/√((TP+FP)(TP+FN)(TN+FP)(TN+FN))
R²                      1 - SS_res/SS_tot
Conv output size        floor((W - K + 2P)/S) + 1
Attention               softmax(QKᵀ/√d_k) V
LSTM cell               Cₜ = fₜ⊙Cₜ₋₁ + iₜ⊙C̃ₜ ;  hₜ = oₜ⊙tanh(Cₜ)
Adam update             w -= η · m̂/(√v̂ + ε)
Bias-variance           E[(y-f̂)²] = Bias² + Var + σ²
TF-IDF                  TF(t,d) · log(N/DF(t))
```

27.3 Glossary (every term in this guide)

Activation function — the non-linear function applied to a neuron's output; the source of a network's expressive power. **AdamW** — Adam optimiser with decoupled weight decay; the transformer standard. **Anomaly detection** — flagging points that deviate from normal patterns. **ANN (architecture)** — Artificial Neural Network; a feedforward multi-layer perceptron. **ANN (search)** — Approximate Nearest Neighbour; fast, slightly-inexact vector search. **Attention** — mechanism letting a model weight the relevance of other positions. **AUC** — area under the ROC curve; probability a positive is ranked above a negative. **Autoencoder** — encoder-decoder network trained to reconstruct its input. **Backpropagation** — the chain rule applied backward through a network to get gradients. **Bagging** — training models on bootstrap samples and averaging; reduces variance. **Batch normalisation** — normalising layer inputs over the batch to stabilise training. **Bias (statistical)** — error from overly simple assumptions; causes underfitting. **BERT** — bidirectional encoder transformer trained with masked language modelling. **Boosting** — sequential ensembling where each model corrects prior errors; reduces bias. **BPE** — Byte-Pair Encoding subword tokenisation. **BPTT** — Backpropagation Through Time, for training RNNs. **Calibration** — agreement between predicted probabilities and observed frequencies. **CatBoost** — gradient boosting with ordered boosting and native categorical handling. **CLIP** — a model aligning image and text embeddings; used to condition diffusion. **CNN** — Convolutional Neural Network; uses filters, parameter sharing, and pooling. **Confusion matrix** — the 2x2 table of TP, TN, FP, FN. **Convolution** — sliding a learnable filter over an input to produce a feature map. **Cross-entropy** — the classification loss; negative log-likelihood of a categorical. **Cross-validation** — rotating held-out folds to estimate generalisation. **Curse of dimensionality** — the breakdown of distance and density in high dimensions. **DBSCAN** — density-based clustering that finds arbitrary shapes and labels noise. **Decision tree** — recursive if/else splits chosen to reduce impurity. **Diffusion model** — generates by learning to reverse gradual noising. **Discriminative model** — models P(y|x), the decision boundary. **DPO** — Direct Preference Optimisation; preference alignment without an RL loop. **Dropout** — randomly zeroing activations during training to prevent overfitting. **Early stopping** — halting training when validation performance stops improving. **Elastic Net** — combined L1 and L2 regularisation. **Embedding** — a dense vector representing meaning; similar items land nearby. **Ensemble** — combining multiple models for better performance. **Entropy** — a measure of uncertainty. **Epoch** — one full pass over the training data. **Exploding gradient** — gradients growing without bound; fixed by clipping. **F1 score** — harmonic mean of precision and recall. **Feature engineering** — creating informative inputs from raw data. **Fine-tuning** — updating a pretrained model's weights on new data. **GAN** — Generative Adversarial Network; generator vs discriminator. **Generative model** — models the data distribution and can produce new samples. **GELU** — a smooth activation function; the transformer default. **Gini impurity** — a decision-tree split criterion. **Gradient** — the vector of partial derivatives; points toward steepest increase. **Gradient boosting** — boosting that fits each model to the loss gradient (residuals). **Gradient descent** — iteratively stepping against the gradient to minimise loss. **GRU** — Gated Recurrent Unit; a lighter LSTM with two gates. **Hallucination** — a model confidently producing false output. **He initialisation** — weight init scaled for ReLU networks. **HNSW** — the dominant approximate-nearest-neighbour graph index. **Hyperparameter** — a setting chosen before training (learning rate, depth). **Imbalanced data** — classes with very unequal frequencies. **Information gain** — the reduction in impurity from a split. **KL divergence** — the asymmetric "distance" between two distributions. **k-NN** — k-Nearest Neighbours; classify by the majority of nearest points. **K-Means** — partition data into k spherical clusters by centroid. **Kernel trick** — computing high-dimensional inner products implicitly. **Layer normalisation** — normalising over features per sample; the transformer choice. **Leakage** — training on information unavailable at prediction time. **LightGBM** — fast leaf-wise gradient boosting with histogram binning. **Logistic regression** — linear classifier through a sigmoid, trained with cross-entropy. **LoRA** — Low-Rank Adaptation; parameter-efficient fine-tuning. **LSTM** — Long Short-Term Memory; gated recurrent cell defeating vanishing gradients. **MAE** — mean absolute error; robust, minimised by the median. **Matrix factorisation** — decomposing an interaction matrix into latent factors. **MCC** — Matthews Correlation Coefficient; a balanced metric using all four cells. **MLE** — Maximum Likelihood Estimation. **MLP** — Multi-Layer Perceptron; a feedforward network. **Mode collapse** — a GAN producing limited diversity. **Momentum** — accumulating past gradients to accelerate descent. **Multi-head attention** — several parallel attention operations. **Naive Bayes** — Bayes classifier assuming conditional feature independence. **NDCG** — normalised discounted cumulative gain; a ranking metric. **Normalisation** — rescaling features to a common range or distribution. **One-hot encoding** — one binary column per category. **Optimiser** — the algorithm that applies gradients to weights. **Overfitting** — memorising training data; low train error, high validation error. **PCA** — Principal Component Analysis; project onto max-variance directions. **PEFT** — Parameter-Efficient Fine-Tuning. **Perceptron** — the single-neuron ancestor

of neural networks. **Perplexity** — the language-modelling metric; exp of cross-entropy. **Pooling** — downsampling a feature map. **Positional encoding** — injecting order into a transformer. **Precision** — TP/(TP+FP); correctness of positive predictions. **PR-AUC** — area under the precision-recall curve; preferred for imbalance. **Pretraining** — self-supervised training of a foundation model. **PPO** — Proximal Policy Optimisation; a stable policy-gradient RL method. **p-value** — probability of data this extreme under the null hypothesis. **Q-learning** — value-based reinforcement learning. **QLoRA** — LoRA with 4-bit quantisation of the frozen base. **Quantisation** — storing weights in lower precision. **RAG** — Retrieval-Augmented Generation. **Random Forest** — bagged decision trees with feature subsampling. **Recall** — TP/(TP+FN); also sensitivity, TPR. **Regularisation** — penalising complexity to reduce overfitting. **ReLU** — max(0,x); the standard activation. **Residual connection** — an identity shortcut aiding deep-network gradients. **RLHF** — Reinforcement Learning from Human Feedback. **RMSE** — root mean squared error; outlier-sensitive, minimised by the mean. **RNN** — Recurrent Neural Network; processes sequences with a hidden state. **ROC curve** — TPR vs FPR across thresholds. **Self-attention** — attention among positions within one sequence. **Self-supervised learning** — deriving labels from the data itself. **Sensitivity** — recall / true positive rate. **SGD** — Stochastic Gradient Descent (in practice, mini-batch). **SHAP** — game-theoretic feature attributions. **Sigmoid** — squashes to (0,1); used for binary output. **Silhouette score** — a clustering quality metric. **SMOTE** — synthetic minority oversampling. **Softmax** — turns logits into a probability distribution. **Specificity** — TN/(TN+FP); true negative rate. **Stacking** — training a meta-model on base-model predictions. **Stationarity** — constant statistical properties over time. **Supervised learning** — learning from labelled data. **SVD** — Singular Value Decomposition. **SVM** — Support Vector Machine; maximum-margin classifier. **Temperature** — randomness control in sampling. **Tensor** — a multi-dimensional array; can live on a GPU and track gradients. **TF-IDF** — term frequency times inverse document frequency. **Token** — a subword unit; the input granularity of an LLM. **Transfer learning** — reusing a pretrained model on a new task. **Transformer** — the attention-based architecture behind modern LLMs. **t-SNE** — a local-structure visualisation method (visualisation only). **Type I / II error** — false positive / false negative. **UMAP** — a faster, more global-structure-preserving alternative to t-SNE. **Underfitting** — model too simple; high error everywhere. **Universal Approximation** — one hidden layer can approximate any continuous function. **Unsupervised learning** — finding structure without labels. **VAE** — Variational Autoencoder; a generative autoencoder with a smooth latent space. **Vanishing gradient** — gradients shrinking toward zero in deep networks. **Variance (statistical)** — error from sensitivity to the training sample; overfitting. **Vision Transformer (ViT)** — a transformer applied to image patches. **Weight decay** — L2 regularisation of network weights. **Word2Vec** — static word embeddings from local context. **Xavier initialisation** — weight init scaled for tanh/sigmoid. **XGBoost** — regularised, second-order gradient boosting.

27.4 Documentation and paper references

Official documentation (the primary sources to actually read): - scikit-learn User Guide — https://scikit-learn.org/stable/user_guide.html — the best single ML reference; the user guide reads like a textbook. - PyTorch docs and tutorials — https://pytorch.org/docs and https://pytorch.org/tutorials - TensorFlow / Keras — https://www.tensorflow.org/learn and https://keras.io - XGBoost — https://xgboost.readthedocs.io ; LightGBM — https://lightgbm.readthedocs.io ; CatBoost — https://catboost.ai/docs - Hugging Face Transformers — https://huggingface.co/docs/transformers ; the free HF LLM and NLP courses — https://huggingface.co/learn - NumPy — https://numpy.org/doc ; pandas — https://pandas.pydata.org/docs - imbalanced-learn — https://imbalanced-learn.org ; SHAP — https://shap.readthedocs.io ; Optuna — https://optuna.readthedocs.io - statsmodels — https://www.statsmodels.org

Foundational papers (know the idea and the one-line contribution): - Rosenblatt (1958), the perceptron. - Rumelhart, Hinton, Williams (1986), backpropagation. - Hochreiter and Schmidhuber (1997), LSTM. - Breiman (2001), Random Forests. - Cortes and Vapnik (1995), Support Vector Networks. - Krizhevsky, Sutskever, Hinton (2012), AlexNet — "ImageNet Classification with Deep CNNs". - Goodfellow et al. (2014), Generative Adversarial Networks. - Kingma and Welling (2013), Auto-Encoding Variational Bayes (VAE). - Kingma and Ba (2014), Adam. - He et al. (2015), Deep Residual Learning (ResNet). - Ioffe and Szegedy (2015), Batch Normalization. - Vaswani et al. (2017), "Attention Is All You Need" (the Transformer). - Devlin et al. (2018), BERT. - Radford et al. (2018-2020), the GPT series. - Ho et al. (2020), Denoising Diffusion Probabilistic Models. - Rombach et al. (2022), Latent Diffusion (Stable Diffusion). - Hu et al. (2021), LoRA. - Ouyang et al. (2022), InstructGPT / RLHF. - Rafailov et al. (2023), Direct Preference Optimization (DPO). - Chen and Guestrin (2016), XGBoost. - Bergstra and Bengio (2012), Random Search for Hyper-Parameter Optimization. - Lundberg and Lee (2017), SHAP.

Books worth owning: - Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning* (free PDF online). The reference. - James et al., *An Introduction to Statistical Learning* (free PDF). The gentler version; start here. - Goodfellow, Bengio, Courville, *Deep Learning* (free online). The deep learning reference. - Bishop, *Pattern Recognition and Machine Learning*. The Bayesian classic. - Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. The best practical book. - Chip Huyen, *Designing Machine Learning Systems*. For the system-design and production rounds.

27.5 A 12-week study plan

Week	Focus	Deliverable
1	Part I (math) + Part II (ML basics)	Answer the Part I checkpoint from memory
2	Part III (data) + Part IV (metrics)	Implement all confusion-matrix metrics from scratch; answer the Part IV checkpoint aloud
3	Part V (5.1–5.6): linear, logistic, regularisation, k-NN, Naive Bayes, SVM	Code logistic regression and k-NN in NumPy
4	Part V (5.7–5.11): trees, RF, boosting, stacking	Train and tune XGBoost with early stopping on a real dataset; use a leak-free pipeline
5	Part VI (unsupervised) + Part VII (theory)	Implement K-Means and PCA from scratch; draw the bias-variance curve from memory
6	Part VIII (deep learning foundations)	Write the full PyTorch training loop from memory; hand-derive backprop for a 2-layer net
7	Part IX (CNNs)	Fine-tune a pretrained ResNet on a small image dataset
8	Part X (RNN/LSTM/GRU)	Explain the LSTM gates and vanishing gradient aloud; build a small text classifier
9	Part XI (transformers/LLMs)	Implement a self-attention head in NumPy; explain RAG vs fine-tuning aloud
10	Part XII (generative) + Part XIII (NLP) + Part XIV (RL)	Explain GAN vs VAE vs diffusion; explain RLHF in RL terms
11	Part XV–XVII (time series, recommenders, responsible AI) + Part XVIII (system design)	Do two mock system-design questions out loud, following the framework
12	Part XIX (all questions + coding) + Part XX (cheat sheets)	Answer every Q1–Q77 aloud without notes, recorded; re-do the from-scratch coding problems

THE FINAL WORD This guide gives you the concepts, the mathematics, the implementations, and the answers. It cannot give you the two things that actually win interviews: **fluency** and **stories**. Fluency comes from saying the answers out loud until they are automatic, which is why every checkpoint says "aloud". Stories come from building things and breaking them, which is why every part has code you should type and deliberately break. As the RAG guide said, and it is just as true here: interviewers do not want definitions, they can read the docs too. They want trade-offs and war stories. Build the projects. Break them. Then walk in and talk about what happened.

End of The Complete AI / ML Interview Mastery Guide. Companion to The Beginner's Guide to Building AI Applications and The Production & MLOps Companion. Between the three, you have the AI/ML foundations, the applied LLM/RAG/agent stack, and the production/MLOps layer: the full picture an international-level AI/ML interview probes.